MalChela User Guide



This Guide covers version 4.4.1

https://github.com/dwmetz/MalChela

Baker Street Forensics

https://bakerstreetforensics.com

Table of contents

1. About 4
2. Installation 6
3. Navigation 8
   - 3.1 Home & At a Glance 8
   - 3.2 Top Toolbar 9
4. Configuration 10
   - 4.1 tools.yaml 10
   - 4.2 API Configuration 12
   - 4.3 Offline Mode 14
5. 4.16 Case Management 16
   - 5.1 Creating a Case 16
   - 5.2 Case Contents 17
   - 5.3 Notes & Tagging 17
   - 5.4 Archiving Cases 18
   - 5.5 Importing Cases 18
   - 5.6 Summary 18
6. AI Integration & MCP Support 19
   - 6.1 Prerequisites 19
   - 6.2 Installation 19
   - 6.3 Available Tools 20
   - 6.4 Recommended Analysis Workflow 21
   - 6.5 Agentic Coding Environments 22
   - 6.6 Kali Linux / Remote Server Setup 22
   - 6.7 Troubleshooting 22
7. Core Tools 23
   - 7.1 Overview 23
   - 7.2 Usage 26
   - 7.3 Analyze 29
   - 7.4 CombineYARA 34
   - 7.5 ExtractSamples 35
   - 7.6 FileAnalyzer 36
   - 7.7 FileMiner 37
   - 7.8 HashCheck 39
   - 7.9 HashIt 41
   - 7.10 TiQuery 42

7.11 MStrings 47
7.12 MITRE Lookup Panel 49
7.13 MZCount 53
7.14 MZHash 55
7.15 NSRLQuery 57
7.16 StringsToYARA 59
7.17 Unpack 61
7.18 XMZHash 65
7.19 YARA Scan 67
8. Mac Analysis 70
8.1 Overview 70
8.2 Code Sign Check 71
8.3 dppExtract 74
8.4 Mach-O Info 76
8.5 Plist Analyzer 78
9. Third-Party Tools 80
9.1 Integrating Third-Party Tools 80
9.2 Configuration Reference 81
9.3 Enhanced Integrations 83
9.4 TShark 84
9.5 PCAP to CSV Conversion 85
9.6 Volatility 3 86
9.7 Installing and Configuring YARA-X 89
9.8 Python Integrations 91
10. REMnux Mode 93
10.1 How REMnux Mode Works 93
10.2 Preconfigured Tools in REMnux Mode 94
10.3 Benefits 95
10.4 Customizing the REMnux Experience 96
11. Support 97
11.1 Support & Contribution 97
11.2 Known Limitations & Platform Notes 97

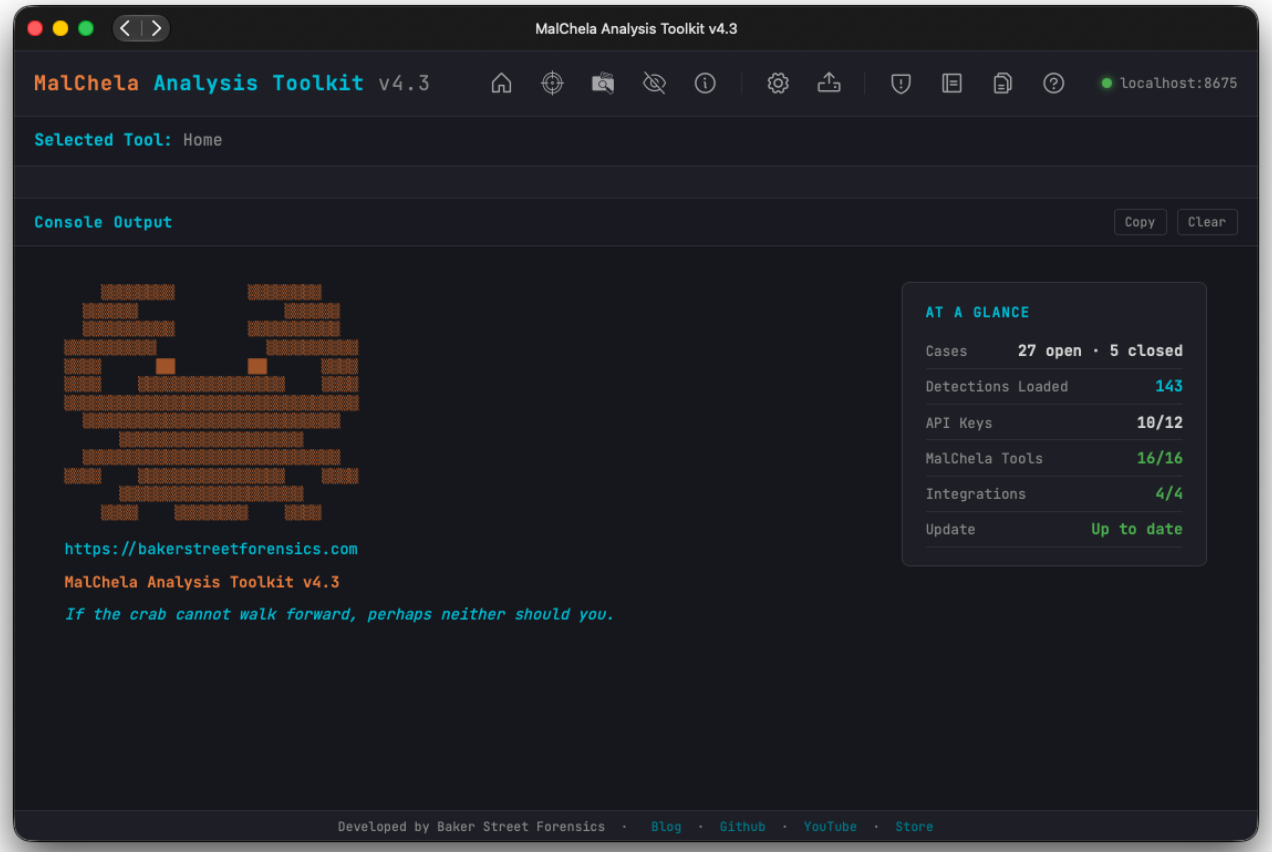
1. About

MalChela is a modular toolkit for digital forensic analysts, malware researchers, and threat intelligence teams. It provides both a Command Line Interface (CLI) and a browser-based web interface for running analysis tools in a unified environment.

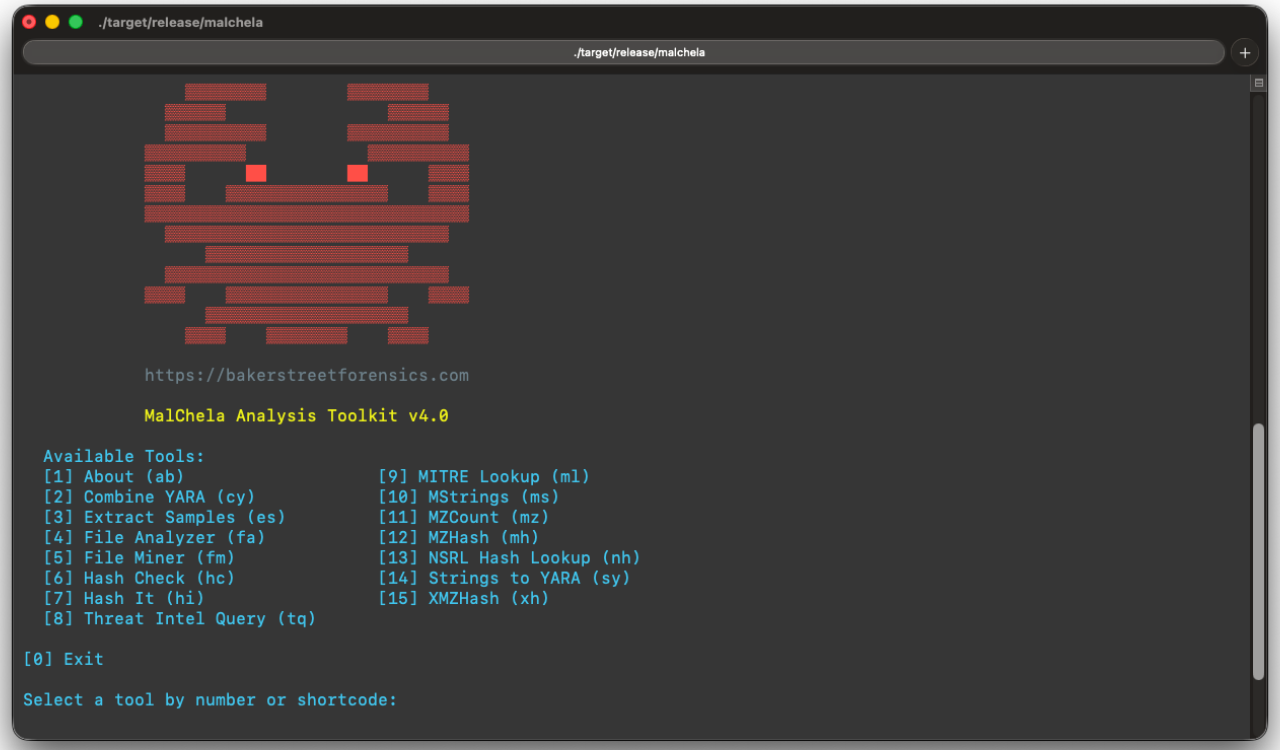
mal — malware

chela — "crab hand"

A chela on a crab is the scientific term for a claw or pincer. It's a specialized appendage, typically found on the first pair of legs, used for grasping, defense, and manipulating things; just like these programs.



MalChela Web Interface



MalChela CLI

2. Installation

2.0.1 Prerequisites

- Rust and Cargo — rustup.rs
- Git
- Unix-like environment (Linux, macOS, or Windows with WSL2)

For CLI-only installations (WSL, Raspberry Pi, etc.), Rust can be installed with:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

2.0.2 System Dependencies

MalChela's YARA engine is YARA-X, compiled in as a Rust crate. There is no system YARA library to install and no `YARA_LIBRARY_PATH` or `BINDGEN_EXTRA_CLANG_ARGS` to set, and HTTPS uses rustls, so OpenSSL isn't needed either. The build only needs a C compiler and `make`, for the few bundled C libraries (ssdeep, bzip2, LZFSE, and xz).

Linux

```
sudo apt install build-essential pkg-config
```

macOS

```
xcode-select --install
```

Upgrading from v4.3 or earlier? The old `yara`, `libyara-dev`, `libjansson-dev`, `openssl`, `clang`, GTK, and GLib packages are no longer used by the build. You can leave them installed, and you can drop the `YARA_LIBRARY_PATH` and `BINDGEN_EXTRA_CLANG_ARGS` exports from your shell profile.

Third-party integrations (TShark, Volatility 3, the YARA-X CLI, and others in `tools.yaml`) are optional and installed separately. See Integrating Third Party Tools for each one.

2.0.3 Clone the Repository

```
git clone https://github.com/dwmetz/MalChela.git
cd MalChela
```

If you cloned MalChela before 17-Apr-2026, you may see a diverging branches error when pulling. Run `git fetch origin && git reset --hard origin/main` to resync — this was a one-time history rewrite to remove a large file.

2.0.4 Build Tools

To build **all tools** in release mode in one step, use the script in the workspace root:

```
chmod +x release.sh
./release.sh
```

This compiles every core tool and generates optimized release binaries under `target/release/`. This is required before first use of the web interface or case features, which rely on prebuilt binaries.

Individual tools can also be built on their own:

```
cargo build -p fileanalyzer --release
```

⚠️ Using `--release` is highly recommended to ensure optimal performance and avoid unexpected behavior when launching tools from the web interface.

2.0.5 Run

PWA (recommended)

On first run, execute the setup script after building the binaries:

```
cd server
./setup-server.sh
```

Then start the server:

```
./start-server.sh
```

The PWA will be accessible from any browser on the local network.

CLI

```
./target/release/malchela
```

The CLI is retained for scripting and automation use cases.

⚠️ **Important:** MalChela binaries must be invoked from the project root directory. Always use `cd /path/to/MalChela && ./target/release/<binary>` rather than calling the binary directly from another path. This is required for correct resolution of API key files, YARA rules, and Sigma rules — all of which are resolved relative to the project root. API keys are read exclusively from files under `api/`; environment variables are not supported.

2.0.6 Windows Notes

- Best experience via WSL2.
- As of October 2025, both the MalChela CLI and PWA operate on Windows under WSL2.
- The web interface is accessible via any browser on Windows (WSL2 recommended for running the server).

2.0.7 What's Next

- Case Management — no need to start with a file or folder; any tool result can be saved to a case.
- tools.yaml Configuration — add third-party or custom tools to the GUI.
- REMnux Mode — a REMnux-tailored `tools.yaml` is loaded automatically when MalChela runs on a REMnux system.
- Offline Mode — run fully air-gapped, with all network calls skipped at the source.
- AI Integration & MCP Support — expose the full tool suite to Claude and other AI agents.

3. Navigation

A quick reference for getting around the MalChela web interface: what greets you on first launch, and what each button in the top toolbar does.

3.1 Home & At a Glance

Home is the PWA's default/startup screen — it displays the MalChela ASCII art, a rotating koan, and an **At a Glance** stats card summarizing the current workspace at a glance:

Field	Shows
Cases	Open vs. closed case counts
Detections	Number of rules currently loaded from `detections.yaml`
API Keys	How many of the supported API key files are configured (e.g. `4/12`)
MalChela Tools	How many cargo-built MalChela binaries are present in `target/release/`, out of the total `tools.yaml` expects (e.g. `17/17`) — this is the count referenced in Installation and troubleshooting build issues
Integrations	How many configured third-party tools (from `tools.yaml`) are actually found on `PATH`
Update Check	Whether a newer commit is available on the tracked branch — skipped automatically when Offline Mode is enabled

The Home screen reloads automatically whenever you click the  **Home** button in the toolbar, refreshing all of the above.

3.2 Top Toolbar

Left to right:

Icon	Button	Opens
	Home	The landing screen — ASCII art, koan, and the **At a Glance** card described above.
	Analyze	The Analyze target picker — point it at a file, folder, `.app` bundle, `.dmg`, or `.pkg` and it auto-runs every tool File Miner suggests.
	Cases	The case management browser.
	Hide Tools Panel	Collapses/expands the left tool sidebar, so the console can use the full width. Persists across reloads.
	About	Version and feature summary.
	Configuration	Dropdown with three items (see below).
	Upload Files	Upload a local file to the server for analysis — useful when the browser and the MalChela server aren't on the same machine.
	MITRE Lookup	The standalone MITRE ATT&CK lookup panel — no internet required.
	Notebook	A scratchpad for recording strings/IOCs/notes across a session.
	View Reports	Browse and open any previously saved report directly.
	User Guide	Opens this documentation site in a new tab.

On narrow screens, everything past the first divider collapses into a ⋯ **More** overflow menu with the same items.

3.2.1 Configuration Dropdown

Icon	Item	Opens
	Server Config	The server URL/connection settings, and the Offline Mode toggle.
	API Keys	See API Configuration.
	tools.yaml	See the tools.yaml reference.

4. Configuration

4.1 tools.yaml

MalChela uses a central `tools.yaml` file to define which tools appear in the web interface, along with their launch method, input types, categories, and optional arguments. This YAML-driven approach allows full control without editing source code.

Key Fields in Each Tool Entry

Field	Purpose
name	Internal and display name of the tool
description	Shown in web interface for clarity
command	How the tool is launched (binary path or interpreter)
exec_type	One of `cargo`, `binary`, or `script`
input_type	One of `file`, `folder`, or `hash`
file_position	Controls argument ordering
optional_args	Additional CLI arguments passed to the tool
category	Grouping used in the web interface left panel

⚠️ All fields except `optional_args` are required.

4.1.1 Swapping Configs: REMnux Mode and Beyond

MalChela supports easy switching between tool configurations via the web interface.



YAML Config Tool

To switch:

- Open the **Configuration Panel**
- Use **"Select tools.yaml"** to point to a different config
- Restart the web interface or reload tools

This allows forensic VMs like REMnux to use a tailored toolset while keeping your default config untouched.

A bundled `tools_remnux.yaml` is included in the repo for convenience.

KEY TIPS

- Always use `file_position: "last"` unless the tool expects input before the script
- For scripts requiring Python, keep the script path in `optional_args[0]`
- For tools installed via `pipx`, reference the binary path directly in `command`

4.1.2 Backing Up and Restoring tool.yaml

The MalChela web interface provides built-in functionality to back up and restore your `tools.yaml` configuration file.

Backup

To create a backup of your current `tools.yaml`:

- Open the **Configuration Panel**
- Click the **“Back Up Config”** button
- A timestamped copy of `tools.yaml` will be saved to the default location

You’ll see a confirmation message when the operation completes successfully.

Restore

To restore from a previous backup:

- Click the **“Restore Config”** button in the Configuration Panel
- Select a previously saved backup file
- The selected file will overwrite the current configuration

This feature makes it easy to experiment with custom tool setups while retaining a safety net for recovery.

4.2 API Configuration

Some tools within MalChela rely on external services. In order to use these integrations, you must configure your API credentials.

4.2.1 Tools That Use API Keys

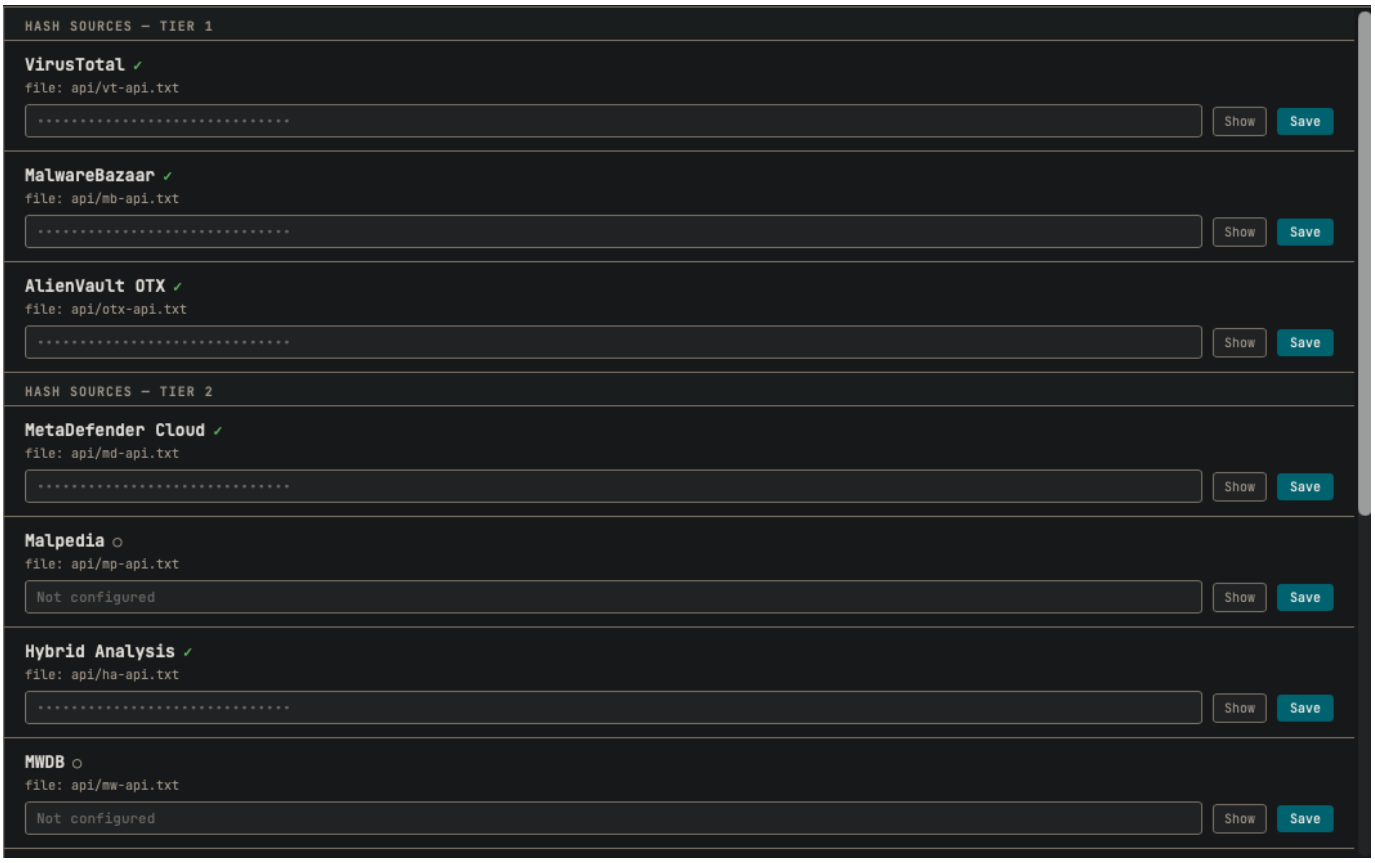
Tool	Service	Key File	Purpose
`fileanalyzer`	VirusTotal	`vt-api.txt`	Hash lookup
`tiquery`	VirusTotal	`vt-api.txt`	Hash, IP, and URL lookup (Tier 1)
`tiquery`	MalwareBazaar	`mb-api.txt`	Multi-source hash lookup (Tier 1)
`tiquery`	AlienVault OTX	`otx-api.txt`	Multi-source hash and IP lookup (Tier 1)
`tiquery`	MetaDefender	`md-api.txt`	Multi-source hash lookup (Tier 1)
`tiquery`	Hybrid Analysis	`ha-api.txt`	Multi-source hash lookup (Tier 2)
`tiquery`	FileScan.IO	`fs-api.txt`	Multi-source hash lookup (Tier 2)
`tiquery`	Malshare	`ms-api.txt`	Multi-source hash lookup (Tier 2)
`tiquery`	Triage	`tr-api.txt`	Multi-source hash lookup (Tier 2)
`tiquery`	urlscan.io	`url-api.txt`	URL lookup (optional — raises rate limits)
`tiquery`	Google Safe Browsing	`gsb-api.txt`	URL lookup
`tiquery`	GreyNoise	`gn-api.txt`	IP lookup (scanner / known-benign-service context; free Community key works)

4.2.2 Where to Configure

MalChela stores API keys as plain text files in the `api/` directory of your workspace:

```
api/vt-api.txt
api/mb-api.txt
api/otx-api.txt
api/md-api.txt
api/ha-api.txt
api/fs-api.txt
api/ms-api.txt
api/tr-api.txt
api/url-api.txt
api/gsb-api.txt
```

Each file should contain a single line with your API key. Keys are read at runtime — tools automatically skip sources whose key file is absent.



API Configuration Utility

4.2.3 Managing Your Keys with the Configuration Utility

The MalChela web interface includes a built-in Configuration Panel that lets you easily **Create or update API key files** without opening a text editor.

Look for the **API Key Management** section in the Configuration Panel. Changes take effect immediately and persist across sessions.

4.2.4 Best Practices

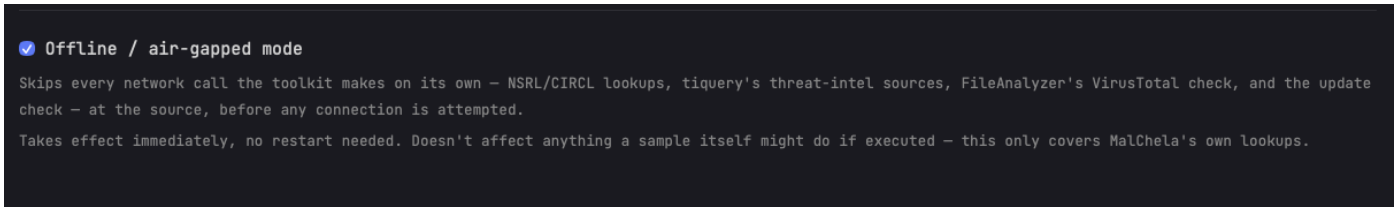
- **Keep these files private.** Do not commit them to Git or share them publicly.

If a tool requires an API key but none is found, it will log a warning and skip external requests.

4.3 Offline Mode

MalChela normally makes a handful of network calls on its own — some obvious (Threat Intel Query's API-key-gated lookups), some less so. Offline Mode is a single switch that skips **every** one of them at the source, before any connection is even attempted. It's built for air-gapped labs and malware analysis scenarios where an unexpected outbound connection attempt — even a harmless, timed-out one — isn't acceptable.

4.3.1



4.3.2 What It Skips

Tool	Call	Notes
`nsrlquery`	`hashlookup.circl.lu`	Despite the name, this is a live lookup against CIRCL's public hash database — not a local NSRL file. No API key involved, so this one is easy to trigger by accident.
`tiquery`	Every configured source	Including Objective-See's malware catalogue (`os`), the one source with no API-key gate of its own — it normally attempts a fetch on any cache miss regardless of which keys you've configured.
`fileanalyzer`	VirusTotal	A separate embedded check from Threat Intel Query's own VirusTotal source — gated on a VT key being configured, but if one is, plain FileAnalyzer also phones home.
`codesign_check -- check-revocation`	`ocsp.apple.com` (or whichever responder the certificate specifies)	Opt-in — off unless `--check-revocation` is passed directly, or the run comes through Analyze, which passes it automatically for each sample's top-level binary. Checked inside the tool itself (via `MALCHELA_OFFLINE`), so it self-skips no matter how it's invoked.
Home screen	Update check (`git remote update`)	Runs automatically every time the Home screen loads, since it's the app's default/startup screen.

Each of these normally fails only after a DNS/connect attempt (and, on a truly air-gapped host, a real timeout). With Offline Mode on, they return a clean "skipped" result immediately instead.

What it doesn't cover: anything a *sample itself* might do if executed. Offline Mode only governs MalChela's own lookups — it's not a sandbox or network isolation layer for the malware you're analyzing. Static analysis is unaffected either way, since nothing in that path ever touches the network.

4.3.3 Enabling It

Web interface: Configuration menu → **Offline / air-gapped mode** checkbox. Takes effect immediately — no server restart needed, and it stays on across restarts until you turn it off.

CLI: set the environment variable before running any tool directly:

```
export MALCHELA_OFFLINE=1
cargo run -p nsrlquery -- d41d8cd98f00b204e9800998ecf8427e
```

```
Offline mode (MALCHELA_OFFLINE) — skipped CIRCL Hash Lookup network request.
```

The web interface toggle and the environment variable both work through the same mechanism — enabling it in the Configuration panel sets `MALCHELA_OFFLINE=1` for every tool the server launches on your behalf, so the two are interchangeable depending on how you're driving MalChela.

4.3.4 Where It's Stored

The toggle persists the same way an API key does — a plain file in the workspace's `api/` directory:

```
api/offline_mode.txt
```

Containing `1` (enabled) or `0` (disabled). This file is read fresh on every check, not cached, which is why the web toggle takes effect without a restart.

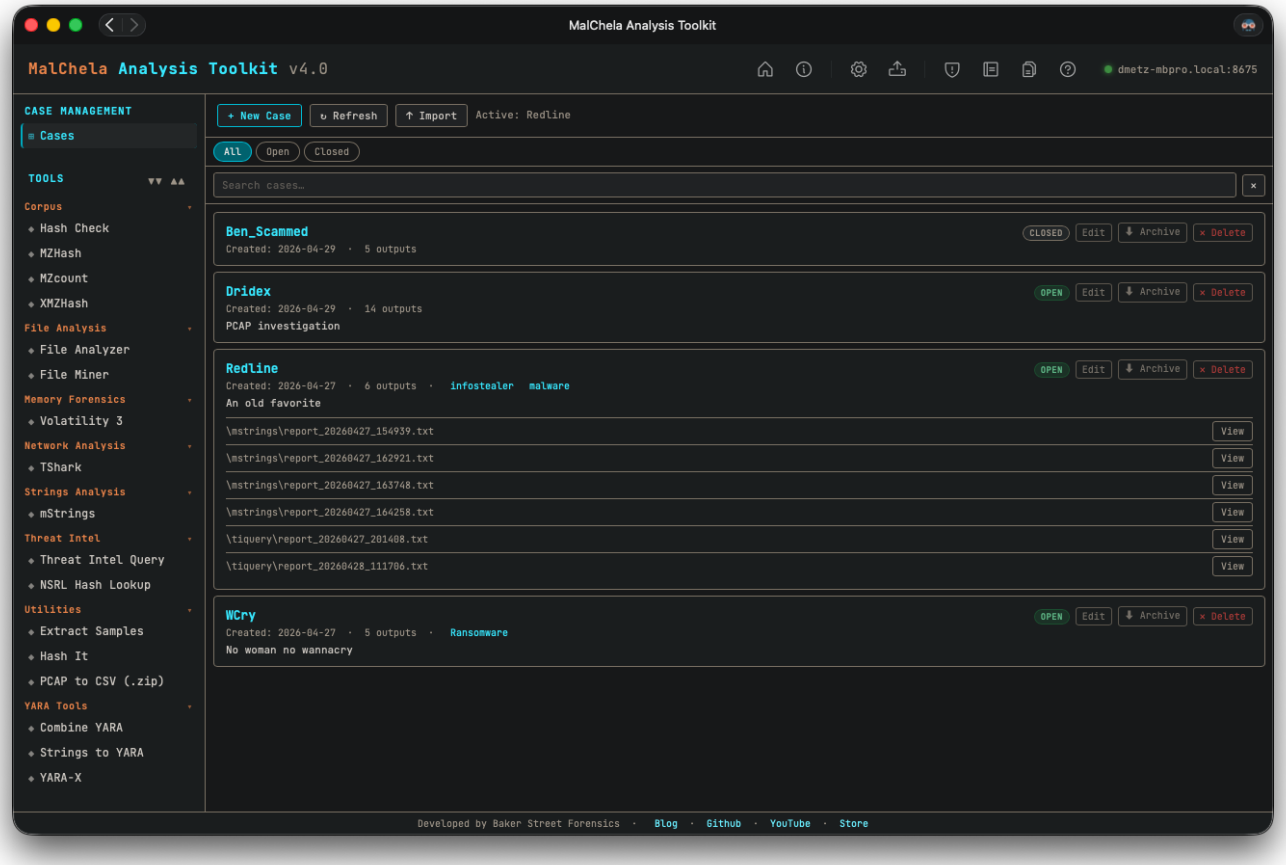
4.3.5 Reducing Network Dependency Further

Two page-load resources the PWA previously fetched from the internet on every load — the app icon and its webfonts (JetBrains Mono, Share Tech Mono) — are now vendored locally (`server/icons/` , `server/fonts/`) rather than pulled from GitHub/Google Fonts. This isn't gated behind Offline Mode; it's just always local now, which also makes page load faster regardless of connectivity.

5. 4.16 Case Management

The Cases feature in MalChela v4.0 introduces a structured way to manage analysis sessions, organize tool results, and preserve analyst notes for long-term reference. Each case acts as a container for a specific file or folder input, and captures relevant metadata, tool output, and custom annotations.

Cases are managed entirely through the web interface.



Case Management

5.1 Creating a Case

A case can be created from the New Case menu, or by selecting 'Save to Case' in any tool.



New Case

After choosing the input, assign a descriptive case name. The web interface will automatically create a new folder under:

```
saved_output/cases/<case_name>/
```

When a **folder** is used as the case input, FileMiner runs automatically and populates an interactive results table. Suggested tools can be launched on a per-file basis directly from the FileMiner results panel.

5.2 Case Contents

Each case folder includes:

- `case.yaml` — combines metadata, case tracking, and user notes in a single structured file
- `fileminer/`, `mstrings/`, `tiquery/`, etc. — tool-specific output directories

5.3 Notes & Tagging

The Notebook allows users to record notes, label items with tags, and track investigative context across sessions.

- Notes can include plain text, YAML fragments, or markdown.

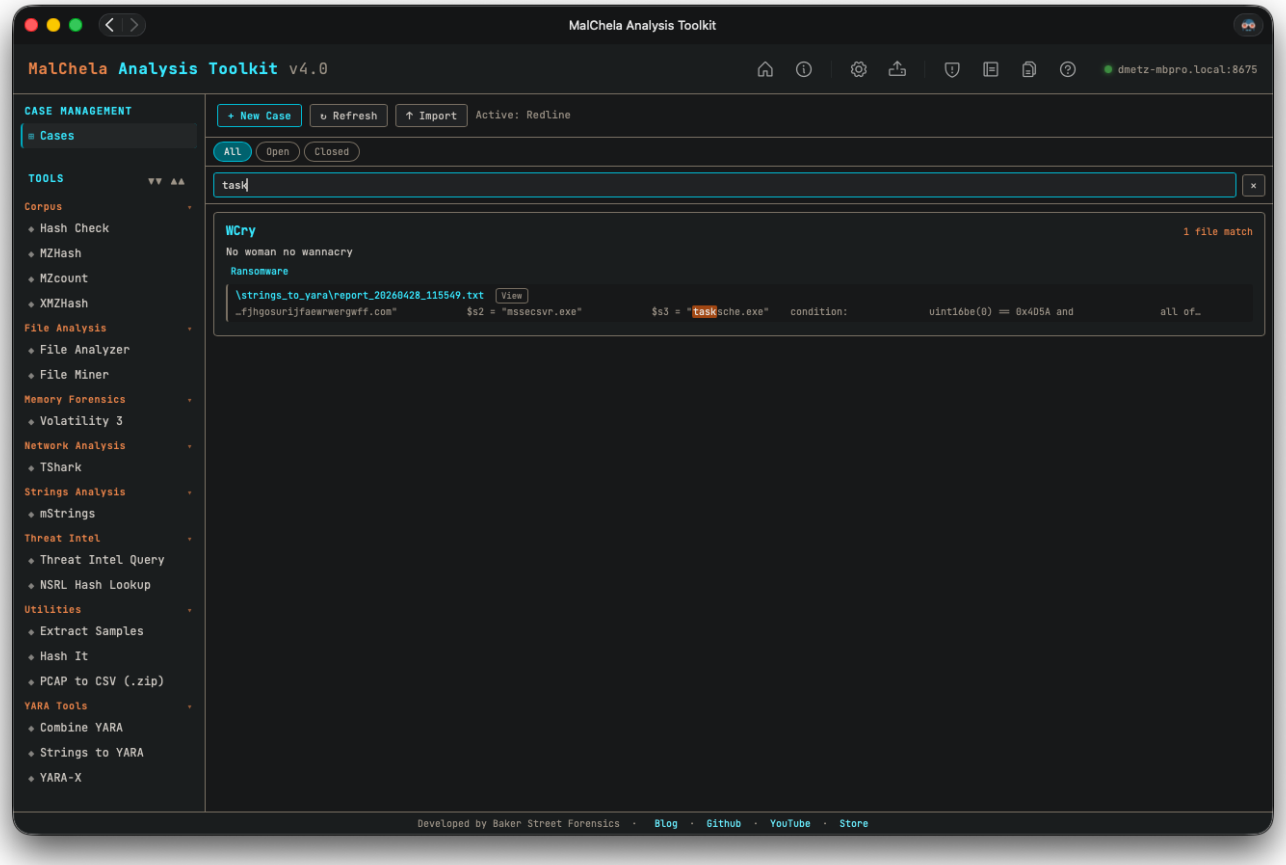
5.3.1 Tagging

- Tags appear in the Workspace panel under the current case.
- Tags help organize and filter case context.

5.3.2 Search

From the Case modal, you can search across:

- All saved tool outputs
- Notes contents



Searching Cases

5.4 Archiving Cases

Cases can be archived into a `.zip` file using the web interface's **Archive Case** feature. This creates a portable snapshot that includes all metadata, notes, and tool outputs.

Archives are saved to:

```
saved_output/case_archives/
```

5.5 Importing Cases

To restore a previously archived case:

1. Open the web interface
2. Use **Import Case**
3. Select a `.zip` archive from `saved_output/case_archives/` (or provide the full path to any valid case archive)

The case will be extracted into `saved_output/cases/` and appear in the workspace. If a case with the same name already exists, you will be prompted to confirm overwrite.

5.6 Summary

Cases enable consistent, organized forensic workflows across sessions and machines. Whether you're triaging suspicious files, running multi-tool pipelines, or preserving findings for reporting, the Cases feature ensures nothing is lost.

6. AI Integration & MCP Support

MalChela exposes its full tool suite — including Analyze and the Mac Analysis stack — to AI agents like Claude through the **Model Context Protocol (MCP)**. Once configured, Claude has persistent, structured access to MalChela's full analysis suite without any manual briefing or context-pasting each session.

For a detailed walkthrough of AI-assisted analysis approaches, see the blog post: MalChela Meets AI: Three Paths to Smarter Malware Analysis.

6.1 Prerequisites

- MalChela built from source — see Installation. This compiles all MalChela tools to `target/release/`.
- Node.js v18 or later
- Claude Desktop (or Claude Code, via the plugin marketplace — see below)
- API keys configured for whichever lookups you want available (see API Configuration)

6.1.1 Set MALCHELA_DIR

```
# Add to your shell profile (~/.zshrc, ~/.bashrc, etc.)
export MALCHELA_DIR=/path/to/MalChela
```

The MCP server uses this to locate the workspace root — where `vt-api.txt` lives, not the `mcp/` subdirectory — and shells out to the compiled binaries under `$MALCHELA_DIR/target/release/`.

6.2 Installation

Via Anthropic community marketplace (Claude Code):

```
claude plugin marketplace add anthropics/claude-plugins-community
claude plugin install malchela@claude-community
```

Via self-hosted marketplace (Claude Code):

```
claude plugin marketplace add dwmetz/MalChela
claude plugin install malchela@malchela-marketplace
```

Manual install (Claude Desktop):

```
cd /path/to/MalChela/.claude-plugin/mcp
npm install
```

Then merge the relevant block from `example_config.json` into your Claude Desktop configuration file:

Platform	Path
macOS	`~/Library/Application Support/Claude/claude_desktop_config.json`
Windows	`%APPDATA%\Claude\claude_desktop_config.json`

Open `example_config.json` and update the paths to match your system, then copy the `malchela` block into your Claude Desktop config — if you already have other MCP servers configured, add it alongside them rather than replacing the whole file. Restart Claude Desktop afterward; the MalChela tools appear in Claude's tool list automatically.

6.3 Available Tools

17 tools map 1:1 to a compiled MalChela binary. Four more are MCP-only, with no binary of their own: `set_case` (session state), `analyze` (an orchestrator that shells out to the other 17 in sequence), and `report_template` / `save_report` (malware analysis reports).

Unlike the PWA, the MCP server uses a **mandatory case model** — call `set_case` once at the start of a session before running any other tool; every subsequent tool's output is saved to that case automatically.

Case Management

Tool	Description
`set_case`	Set (or create) the active investigation case for the session. All later tool output is saved to it automatically. Call this first.

File Analysis

Tool	Description
`fileanalyzer`	Hashes, entropy, packing detection, PE metadata, YARA rule matches, and VirusTotal status. Best first step for any unknown file.
`fileminer`	Scans a folder for file type mismatches and metadata anomalies; suggests follow-up tools per file. Entry point for an unknown directory or sample set.
`analyze`	One-shot auto-triage: runs `fileminer` against a file, folder, `.app` bundle, `.dmg`, or `.pkg`, then automatically dispatches every tool `fileminer` suggests, producing a combined rollup report. `.dmg` / `.pkg` targets are auto-unwrapped via `dpp_extract` first. Requires an active case (`set_case` first).
`unpack`	Static packer detection and unpacking for PE, ELF, and Mach-O — never executes the sample. UPX (including tampered headers, repaired before `upx -d`), fat Mach-O slices, PyInstaller entry scripts, .NET/Costura resources, NSIS/Inno Setup payloads, overlays, and embedded executables; recurses into each recovered layer. Protectors (Themida, VMProtect) are identified with evidence. `analyze` runs it automatically on every executable.
`report_template`	Lenny Zeltser's Malware Analysis Report template (v1.1.0, CC BY 4.0, bundled in `assets/report_templates/` for offline use), plus a guide to which MalChela output fills each section. No case needed.
`save_report`	Save a finished report to the active case's `report/` folder and register it in `case.yaml`. Won't overwrite an existing report unless asked.

Strings Analysis

Tool	Description
`mstrings`	String extraction with IOC detection and MITRE ATT&CK mapping via Sigma-style rules. Supports macOS Mach-O binaries and `.app` bundles (main executable auto-resolved).

Threat Intel

Tool	Description
`tiquery`	Multi-source hash lookup across MalwareBazaar, VirusTotal, AlienVault OTX, and InQuest Labs — combined results matrix with detections, families, and source links.
`nsrlquery`	Checks a hash (MD5 or SHA1) against the NIST NSRL known-good database.

Hashing Tools

Tool	Description
`hashit`	Generates MD5, SHA1, and SHA256 hashes for a single file.
`hashcheck`	Checks a hash against a local lookup file of known hashes.
`mzhash`	Recursively hashes files with MZ headers (Windows PE/DLL) — one hash file per algorithm plus a path lookup table.
`xmzhash`	Like `mzhash` but inverted — hashes files that do *not* have MZ, ZIP, or PDF headers (Linux/Mac/unusual samples).
`mzcount`	Counts and summarizes file types within a directory for quick triage.

YARA Tools

Tool	Description
`strings_to_yara`	Converts a text file of extracted strings into a formatted YARA rule draft. Typically used after `mstrings`.
`combine_yara`	Merges multiple YARA rule files from a folder into a single consolidated ruleset.
`yara_scan`	Scans a file or folder with YARA rules (a file or a folder of rules) on the embedded YARA-X engine. Returns NDJSON with each matching rule's namespace, tags, metadata, and strings; saves a Markdown report to the active case.

Mac Analysis

Tool	Description
`plist_analyzer`	Parses `.plist` files and `.app` bundle `Info.plist` for malware indicators: hidden background agent, ATS disabled, custom URL schemes, env injection, and more.
`macho_info`	Parses Mach-O binaries: architecture, PIE/ASLR status, linked libraries, RPATH entries, section entropy, symbol status, and deprecated crypto library detection.
`codesign_check`	Inspects macOS code signing: Developer-signed vs. ad-hoc vs. unsigned, Team ID, Bundle ID, entitlements, and the `get-task-allow` flag.

Utilities

Tool	Description
`extract_samples`	Extracts password-protected malware archives (ZIP/RAR) using common passwords.
`dpp_extract`	Unwraps a `.dmg` or `.pkg` container (UDIF → HFS+/APFS → XAR → PBZX/CPIO) to reach the real payload files inside, including PKG Scripts archives. `analyze` calls this automatically for `.dmg` / `.pkg` targets.

6.4 Recommended Analysis Workflow

For initial triage of an unknown file, ask Claude to run the tools in this order:

1. `set_case` — create or select the case everything below will be saved to
2. `fileanalyzer` — establish baseline: hashes, entropy, PE headers, initial VT verdict
3. `mstrings` — extract strings, surface IOCs and ATT&CK technique indicators
4. `tiquery` — pull full community threat intel with AV verdicts
5. `nsrlquery` — confirm or rule out known-good status
6. `report_template` → `save_report` — when you want a written report: Claude fills Lenny Zeltser's template from the case's findings and saves it to the case

Or skip the manual sequencing entirely and let `analyze` do it in one call. You don't need to invoke tools individually — just describe what you want:

"Analyze /path/to/suspicious.exe using MalChela and give me a full triage report."

Claude will run the appropriate tools in sequence and synthesize the results. For a formal write-up, ask for a malware analysis report — Claude pulls the bundled Zeltser template with `report_template`, fills it from the case, and saves it with `save_report`:

"Write a malware analysis report for this sample using the Zeltser template and save it to the case."

6.5 Agentic Coding Environments

For agentic coding environments (OpenCode and similar), MalChela ships an `AGENTS.md` describing available tools and usage patterns for autonomous discovery.

6.6 Kali Linux / Remote Server Setup

If you're running MalChela on a remote Kali system (such as a Raspberry Pi field kit) rather than locally on your Mac, see the **Approach 1** section of the MalChela Meets AI blog post for instructions on integrating MalChela into the `mcp-kali-server` setup instead.

6.7 Troubleshooting

Tools don't appear in Claude Desktop after restart Verify the path in `args` points to the correct `index.js` and that `MALCHELA_DIR` is set to the MalChela root. Check that `npm install` completed without errors.

`tiquery` **returns no results** Confirm the relevant API key file (e.g. `vt-api.txt`) exists in the MalChela root and contains a valid key with no trailing whitespace or newline issues.

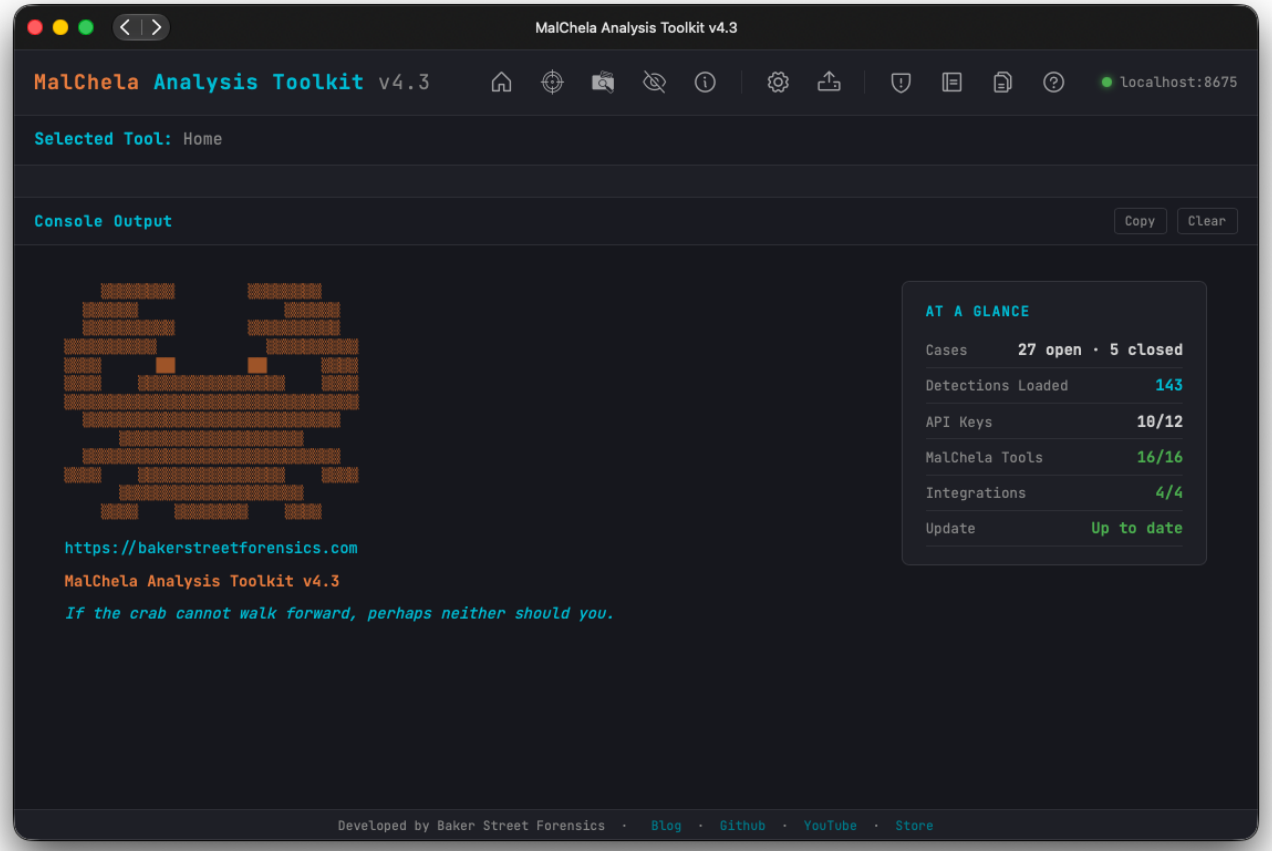
`fileanalyzer` **or** `mstrings` **fails with "workspace root not found"** The binary is not being run from the MalChela root directory. Verify `MALCHELA_DIR` in your config points to the correct path and that `mcp/index.js` is using it in the `cd` command.

`analyze` **fails with an error about no active case** Call `set_case` first — `analyze` requires an active case, unlike the PWA where Save-to-Case is opt-in.

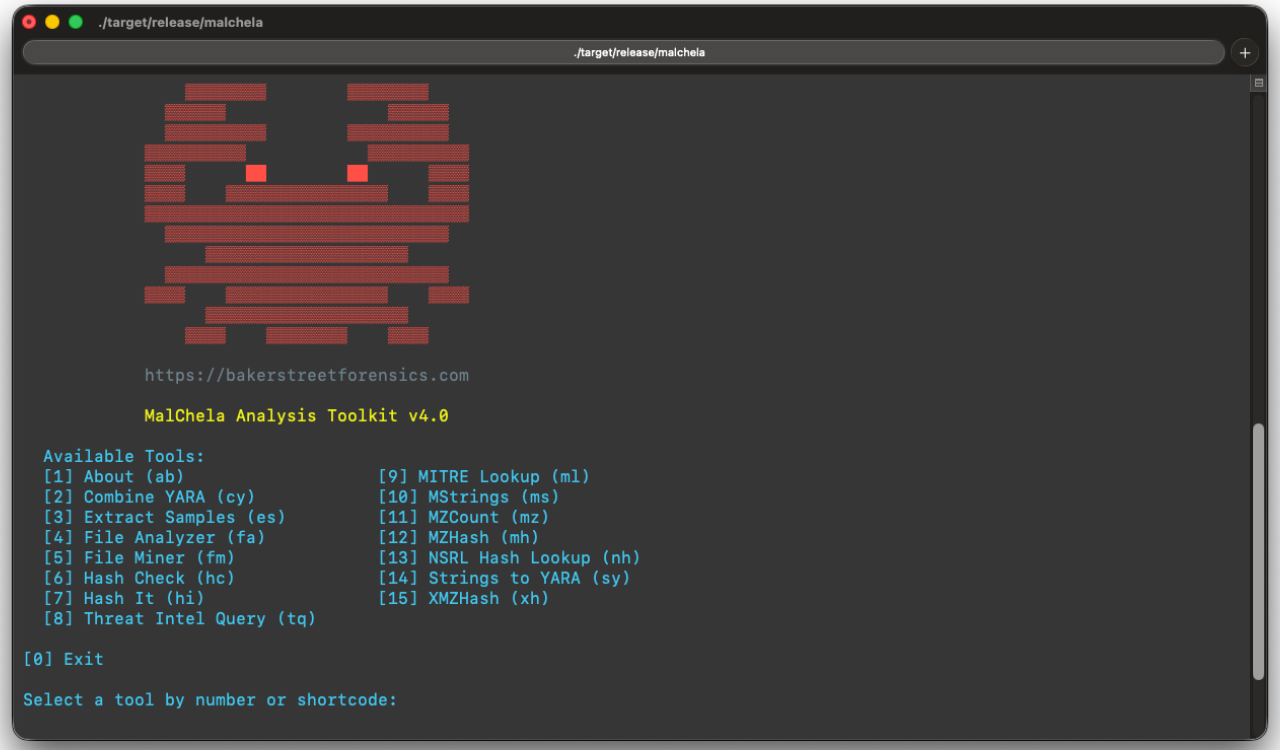
NSRL queries return "Offline or Error" The NSRL database requires a local copy to be present. This is expected behavior if the NSRL database has not been set up — it does not affect other tools. See also Offline Mode, which intentionally skips this call when enabled.

7. Core Tools

7.1 Overview



MalChela Web Interface



MalChela CLI

7.1.1 Analyze

One-click auto-triage: point Analyze at a file, folder, or .app bundle and it classifies everything with File Miner, dispatches every tool it suggests, and produces a combined **MalChela Summary** rollup report. See the Analyze page for details.

7.1.2 MalChela Core Tools

These built-in programs provide fast, flexible functionality for forensics and malware triage.

Program	Function
Combine YARA	Point it at a directory of YARA files and it will output one combined rule
Extract Samples	Point it at a directory of password protected malware files to extract all
File Analyzer	Get the hash, entropy, packing, PE info, YARA and VT match status for a file
File Miner	Scans a folder for file type mismatches and metadata, and provides suggested tools
Hash Check	Check a hash against a .txt or .tsv lookup table
Hash It	Point it to a file and get the MD5, SHA1 and SHA256 hash
mStrings	Analyzes files — including Mach-O binaries and .app bundles — with Sigma rules (YAML), extracts strings, matches ReGex
MZHash	Recurse a directory, for files with MZ header, create hash list and lookup table
MZcount	Recurse a directory, uses YARA to count MZ, Zip, PDF, other
NSRL Query	Query a MD5 or SHA1 hash against NSRL
Strings to YARA	Prompts for metadata and strings (text file) to create a YARA rule
Unpack	Detects packers and installers in PE, ELF, and Mach-O and statically recovers the payload — UPX (including tampered headers), PyInstaller, .NET/Costura, NSIS, Inno Setup, fat Mach-O, overlays, embedded executables
Threat Intel Query	Multi-source hash and URL lookup. Hash: VT, MB, OTX, HA, FileScan, Malshare, MetaDefender, ObjectiveSee. URL: VT, urlscan.io, Google Safe Browsing. Web interface supports file-to-hash and QR code decode.*
XMZHash	Recurse a directory, for files without MZ, Zip or PDF header, create hash list and lookup table
YARA Scan	Scan a file or folder with YARA rules on the embedded YARA-X engine; flags follow `yr scan`

MalChela also includes a dedicated Mac Analysis tool stack for static analysis of macOS binaries, bundles, and property lists.

***Threat Intel Query supports optional API keys for its sources.** Keys can be managed via the API Keys panel in the MalChela web interface or by placing them in the `api/` directory. Sources without configured keys are skipped automatically.

7.2 Usage

7.2.1 Getting Started

Before runnning MalChela for the first time, you need to build the release binaries. There is a script provided in the workspace root.

Building the Releases

```
chmod +x release.sh
./release.sh
```

7.2.2 Execution

MalChela supports three main workflows:

Direct Tool Execution (CLI)

```
./target/release/toolname [input] [flags]
```

MalChela CLI Launcher Menu

```
./target/release/malchela
```

MalChela Web Interface

```
python server/malchela_server.py
```

Then open your browser to `http://localhost:8675`.

7.2.3 CLI Usage Notes

- Tools that accept paths (files or folders) should be run from the `target/release` directory after building with `release.sh`: `bash ./target/release/fileanalyzer /path/to/file -o`

Most tools now support a `--case <name>` argument to redirect saved output to a specific case folder under `saved_output/cases/`. Cases must be initiated with either a file or folder as the input. Hash-only workflows can be added to an existing case but cannot start one.

Note: Some tools (e.g., `mstrings`, `fileanalyzer`) require the `-o` flag to trigger output saving—even when `--case` is specified. Others (like `strings_to_yara` or `mzcount`) save automatically when a case is provided. Refer to the Tool Behavior Reference below for details.

Output Formats

All tools that support saving reports use the following scheme: `saved_output/<tool>/report_<timestamp>.<ext>`

To save output, use:

```
-o -t   # text
-o -j   # json
-o -m   # markdown
```

- `-o` enables saving (CLI output is not saved by default)

If a `--case` argument is supplied, the report will be saved to: `saved_output/cases/<case_name>/<tool>/report_<timestamp>.<ext>`

Example:

```
cargo run -p mstrings — path/to/file — -o -j
```

- If `-o` is used without a format (`-t`, `-j`, or `-m`), an error will be shown

7.2.4 Web Interface Notes

Web Interface Features Summary

- Categorized tool list with input type detection (file, folder, hash)
- Arguments textbox and dynamic path browser
- Console output with ANSI coloring
- Status bar displays CLI-equivalent command
- Alphabetical sorting of tools within categories
- Tool descriptions shown alongside tool names

Web Interface Walkthrough

Layout

- Top Toolbar: navigation and utility buttons — see Navigation for a full key with icons
- Left Panel: Tool categories and selections — collapsible via the toolbar's Hide Tools Panel button, so the console can use the full width
- Center Panel: Dynamic tool input options
- Console Panel: Output display

Running Tools

- Select a tool
- Fill in input fields
- Configure options (save report, format, etc.)
- Click Run

Save Report

- Formats:
- .txt Analyst-readable summary
- .json Machine-parsable, structured output
- .md Shareable in tickets, wikis, etc.
- Location: saved_output//report_. (only one file is generated per run)

Notebook

- An integrated notepad for recording strings, indicators or notes
- Supports saving as text, markdown and YAML formats
- Integrated “Open in VS Code” button for saved notes
- Any line starting with `hash:` is ignored when using the Notebook as a source for String_to_Yara to generate YARA rules

7.2.5 Tool Behavior Reference

Tool	Input Type	Supports `-o`	Prompts if Missing	Notes
combine_yara	folder	❌	✅	Combines multiple YARA rules
extract_samples	file	❌	✅	Extracts archive contents
fileanalyzer	file	✅	✅	Uses YARA + heuristics
hashit	file	✅	✅	Generates hashes
hashcheck	hash and lookup file	❌	✅	Checks files against known hashes
fileminer	folder	✅	✅	Identifies mismatches
mstrings	file	✅	✅	Maps strings to MITRE
mzhash	folder	✅	✅	Hashes files with MZ header
nsrlquery	file	✅	✅	Queries CIRCL
strings_to_yara	text file and metadata	Case Only	✅	Saves to case folder if `--case` is provided
mzcount	folder	❌	✅	Will save to case folder if `--case` is provided
xmzhash	folder	✅	✅	Hashes files without known headers

7.3 Analyze

Analyze is a one-click auto-triage workflow: point it at a file, folder, `.app` bundle, `.dmg`, or `.pkg`, and it classifies everything with File Miner, then automatically dispatches every tool File Miner suggests for each file it finds. There's no need to read File Miner's suggestions and run each tool by hand — Analyze closes that loop for you and produces a single combined report at the end.

Unlike the other Core Tools, Analyze is not a standalone Rust binary — it's a workflow built on top of the existing tools, available through the **PWA** and the **MCP server**.

How It Works

1. If the target is a `.dmg` or `.pkg`, dpp Extract unwraps it first (UDIF → HFS+/APFS → XAR → PBZX/CPIO) and Analyze continues against the extracted files. A `.zip` (the only way to get a directory-based sample like an `.app` bundle through the web interface's file-only upload widget) is auto-extracted the same way.
2. File Miner scans the target (a single file, every file in a folder, or every file inside an `.app` bundle or extracted container) and classifies each one. A `.zip` / `.dmg` / `.pkg` found **mid-scan** — not just as the top-level target, e.g. a sample that's simply a lone archive sitting in its own folder — is extracted the same way and its contents folded into the same run, breadth-first (an archive nested inside an extracted archive also gets unwrapped), capped at 5 nested extractions per run as a guard against zip bombs or pathological nesting.
3. Every executable (PE, ELF, Mach-O) is run through Unpack. When it recovers payloads — a UPX-decompressed binary (including tampered UPX), fat Mach-O slices, PyInstaller entry scripts, Costura assemblies, installer contents, carved overlays or embedded executables — each payload is classified by File Miner and added to the run as a file of its own, marked **Unpacked from** its parent in the rollup. Clean executables add nothing.
4. For each file, Analyze runs every tool File Miner suggests — the same suggestions you'd see running File Miner manually, just dispatched automatically instead of one at a time.
5. Results are combined into a single **MalChela Summary** rollup report (`malchela_summary_<timestamp>.md`), saved alongside the individual tool reports it summarizes.

A single sample or small bundle is the intended use case — Analyze auto-dispatches up to 25 files per run. For corpus-scale scans, use MZHash, MZCount, or XMZHash instead.

Over the 25-file cap (routine for a dpp-extracted `.pkg` payload — a trojanized installer commonly bundles the full legitimate app it trojanized alongside the injected malicious file(s); EvilQuest's sample is 624 files for one Mach-O of actual interest), Analyze doesn't error out or auto-run everything:

- **PWA:** hands off to File Miner's own interactive table, pointed at the extracted target — real per-row "run this tool" buttons instead of a static summary.
- **MCP:** returns a file listing directly as text (extension mismatches first, then up to 40 more with a note to call File Miner directly for the rest), since there's no browser to redirect an AI agent to — it can act on the listing immediately.

The MalChela Summary Rollup Report

The rollup is laid out in the order of Lenny Zeltser's Malware Analysis Report template (bundled in `assets/report_templates/`), so it lines up section for section with a written report. It fills the evidence sections only. Executive Summary, a family confidence call, MBC behaviors, What We

Don't Know, and Infection Vector need an analyst's judgment, and a mechanical rollup that filled them would read as conclusions nobody drew, so they're left for the written report. The MCP plugin's `report_template` and `save_report` tools build that report from the case.

Section	Source	What It Shows
Sample Snapshot	Threat Intel Query, FileAnalyzer, Unpack, Code Sign Check	Threat intel family names and tags with the source behind each; VirusTotal verdicts (cross-referenced from FileAnalyzer and Threat Intel Query, so Mach-O samples get the same coverage as PE); target platform and architecture; the primary artifact's SHA-256; packing and obfuscation; a revoked Developer ID certificate when `--check-revocation` finds one; file counts, with duplicate content (identical SHA-256 under different names) grouped into one write-up
Component Inventory	Unpack, File Miner	One row per file: its role (primary artifact, or a recovered payload and how it was recovered), file type, and notes such as **unpacked from** its parent
Capabilities	mStrings, Unpack, tool flags	One row per ATT&CK technique: tactics, the rules or flags behind it, and the files that hit it, plus the rule-hit count and a pointer to the interactive matrix (see below). These are rule hits: evidence to corroborate, not confirmed behavior. Hits from a bundler's bootloader or runtime describe the wrapper, not the sample. Low-confidence flag mappings are marked
Indicators of Compromise	mStrings, hashes	The template's tiers: Hash Values (every file), IP Addresses, Domain Names, Network Artifacts (URLs), and Host Artifacts (dropped files and paths), each linked to the file it came from. Network indicators are defanged (`hxxp://` , `[.]`). Long lists are capped with a pointer to the full mStrings report
Analysis Details	Every tool	Tool flags (RPATH entries, hidden-Dock plists, Team ID mismatches, and similar) attributed to the file and tool, then each file's own section with every tool's formatted report. MalChela is static only, so behavioral and memory analysis are marked as not performed
Appendix: Analysis Environment	Analyze	MalChela version, analysis depth (static only), the tools run and how many times, whether network lookups were on, the host, and when it ran



Every filename and IOC in the rollup is a link that jumps straight to that finding's section further down the same report — not out to the indicator itself, which is exactly why network indicators are defanged: the link target is always an internal anchor, but the *displayed text* shouldn't read like a live, clickable URL regardless.

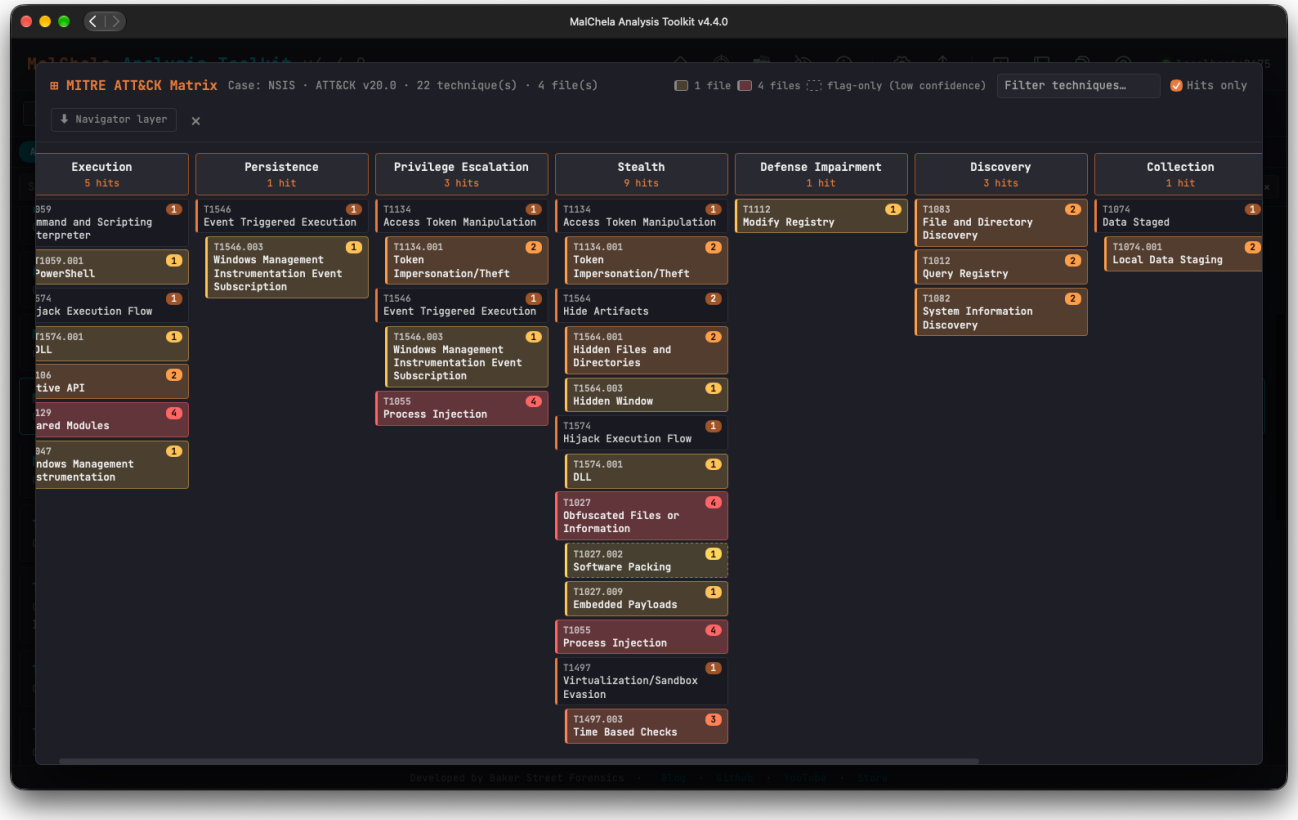
formatted report — real tables and headers pulled from each tool's own Markdown output, not raw console text. The rollup reads cleanly whether you're viewing it in the PWA or opening the file directly.

7.3.1 Each file's section embeds every tool's actual formatted report — real tables and headers pulled from each tool's own Markdown output, not raw console text. The rollup reads cleanly whether you're viewing it in the PWA or opening the file directly.

MITRE ATT&CK Matrix

Unpack, recursive extraction, and the Mac tools mean a single Analyze run can produce ATT&CK findings from many files and several tools. Analyze rolls all of them into one **Enterprise matrix** (ATT&CK v20 — Stealth and Defense Impairment in place of the retired Defense Evasion), saved next to the rollup:

```
mitre_matrix_<timestamp>.json      # drives the PWA's interactive matrix
navigator_layer_<timestamp>.json   # ATT&CK Navigator layer (format 4.5)
```



MITRE ATT&CK Matrix

Techniques come from three sources:

Source	Techniques
mStrings	Every `detections.yaml` rule hit, plotted under the tactics the rule specifies
Unpack	T1027.002 (Software Packing) and T1027.009 (Embedded Payloads)
Tool flags	Mach-O Info, Code Sign Check, Plist Analyzer, and File Analyzer flags mapped through `assets/flag_attack_map.yaml` — for example `system()` / `popen()` imports → T1059.004, `LSUIElement` → T1564.003, a revoked certificate → T1553.002

Flag mappings carry a confidence: **high** when the flag is direct evidence of the technique, **low** when it's consistent with it but common in benign software too (ad-hoc signing, a stripped symbol table). Low-confidence cells are drawn with a dashed border and score half. Some flags are deliberately left unmapped — RPATH entries, ATS exceptions, and similar describe a weakness or a configuration, not adversary behavior.

A cell's score is the number of distinct files that hit the technique (duplicate content counts once), so a fat Mach-O whose two slices both hit a rule scores 2.

formatted report — real tables and headers pulled from each tool's own Markdown output, not raw console text. The rollup reads cleanly whether you're viewing it in the PWA or opening the file directly.

In the PWA:

- After a run, click ⊞ **MITRE ATT&CK matrix** at the top of the results.
- For a saved case, click ⊞ **MITRE** on the case card. This combines every Analyze run in the case into one matrix, along with every standalone report saved to the case by a tool that emits techniques (mStrings, Unpack, Mach-O Info, Code Sign Check, Plist Analyzer, File Analyzer). Analyze saves its tools' reports into the case too, so a finding already counted from a run isn't counted again from its report. Older cases work as well: the matrix is rebuilt from each run's summary and each tool report, so a case saved before the matrix existed still gets one.
- **Hits only** shows just the tactics and techniques with findings; turn it off for the full Enterprise matrix with hits highlighted in place. Sub-technique hits are nested under their parent.
- Click a hit to see every file, tool, and rule or flag behind it, with links to MITRE Lookup and attack.mitre.org. mStrings hits list the matched strings (as in the mStrings report: up to eight per rule, URLs and IPs defanged).
- ⬇ **Navigator layer** downloads the layer for the run or the combined case; open it in ATT&CK Navigator with **Open Existing Layer → Upload from local**.

Save to Case

Analyze's Save to Case control matches every other tool panel — a checkbox and case dropdown, opt-in. When enabled, every tool Analyze dispatches saves its report to the case (and registers it in `case.yaml`, so it shows up in the case browser), and the rollup itself is saved to:

```
saved_output/cases/<case_name>/analyze/
```

When no case is selected, individual tool reports still land in their normal default locations (e.g. `saved_output/mstrings/`), and the rollup is saved to:

```
saved_output/analyze/
```

Analyze dispatches nsrlquery, Threat Intel Query, and FileAnalyzer's VirusTotal check automatically along with everything else — with Offline Mode enabled, those tools skip their network call cleanly and the rollup still completes normally, just without the results that require connectivity.

IP reputation. After mStrings runs, Analyze looks up the first five standalone public IP IOCs with Threat Intel Query (VirusTotal, AlienVault OTX, and GreyNoise, whichever have keys). It adds each result next to the IP in the summary's Indicators of Compromise table, flagged with ⚠ when VirusTotal engines detect it or GreyNoise classifies it as malicious. IP Address Lists and public DNS resolvers are never looked up. The cap keeps a run inside a free VirusTotal key's limits (4 lookups a minute, 500 a day). Set `MALCHELA_IP_LOOKUPS` to change it, or to `0` to turn lookups off. Each lookup is saved to the case as `tiquery/ip_<address>_<timestamp>.md`, and the Analysis Environment appendix records how many IPs were looked up and how many weren't. Offline Mode skips it.

Analyze also passes Code Sign Check's `--check-revocation` flag automatically, scoped to each sample's top-level binary only (not embedded frameworks/plugins/XPC helpers — a single bundle can carry dozens of those, each of which would otherwise trigger its own OCSP query and slow the run down considerably). Like the other network-dependent tools above, this is skipped automatically with Offline Mode enabled.

PWA Usage

Click the **Analyze** button in the top toolbar, choose a file, folder, `.app` bundle, `.dmg`, or `.pkg`, then configure the run before clicking **Run**:

- **Save to Case** — opt-in, same pattern as every other tool panel.
- **Concise Output** — on by default. Renders the MalChela Summary rollup inline instead of the full expanded per-tool output. Turn it off to see everything each tool produced without opening the saved report separately.

There's also a shortcut straight from File Miner's results table: each row's **Analyze** button runs every suggested tool for that one file and produces the same rollup, without going through the toolbar button first.

---

formatted report — real tables and headers pulled from each tool's own Markdown output, not raw console text. The rollup reads cleanly whether you're viewing it in the PWA or opening the file directly.

MCP Usage

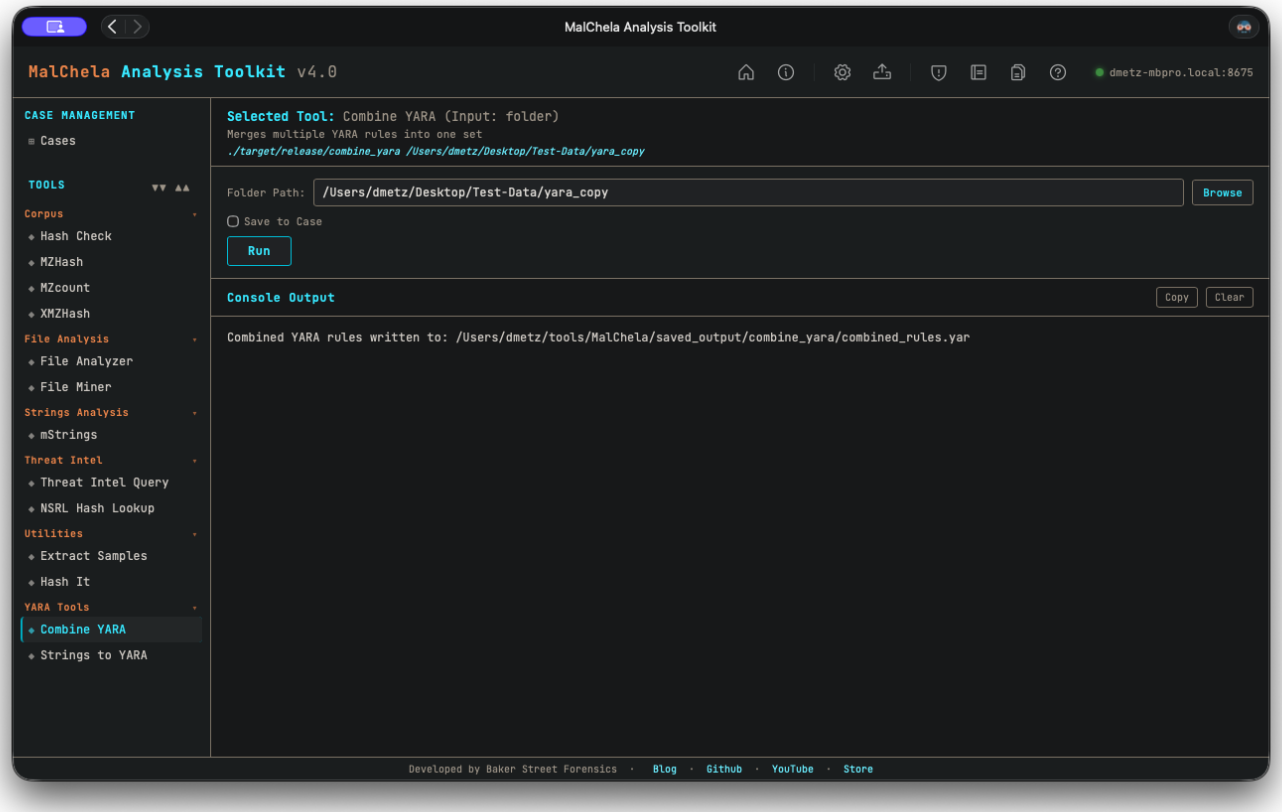
Analyze is available to AI agents as the `analyze` tool in the MCP server. Like every other MCP tool, it requires an active case first — call `set_case` before `analyze`. This keeps the MCP's behavior consistent across all of its tools rather than special-casing this one workflow; there's no "no case" mode on the MCP side the way there is in the PWA.

`dpp_extract` is also exposed as its own MCP tool, for when an agent needs the extracted files without Analyze's auto-dispatch — `analyze` calls it automatically when pointed at a `.dmg` / `.pkg`, so calling it directly first isn't necessary for the normal triage flow.

See `.claude-plugin/mcp/README.md` for MCP setup.

7.4 CombineYARA

Combine YARA merges multiple YARA rule files into a single consolidated rule set. It recursively scans a folder for .yar or .yara files and combines them into one output file. Ideal for organizing or deploying rule collections.



Combine YARA

🔧 CLI Syntax

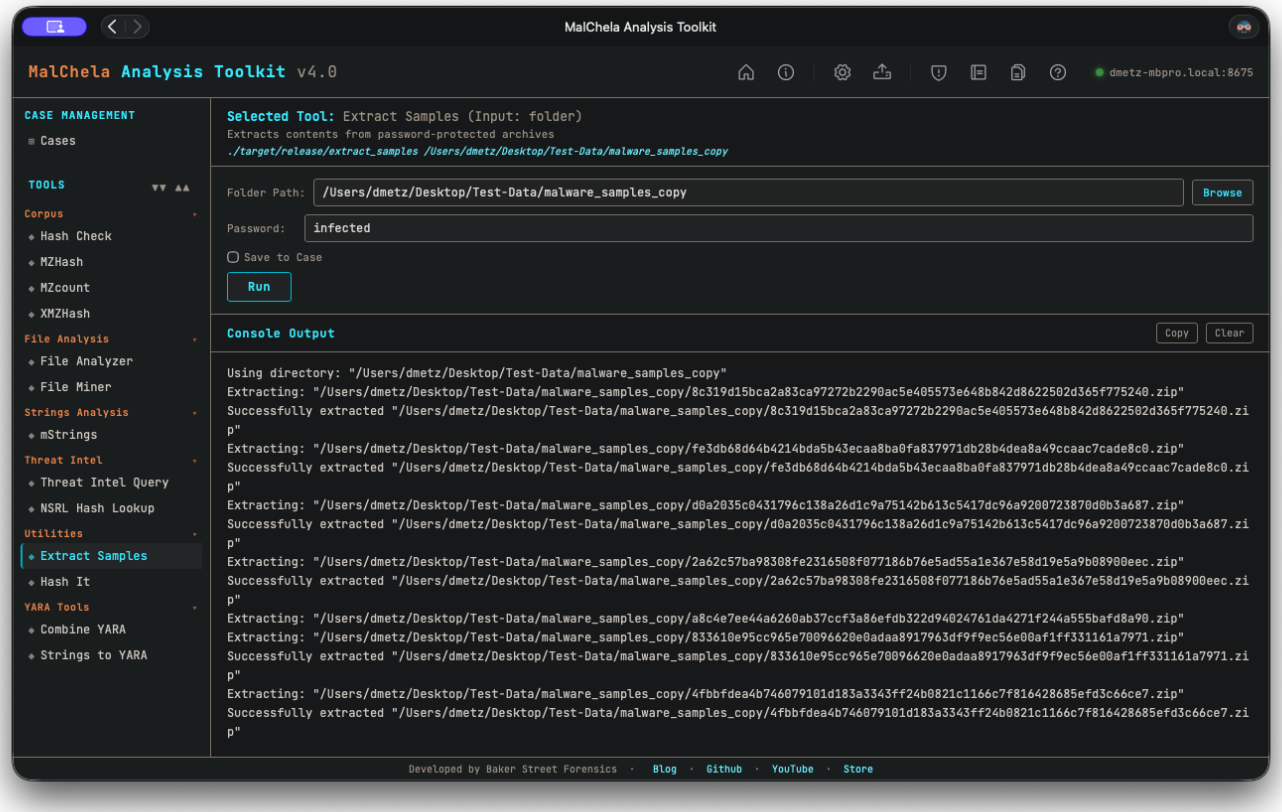
```
cargo run -p combine_yara /path_to_yara_rules/
```

If no path is provided, the tool will prompt you to enter the directory interactively.

```
Enter the directory path to scan for YARA rules:
```

7.5 ExtractSamples

Extract Samples recursively unpacks password-protected archives commonly used in malware sharing (e.g., .zip, .rar, .7z). It uses default malware research passwords like infected and malware to extract samples in bulk for analysis.



Extract Samples

🔧 CLI Syntax

```
# Example 1: No case name
cargo run -p extract_samples /path_to_directory/ infected
```

In this mode, extracted files will be placed in the same location as each archive found.

```
# Example 2: With case name
cargo run -p extract_samples /path_to_directory/ infected --case Case123
```

When `--case` is provided, all extracted files will be saved under:

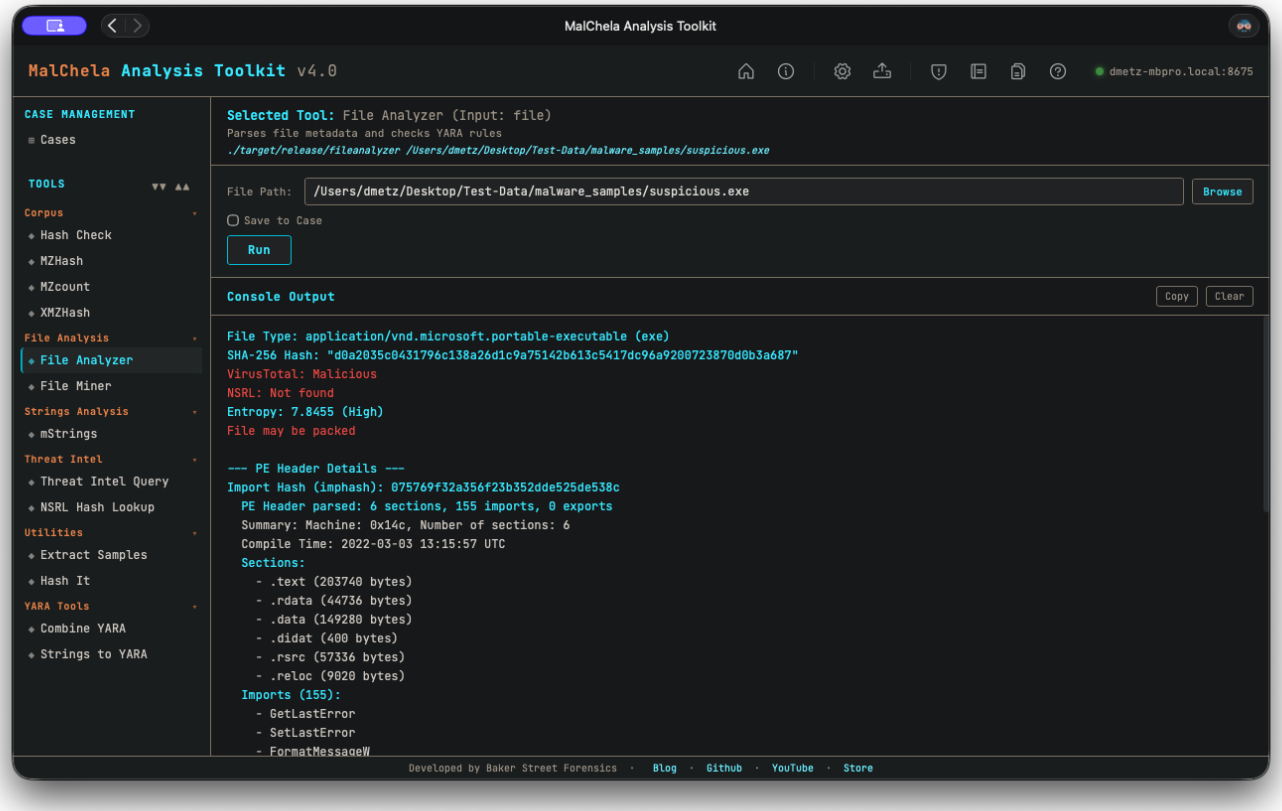
```
/saved_output/cases/Case123/extract_samples/
```

If no path or password is provided, the tool will prompt for them interactively.

7.6 FileAnalyzer

FileAnalyzer performs deep static analysis on a single file. It extracts hashes, entropy, file type metadata, YARA rule matches, NSRL validation, and — for PE files — rich header details including import/export tables, compile timestamp, and section flags. Ideal for triaging unknown executables or confirming known file traits.

FileAnalyzer makes two network calls of its own as part of a normal run — a VirusTotal hash check (if a VT API key is configured) and an NSRL lookup via nsrlquery (no key needed). Both are skipped cleanly with Offline Mode enabled.



File Analyzer

- YARA rules for `fileanalyzer` are stored in the `yara_rules` folder in the workspace. You can modify or add rules here.

🔧 CLI Syntax

```
# Example 1: No case name
cargo run -p fileanalyzer -- /path_to_file/ -o -t

# Example 2: With case name
cargo run -p fileanalyzer -- /path_to_file/ -o -t --case Case123
```

When `--case` is provided, output will be saved under:

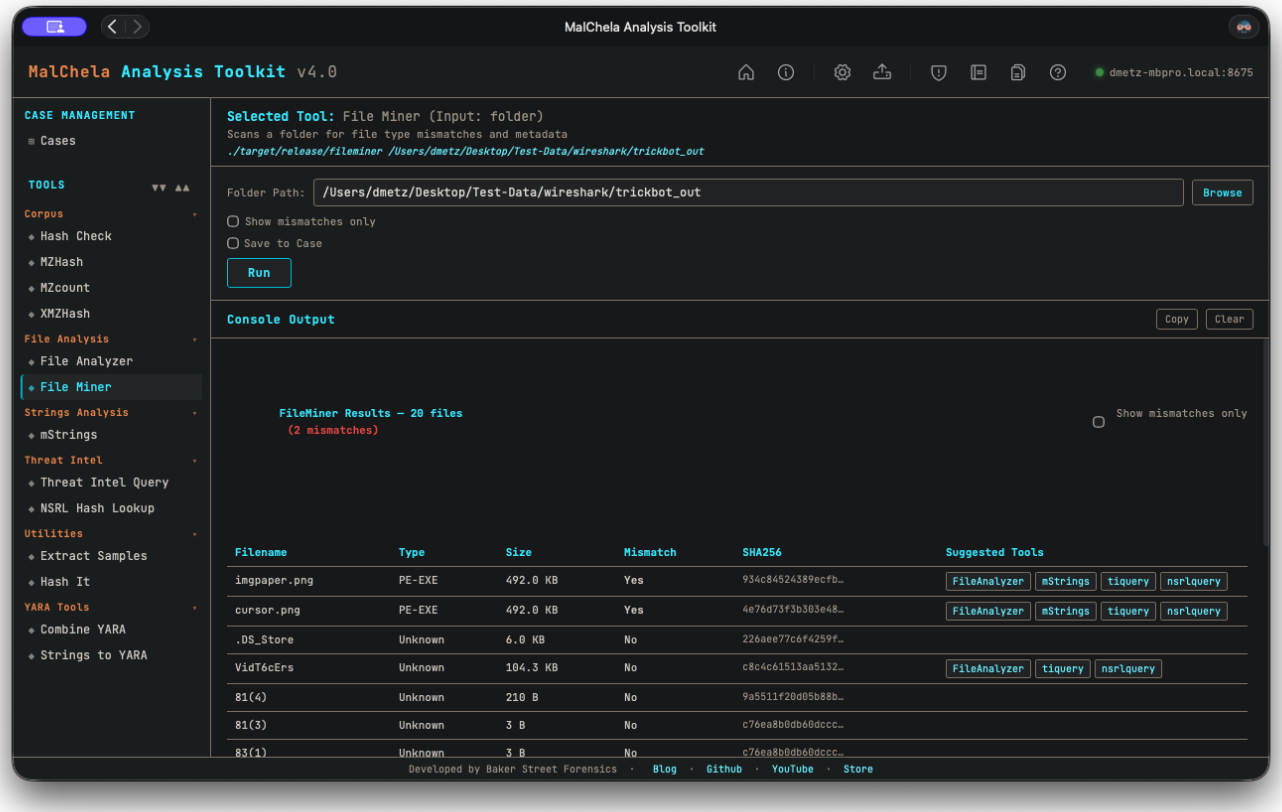
```
/saved_output/cases/Case123/fileanalyzer/
```

If `--case` is not specified, files will be saved in the default `saved_output/fileanalyzer/` folder.

7.7 FileMiner

Note: `FileMiner` replaces the deprecated `MismatchMiner`.

FileMiner is a command-line tool that recursively scans a directory to analyze files by magic bytes and hash, identifying mismatches between file extensions and true types. It is useful for forensic triage, anomaly detection, and preparing follow-up analysis using other tools in the MalChela suite.



File Miner

7.7.1 Function Overview

- Identifies file types using magic byte detection (`infer`)
- Computes SHA-256 hashes for all files
- Detects extension mismatches
- Suggests relevant analysis tools (e.g., FileAnalyzer, mStrings, TiQuery)
- Outputs results in a styled table or optional JSON format
- Integrates with case management via the `--case` flag
- Automatically launches in the web interface when a folder-based case is created or restored
- Results populate an interactive table in the web interface
- Users can launch suggested tools on a per-file basis directly from the web interface
- Each row also has an **Analyze** button — runs every suggested tool for that one file in a single pass and produces a combined rollup, instead of launching each suggested tool individually (see Analyze)

7.7.2 CLI Usage

```
cargo run -p fileminer -- [OPTIONS] [DIR]
```

Options

Option	Description
`DIR`	Directory to analyze. Optional — will prompt if not supplied.
`--json`	Save results to JSON. Defaults to `fileminer_output.json` unless `--output` is used.
`--output <filename>`	Overrides the default output file name. Used internally by the web interface.
`--case <case-name>`	Saves output under `saved_output/<case-name>/fileminer/` . Also passes case name to downstream tools.
`-m` , `--mismatches-only`	Only display entries with extension mismatches.

Examples

```
# Analyze interactively
cargo run -p fileminer --

# Analyze directory and save JSON
cargo run -p fileminer -- /path/to/files --json

# Save to specific case folder
cargo run -p fileminer -- /path/to/files --case case123

# Filter mismatches only
cargo run -p fileminer -- /path/to/files -m

# Combine all
cargo run -p fileminer -- /path/to/files --case suspicious_usb -m
```

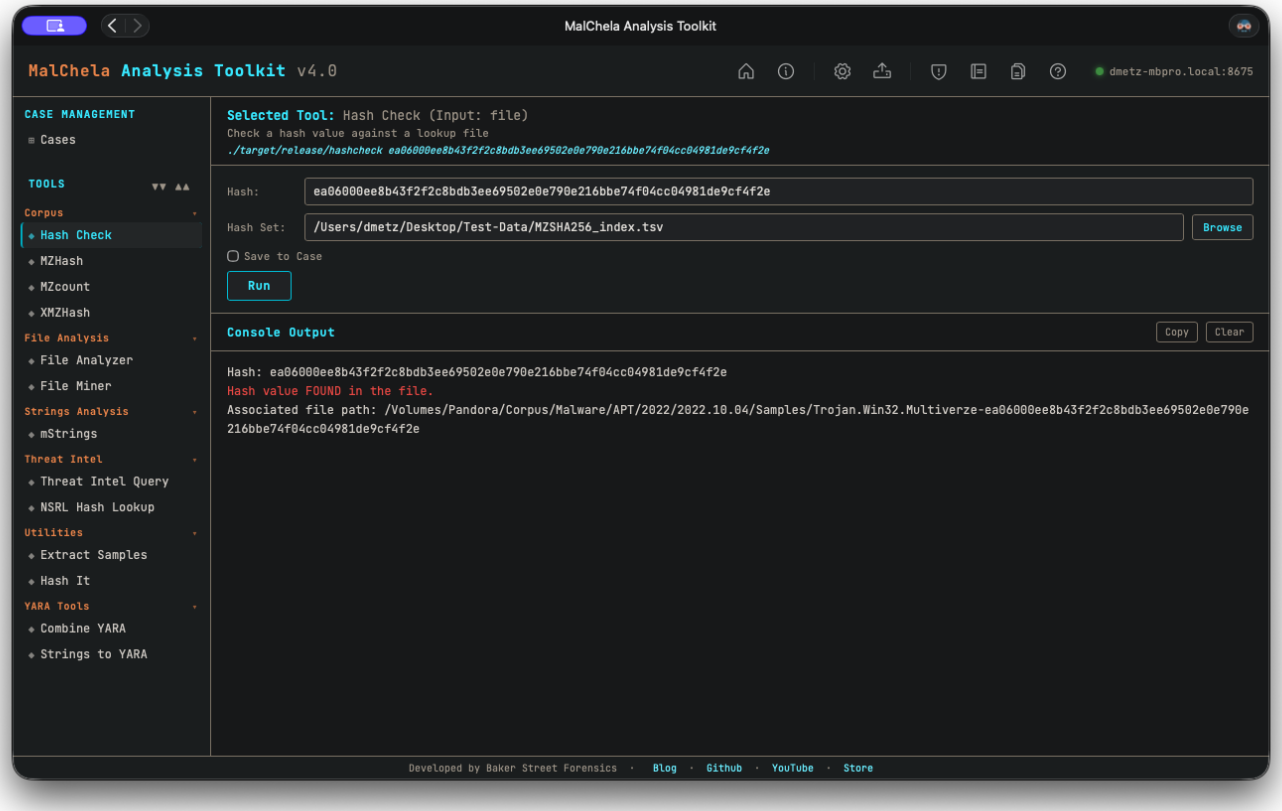
7.7.3 Web Interface Notes

- When a new case is created or restored using a folder, FileMiner runs automatically in the web interface.
- Results are saved under `saved_output/cases/<case-name>/fileminer/` .
- FileMiner displays an interactive table of results with suggested tools per file.
- Suggested tools can be launched directly from within the web interface results panel.
- Analyze also lands here automatically when its target (including a dpp Extract-unwrapped `.dmg` / `.pkg`) has more files than its 25-file auto-run cap — File Miner's interactive table replaces Analyze's own summary for large targets.

7.8 HashCheck

HashCheck lets you quickly verify whether a given set of files (or hash values) match any entry in one or more known-good or known-bad hash lists. It's designed to help analysts triage large collections of files by comparing against reference datasets — for example, malware repositories, NSRL exports, or your own curated lists.

Hash lists should be in `.tsv` format (tab-separated values) for best compatibility, though `.txt` files are also accepted.



Hash Check

You can generate .tsv lookup files using MZHash or XMZHash.

HashCheck supports **MD5**, **SHA1**, and **SHA256** formats.

🔧 CLI Syntax

```
# Example 1: Basic usage
cargo run -p hashcheck ./hashes.tsv 44d88612fea8a8f36de82e1278abb02f

# Example 2: Save output as .txt
cargo run -p hashcheck ./hashes.tsv 44d88612fea8a8f36de82e1278abb02f -- -o -t

# Example 3: Save output to case folder
cargo run -p hashcheck ./hashes.tsv 44d88612fea8a8f36de82e1278abb02f -- -o -t --case CaseName
```

HashCheck accepts a hash and a lookup file (TSV or TXT). If not provided, you'll be prompted interactively.

When `--case` is used, output will be saved under:

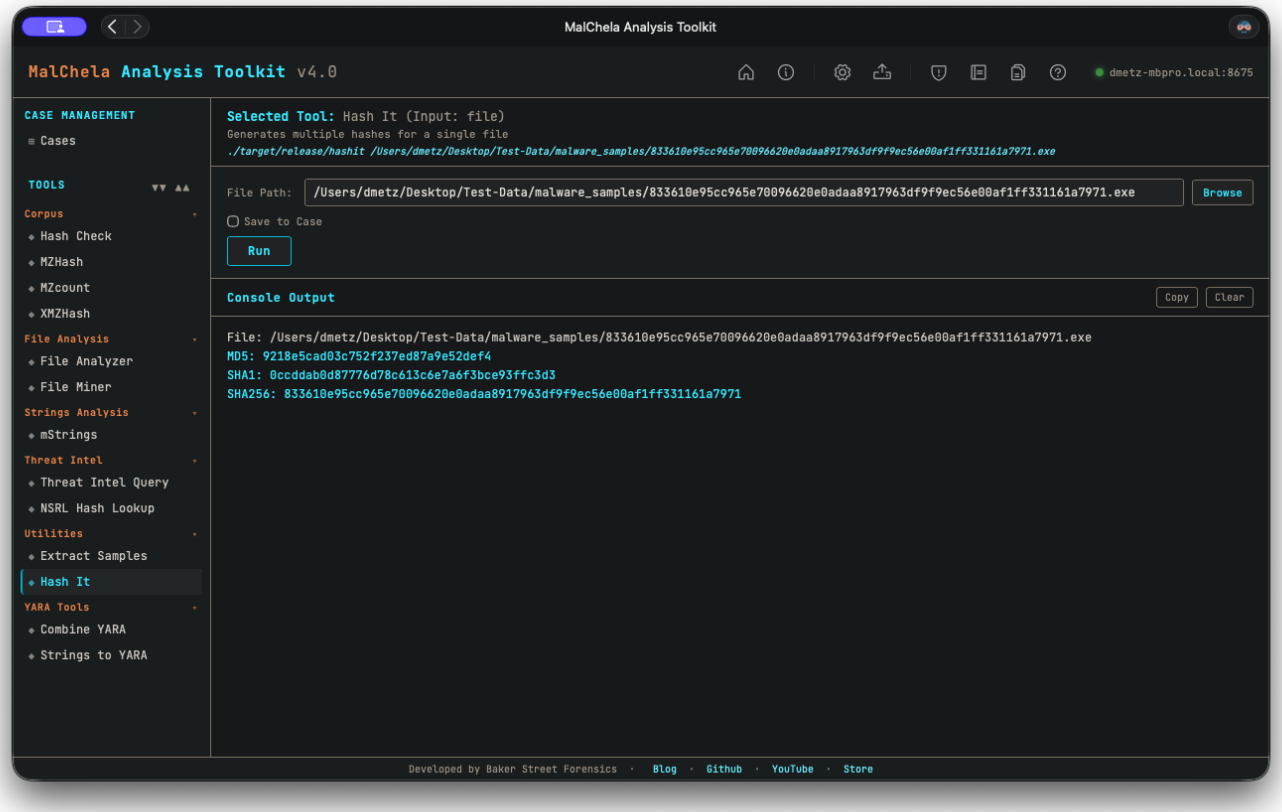
```
saved_output/cases/CaseName/hashcheck/
```

Without `--case`, reports are saved to the default:

```
saved_output/hashcheck/
```

7.9 HashIt

`hashit` is a flexible hashing utility that supports calculating multiple hash types for one or more files. It can be used to verify file integrity, generate forensic reports, or build hash sets for analysis. Output can be saved in text, JSON, or Markdown format, and optionally redirected into a case folder.



Hash It

The tool supports batch operations on directories and automatically recurses through subfolders. It's especially useful during triage to generate hash inventories or validate file integrity against known datasets.

🔧 CLI Syntax

```
# Example 1: Show hash values only
cargo run -p hashit -- /path_to_file/

# Example 2: Save as .txt
cargo run -p hashit -- /path_to_file/ -o -t

# Example 3: Save to case folder
cargo run -p hashit -- /path_to_file/ -o -t --case CaseName
```

Use `-o` to save output and include one of the following format flags: - `-t` → Save as `.txt` - `-j` → Save as `.json` - `-m` → Save as `.md`

If no file is provided, the tool will prompt you to enter the path interactively.

When `--case` is used, output will be saved under:

```
saved_output/cases/CaseName/hashit/
```

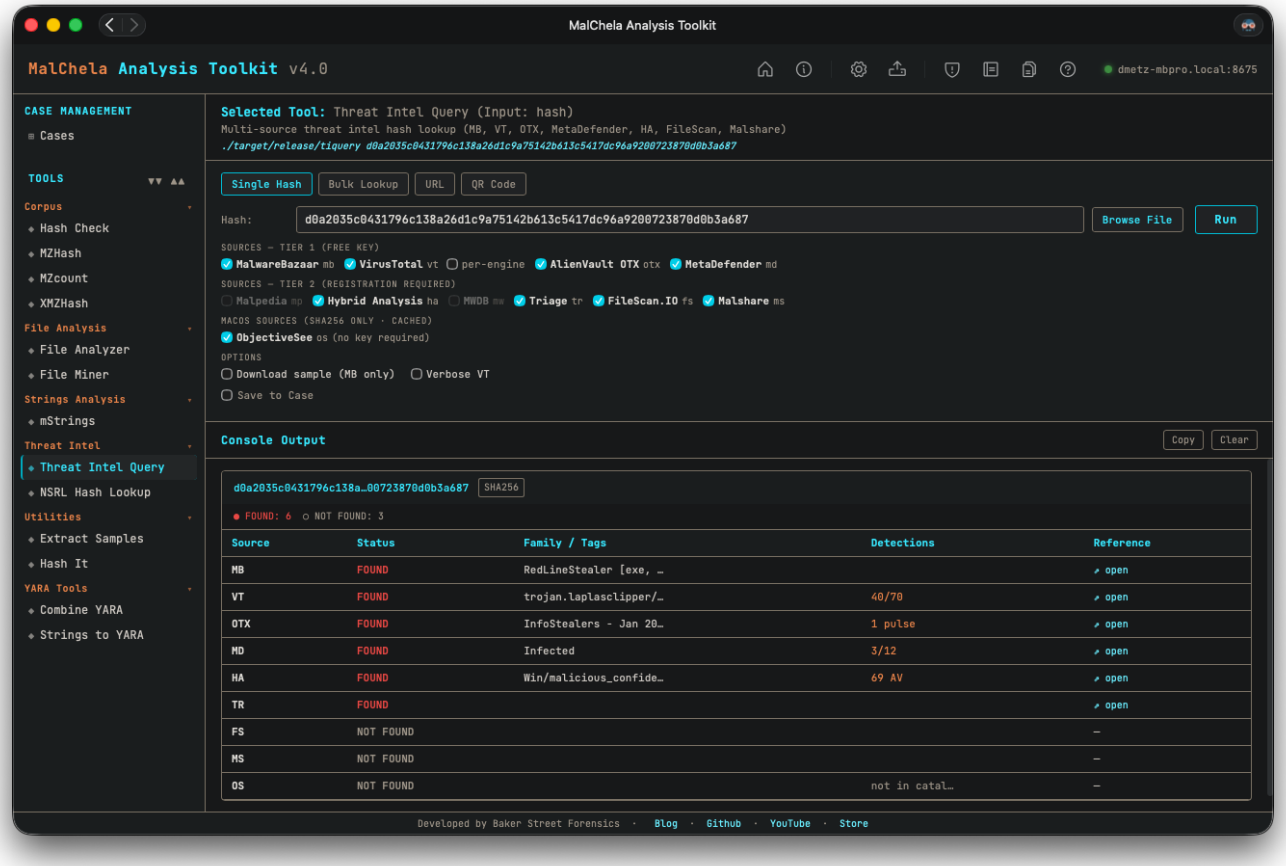
Otherwise, reports are saved to:

```
saved_output/hashit/
```

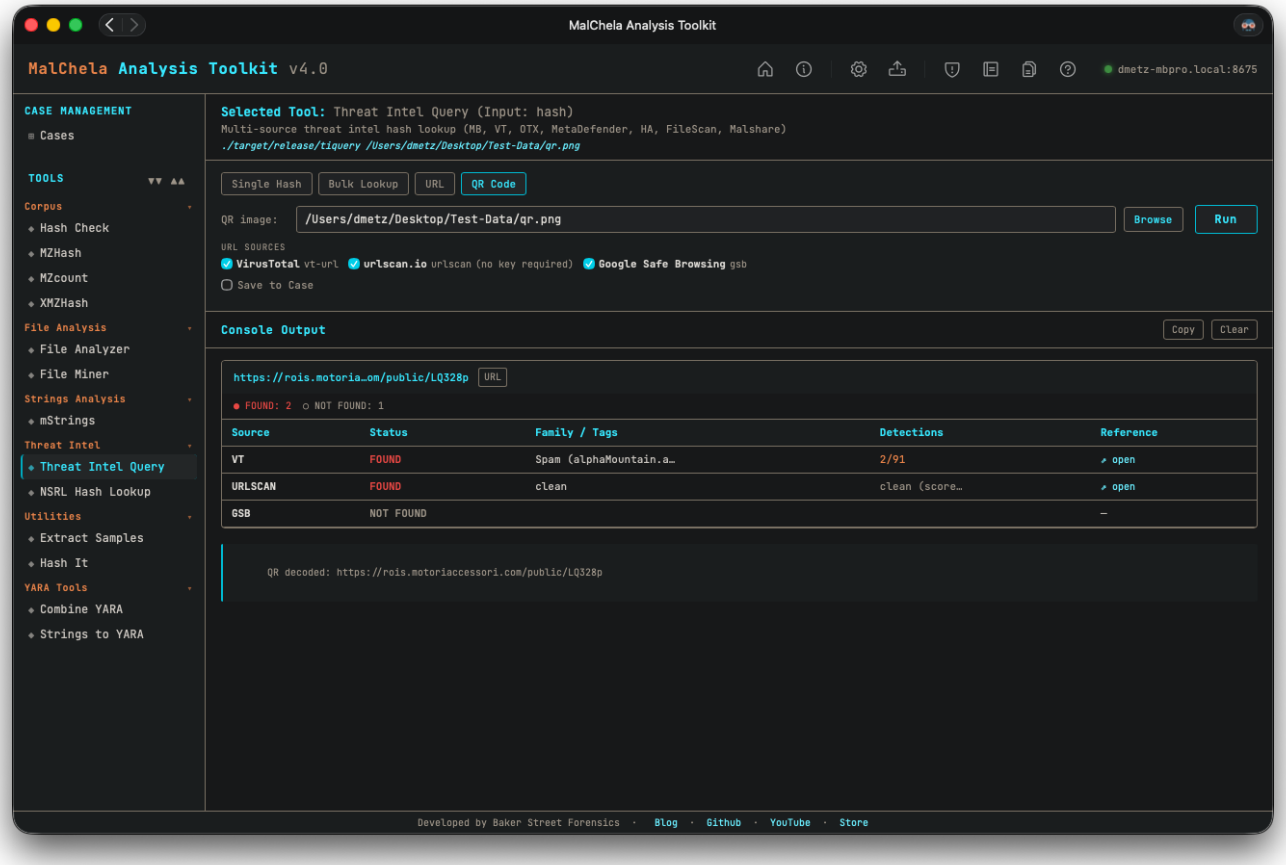
7.10 TiQuery

Threat Intel Query (`tiquery`) is a multi-source threat intelligence lookup tool. It queries multiple threat intelligence platforms in parallel and presents results in a unified table — giving analysts a fast, consolidated view of whether a hash, IP address, or URL is known, what family/category it belongs to, and how widely it has been detected.

`tiquery` accepts a **file hash** (MD5/SHA1/SHA256), an **IP address** (IPv4 or IPv6, defanged forms like `1[.]2[.]3[.]4` accepted), an **http/https URL**, or a **QR code image** as input and automatically routes the query to the appropriate set of sources. In the web interface, the Single Hash tab also includes a **Browse** button — pick any file and the SHA256 is computed and submitted automatically (the file itself is never uploaded, only its hash).



TIQuery Hash Lookup



TIQuery QR Code Lookup

Supported Sources — Hash Lookups

ID	Source	Tier	Key Required
mb	MalwareBazaar	1	Optional
vt	VirusTotal	1	Yes
otx	AlienVault OTX	1	Yes
md	MetaDefender	1	Yes
ha	Hybrid Analysis	2	Yes
fs	FileScan.IO	2	Yes
ms	Malshare	2	Yes
tr	Triage (RF)	2	Yes
os	Objective-See	2	No (SHA256 only)

Supported Sources — URL Lookups

ID	Source	Key Required
vt	VirusTotal (URL API)	Yes
urlscan	urlscan.io	Optional (raises rate limits)
gsb	Google Safe Browsing	Yes

Supported Sources — IP Lookups

ID	Source	Key Required	What it adds
vt	VirusTotal (IP API)	Yes	Engine detections, community reputation, AS owner, and country
otx	AlienVault OTX	Yes	Pulse count, AS, and country
gn	GreyNoise (Community API)	Yes (free key works)	Whether the IP is a known internet scanner ("noise") or a known benign business service ("RIOT": cloud, CDN, public DNS), and GreyNoise's classification. IPv4 only

Private, loopback, link-local, multicast, documentation, and other non-routable addresses are refused before any source is queried, so they never spend rate-limited quota. A free VirusTotal key allows 4 lookups a minute and 500 a day.

All sources with a configured API key are queried automatically. No flag is needed to enable Tier 2 sources — if the key file exists, the source is included. When the input is a URL or an IP address, only the sources for that input type are queried. Pass `--sources` to restrict the query to specific sources.

🔧 CLI Syntax

```
# Hash lookup — queries all hash sources with a configured API key
tiquery <hash>

# URL lookup — queries VirusTotal (URL), urlscan.io, and Google Safe Browsing
tiquery https://example.com/path

# IP lookup — queries VirusTotal (IP), AlienVault OTX, and GreyNoise
tiquery 185.220.101.4
tiquery 185[.]220[.]101[.]4

# Restrict to specific sources
tiquery <hash> --sources mb,vt,md,ha,fs
tiquery https://example.com --sources vt,urlscan

# Include per-engine VirusTotal detections (hash mode)
tiquery <hash> --verbose-vt

# Output as JSON
tiquery <hash> --json

# Output as CSV
tiquery <hash> --csv

# Save text report
tiquery <hash> -o -t

# Save report to a case folder
tiquery <hash> -o -t --case Case123

# Download sample from MalwareBazaar (SHA256 only)
tiquery <hash> -d

# Bulk lookup — process multiple hashes from a file
tiquery --bulk hashes.txt
tiquery --bulk hashes.csv --json
tiquery --bulk hashes.txt --csv

# QR code decode + URL lookup — extract URL from a QR image and query it
tiquery --qr screenshot.png
tiquery --qr qr_image.jpg --json
```

Accepts MD5, SHA1, and SHA256 hashes, or any `http://` / `https://` URL. If no input is provided, the tool will prompt interactively.

Bulk mode reads a `.txt` or `.csv` file and extracts all valid MD5/SHA1/SHA256 hashes (one per line, or mixed with other content). Each hash is queried sequentially with a short delay between requests to respect API rate limits. Results are displayed as a table per hash in human mode, or as a JSON array / CSV stream when `--json` / `--csv` is specified.

QR mode decodes a QR code image (PNG/JPG/GIF/BMP/WebP) and uses the extracted URL as input to the standard URL lookup pipeline. Useful for triaging quishing (QR phishing) samples from emails and screenshots without needing the web interface.

Output

Results are presented in a matrix showing source, status, malware family/tags, detection summary, and a reference link:

```
tiquery <hash> (SHA256)
________  ___________  ____________________  _____________  ____________________________________________
Source    Status       Family / Tags         Detections     Reference
________  ___________  ____________________  _____________  ____________________________________________
MB        FOUND        Emotet                ...            https://bazaar.abuse.ch/sample/...
VT        FOUND        Trojan.Emotet         58/72          https://virustotal.com/gui/file/...
OTX       FOUND                              4 pulses       https://otx.alienvault.com/...
MD        NOT FOUND                                         -
```

Saving Output

Use `-o` to save output and include one of the following format flags:

- `-t` → Save as `.txt`

When `--case` is used, output is saved to:

```
saved_output/cases/Case123/tiquery/
```

Otherwise, reports are saved to:

```
saved_output/tiquery/
```

API Keys

`tiquery` uses the same `api/` key file convention as other MalChela tools. Each source reads from its own file:

```
api/vt-api.txt        # VirusTotal (used for both hash and URL lookups)
api/mb-api.txt        # MalwareBazaar
api/otx-api.txt       # AlienVault OTX
api/md-api.txt        # MetaDefender
api/ha-api.txt        # Hybrid Analysis
api/fs-api.txt        # FileScan.IO
api/ms-api.txt        # Malshare
api/tr-api.txt        # Triage
api/url-api.txt       # urlscan.io (optional — raises rate limits)
api/gsb-api.txt       # Google Safe Browsing
api/gn-api.txt        # GreyNoise (IP lookups; a free Community key works)
```

Keys can be managed via the **API Keys** panel in the MalChela web interface (Configuration menu → API Keys) or by placing the key directly in the appropriate file.

See API Configuration for details.

Note that `os` (Objective-See) has no API-key gate of its own — it queries a locally cached copy of the Objective-See malware catalogue (refreshed every 24h) and always attempts that fetch on a cache miss, regardless of which other keys are configured. In an air-gapped environment, enable Offline Mode to skip every source's network call, keyed or not.

Bulk Lookup

Bulk lookup processes a `.txt` or `.csv` file containing hashes (one per line, or mixed with other content — the tool extracts valid MD5/SHA1/SHA256 values automatically) and runs all lookups in a single operation.

CLI: use the `--bulk <file>` flag (see CLI Syntax above). Results are printed as a table per hash, or as a combined JSON array / CSV when `--json` / `--csv` is specified.

Web interface: the TIquery panel's **Bulk Lookup** tab lets you browse to a file and run all lookups in one click. Results are displayed in a consolidated grid.

Web Interface Input Modes

The MalChela web interface's Threat Intel Query panel offers five input modes, selected via tabs at the top of the panel:

- **Single Hash** — paste a hash or click **Browse...** to pick a file; the web interface computes the file's SHA256 and populates the hash field automatically.
- **Bulk Lookup** — as described above, for batches of hashes in a file.
- **URL** — paste an `http://` or `https://` URL to query VirusTotal, urlscan.io, and Google Safe Browsing in parallel.
- **IP** — paste an IPv4 or IPv6 address (defanged is fine: `1.2.3[.]4`) to query VirusTotal, AlienVault OTX, and GreyNoise. Because an IP being *known* to a source isn't a verdict (8.8.8.8 is in every one), each row shows what that source concluded instead of FOUND: **FLAGGED** (VirusTotal engines or a GreyNoise malicious scanner), **IN *N* PULSES** (OTX: discussed, which is context), or **CLEAN** / **KNOWN BENIGN** (no engine detections, or a GreyNoise RIOT business service such as a CDN or public DNS).
- **QR Code** — pick a QR code image (PNG/JPG/GIF/BMP/WebP); the web interface decodes the embedded URL and populates the URL field for review, then runs the standard URL lookup pipeline. Useful for triaging "quishing" (QR phishing) samples from emails and screenshots.

In URL, IP, and QR modes, source checkboxes auto-enable based on which API keys are configured, exactly as in hash mode.

7.11 MStrings

mStrings extracts strings from files and classifies them using regular expressions, YARA rules, and MITRE ATT&CK mappings. It highlights potential indicators of compromise and suspicious behavior, grouping matches by tactic and technique. Ideal for quickly surfacing malicious capabilities in binaries, scripts, and documents.

Also accepts macOS Mach-O binaries and `.app` bundles — when given a bundle, the main executable is auto-resolved via `Info.plist`'s `CFBundleExecutable`, falling back to the sole binary in `Contents/MacOS/` if that lookup fails.

Note: The MITRE Technique Lookup bar, introduced in v3.0.1 has been removed. It has been replaced with a full MITRE lookup utility (no internet required.)

IOC Extraction

Alongside the Sigma-style detection matches, mStrings pulls out well-formed **Potential Filesystem IOCs** and **Potential Network IOCs** into their own report sections — paths, URLs, domains, and IPs found either as standalone strings or embedded in a longer one (a command line, a query string). Reserved/documentation IP ranges (loopback, broadcast, RFC 5737 test ranges, and similar) are excluded automatically, since they show up constantly in any binary's networking stack but are never real indicators.

Network IOCs are shown defanged (`http://` → `hxxp://`, every `.` → `[.]`) everywhere mStrings displays them — CLI output, saved reports, and the Analyze rollup that embeds them. This is deliberate: a raw URL rendered as clickable-looking text is the wrong thing to put in front of an analyst repeatedly. If you need the real string back — to build a YARA rule, for example — Strings to YARA automatically refangs any defanged line before it becomes a rule.

IP Address Lists

When ten or more IP addresses are stored together, each as a string of its own, within a few hundred bytes of each other (a tuple in a Python stealer, a table in a binary's data section), mStrings reports them as one **IP Address List** instead of as individual Network IOCs. Such a run is most often an anti-analysis blocklist of sandbox and analysis-host IPs the sample checks for before it runs (Luna Grabber carries 59), and sometimes a pool of C2 servers. Either way, it isn't 59 separate C2 indicators. The list keeps every address (defanged) and its offset range, so you can look the addresses up or check the code around them. An IP inside a longer string, such as a command line, always stays a standalone Network IOC.

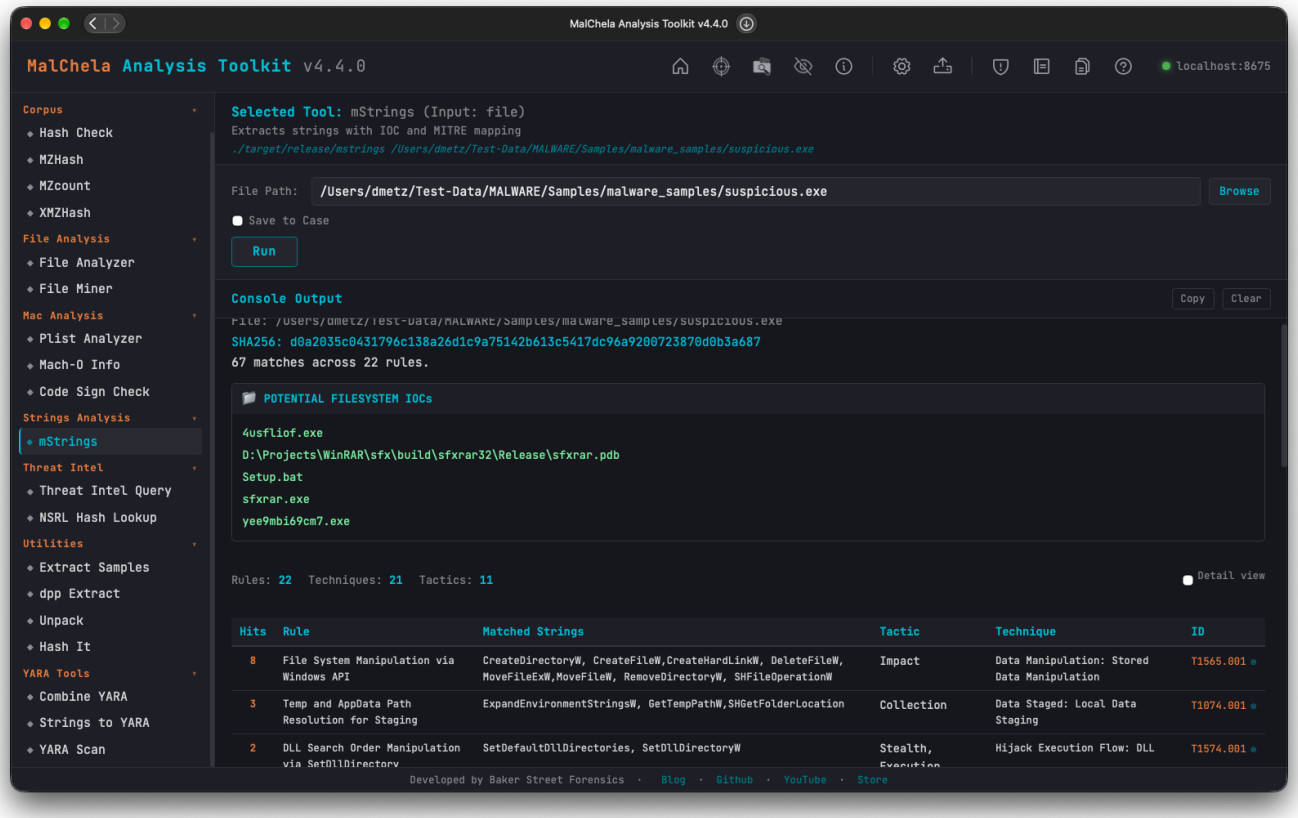
Public DNS resolvers (Google, Cloudflare, Quad9, and others) are also listed separately, as **Public DNS Resolvers Referenced (not C2)**.

Multi-Layer Obfuscation

mStrings recursively decodes base64 content it finds — including base64 nested inside base64, up to 12 layers deep — and re-runs detection against whatever it unwraps to. When a detection only fires after peeling **more than one** layer, that's flagged directly on the finding (`⚠ found after N layers of base64 decoding` in the summary table, `Base64×N` in the detail table's encoding column) — a single decode is routine, but deliberate re-encoding multiple times over is itself an evasion signal worth noting.

Encoded String Tables

Some malware keeps its entire payload as hundreds or thousands of encrypted, base64-alphabet strings and decodes them at runtime — the AMOS stealer stores its whole AppleScript this way, about 1,900 strings in one section. Per-string matches are noise at that scale, and the real behavior never appears as plaintext, so no other rule fires. When the large-base64-blob rule matches 25 or more distinct strings, mStrings adds one **Encoded string table** row instead: how many strings, their total size, their share of the file, and the offset range they span, mapped to T1027.013 (Encrypted/Encoded File). Treat it as a pointer to dynamic analysis or a family-specific decoder rather than a dead end.



MStrings

🔧 CLI Syntax

```
# Example 1: Scan a file
cargo run -p mstrings -- /path_to_file/

# Example 2: Save output as .txt
cargo run -p mstrings -- /path_to_file/ -o -t

# Example 3: Save output to a case folder
cargo run -p mstrings -- /path_to_file/ -o -t --case CaseXYZ

# Example 4: Scan a .app bundle (main executable resolved automatically)
cargo run -p mstrings -- /path/to/Sample.app
```

Use `-o` to save output and include one of the following format flags: - `-t` → Save as `.txt` - `-j` → Save as `.json` - `-m` → Save as `.md`

If no file is provided, the tool will prompt you to enter the path interactively.

When `--case` is used, output is saved to:

```
saved_output/cases/CaseXYZ/mstrings/
```

Otherwise, results are saved to:

```
saved_output/mstrings/
```

The MITRE Lookup feature is only available in the web interface version of mStrings.

The MITRE Reference Panel provides a built-in way to explore MITRE ATT&CK techniques directly within MalChela.

This panel supports live search by **technique ID** (e.g., `T1059`) or **keyword** (e.g., "scripting"). It is context-aware and optimized for use alongside tools like `mstrings`.

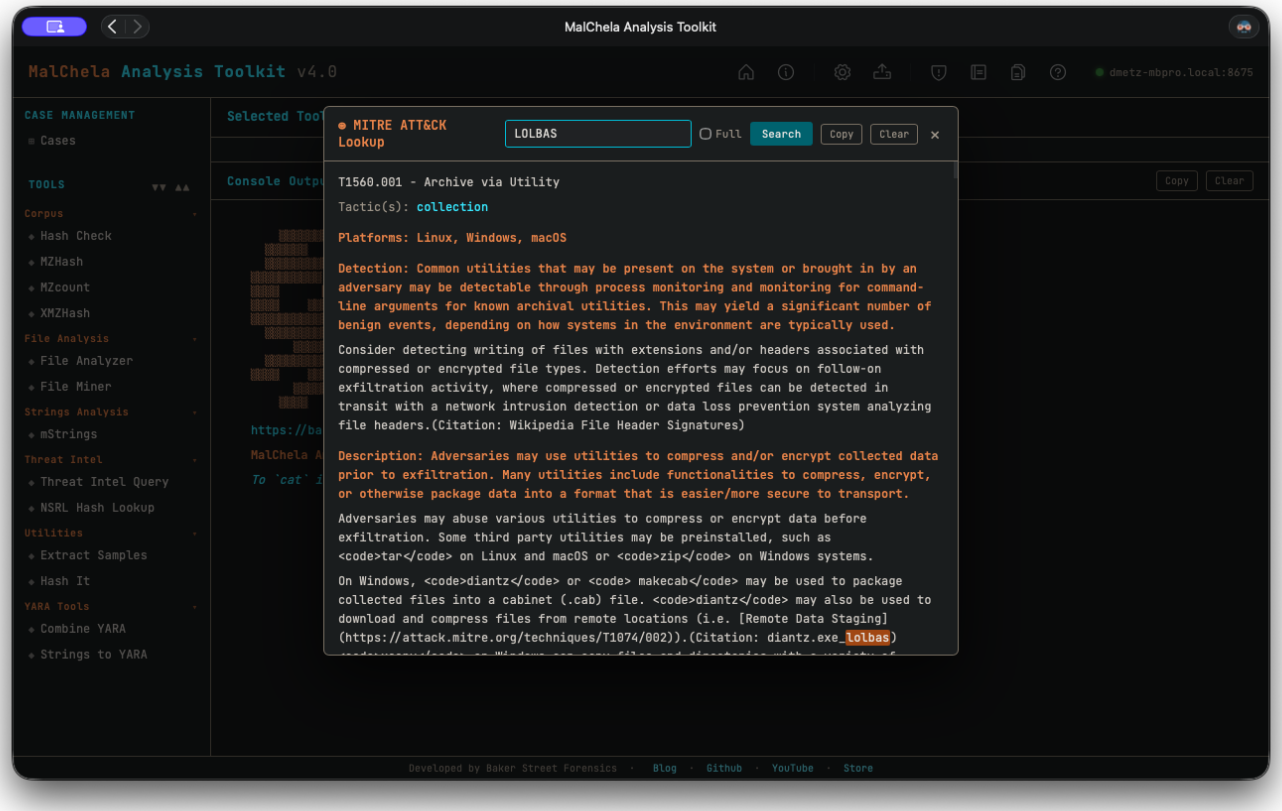
7.12 MITRE Lookup Panel

7.12.1 Features

- Full display of technique details:
- **Technique title**
- **Tactic(s)**
- **Platforms**
- **Description**
- **Detection guidance**
- **Malware**
- **Tools**
- **Intrusion Sets**
- Modal view floats above the main console, allowing uninterrupted output viewing
- --full option to support long-form content (no truncation)
- Save lookups for later reference or save to a Case
- Markdown formatting for MITRE reports for inclusion in forensics reports

7.12.2 Accessing from the MalChela Web Interface:

- Select **MITRE Lookup** in the left-hand **Toolbox** menu
- Use the input field at the top of the modal to enter a keyword or technique ID (e.g., `T1059` or `registry`)
- Use the "Full" checkbox for un-truncated output



MITRE Lookup

7.12.3 Accessing from CLI

There are two ways to use the MITRE Lookup from the command line:

1. From the main MalChela CLI menu

Run the MalChela CLI interface and select `MITRE Lookup` from the Toolbox section:

```
cargo run -p malchela
```

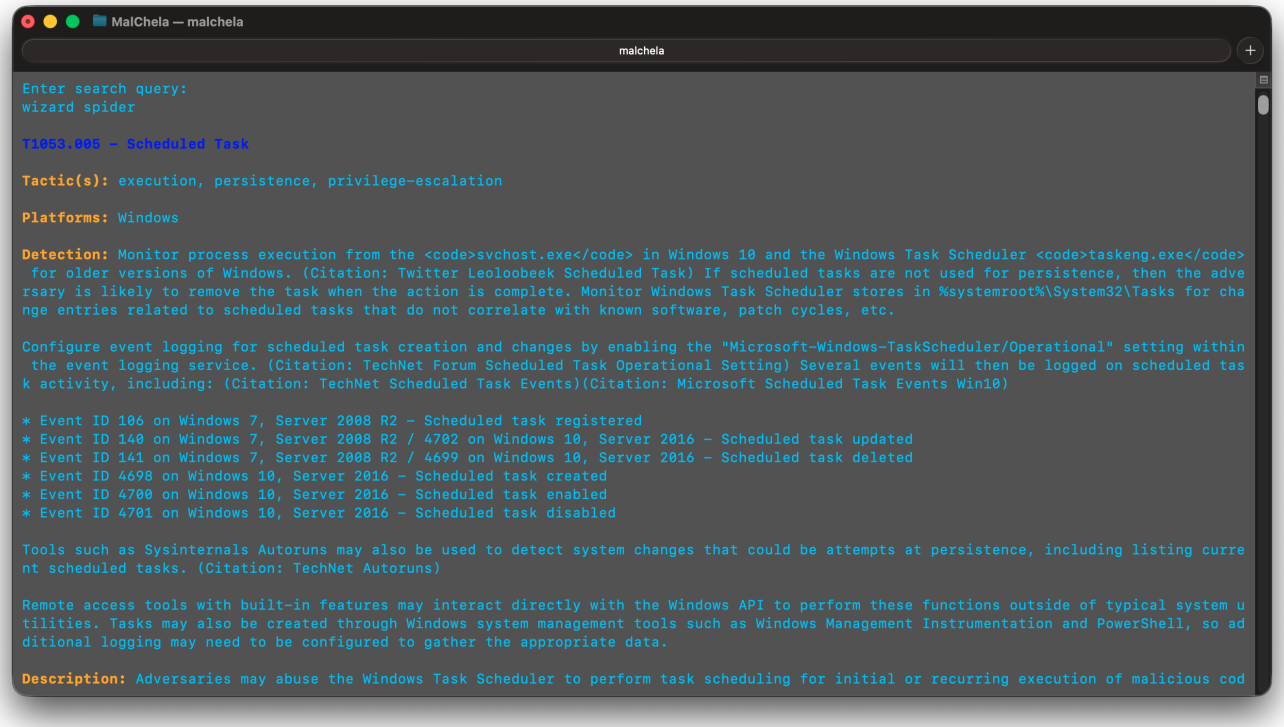
Then choose `MITRE Lookup` (or press `ml`) from the menu.

2. Direct tool execution

You can run the MITRE Lookup tool directly with a search term or flags:

```
# Basic search by keyword or technique ID
./target/release/MITRE_lookup -- t1027.004

# Use --full to display untruncated results
./target/release/MITRE_lookup --full -- t1027.004
```



MITRE Lookup (CLI)

7.12.4 Example

Searching for `T1027.004` yields:

```
MITRE ATT&CK Enterprise v20.0

T1027.004 - Compile After Delivery
Tactic(s): stealth
Platforms: Linux, macOS, Windows
Detection: Detection strategies: Detection Strategy for Compile After Delivery - Source Code to Executable Transformation (DET0501)
Description: Adversaries may attempt to make payloads difficult to discover and analyze by delivering files to victims as uncompiled code. Text-based...
```

7.12.5 Tactics

Tactics have their own entries. Look one up by ID (`TA0002`), by name (`execution` , `command and control`), or by shortname (`command-and-control`) to see its description and every technique under it, with sub-technique counts:

```
TA0002 - Execution (tactic)
Description: The adversary is trying to run malicious code or instructions on a system. ...
Techniques (22 techniques, 44 sub-techniques):
- T1047 Windows Management Instrumentation
- T1053 Scheduled Task/Job (+5 sub-techniques)
- T1059 Command and Scripting Interpreter (+13 sub-techniques)
...
```

- `tactics` lists all 15 Enterprise tactics in matrix order, with technique counts.
- `--full` expands every sub-technique under its parent.
- `TA0005` or `defense evasion` shows both Stealth (TA0005) and Defense Impairment (TA0112), with a note on the v20 split.

Tactic data lives in `assets/mitre_tactics.json` , which `scripts/build_mitre_techniques.py` writes alongside the technique data.

7.12.6 ATT&CK Version

The lookup data (`assets/mitre_techniques.json`) is built from MITRE's official ATT&CK STIX release, currently **Enterprise v20.0**. The version is shown at the top of every lookup.

- **ATT&CK v20 tactic change:** the Defense Evasion tactic was retired. TA0005 is now **Stealth** (hiding, obfuscation, masquerading), and a new tactic, **Defense Impairment** (TA0112), covers disabling or tampering with security tools, logging, and firewalls. Searching `defense evasion` (or `TA0005`) returns techniques from both, with a note. Tactic searches ignore hyphens, so `defense impairment` matches `defense-impairment` .
- **Deprecated techniques still resolve:** looking up a retired ID such as `T1562.001` shows it marked deprecated, with its replacement (`T1685 Disable or Modify Tools`). Current techniques are listed before deprecated ones.
- **Detection field:** ATT&CK v18 replaced per-technique detection text with Detection Strategies, so for current techniques the Detection line lists the strategy names and IDs (`DET…`).

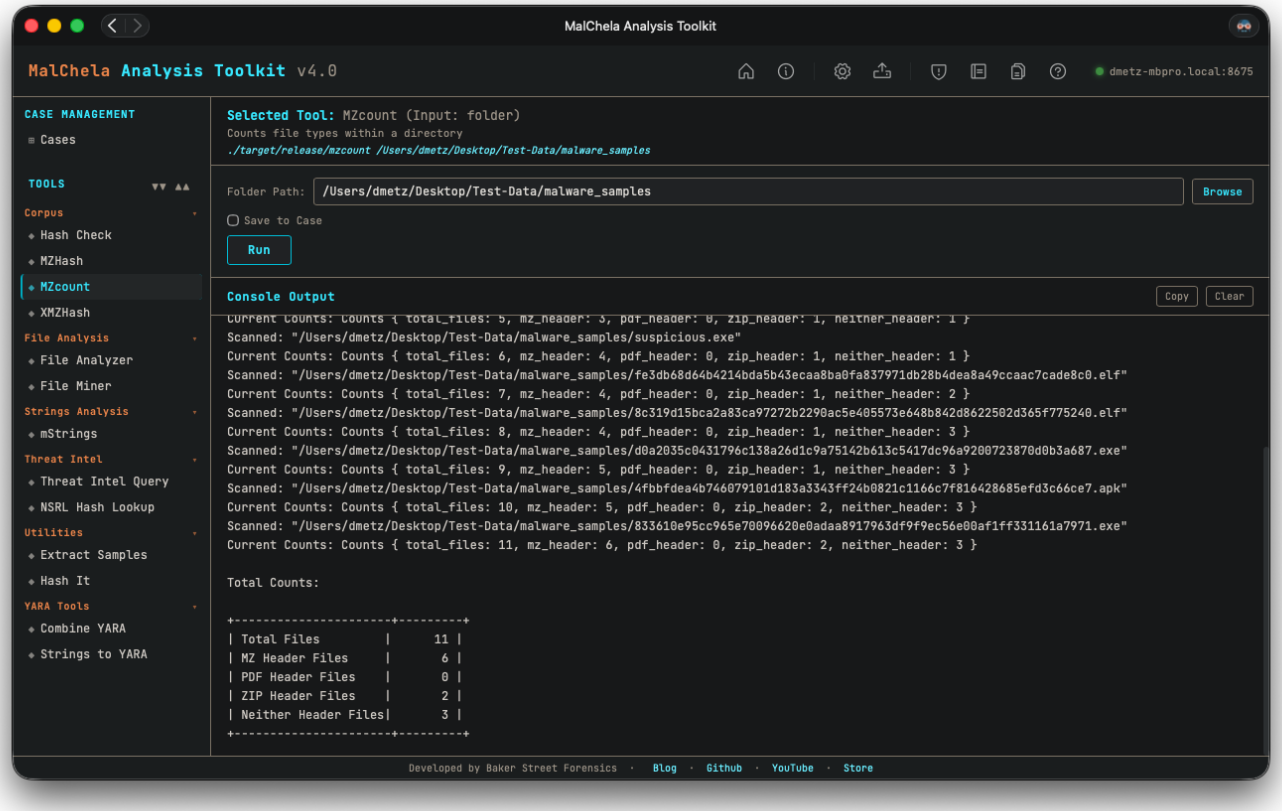
Updating to a new ATT&CK release

```
# Rebuild the technique and tactic data from the latest Enterprise release (or --version 20.0, or --bundle <file> offline)
python3 scripts/build_mitre_techniques.py

# Then migrate detections.yaml mappings (tactics, renamed/revoked techniques, tags),
# passing the previous dataset so author-chosen tactics are preserved
git show HEAD:assets/mitre_techniques.json > /tmp/mitre_old.json
python3 scripts/migrate_detections_attack.py --old /tmp/mitre_old.json --dry-run   # review
python3 scripts/migrate_detections_attack.py --old /tmp/mitre_old.json
```

7.13 MZCount

MZcount recursively scans a directory and counts the number of files that match key signatures like MZ (Windows executables), ZIP, PDF, and others. It uses lightweight YARA rules to classify files by type, giving a quick overview of the content breakdown within a dataset. Results can be displayed in either a detailed per-file view or a clean summary table, depending on your analysis needs.



MZCount

🔧 CLI Syntax

```
# Example 1: Scan a directory and view results in terminal
cargo run -p mzcount -- /path_to_scan/

# Example 2: Enable table mode
MZCOUNT_TABLE_DISPLAY=1 cargo run -p mzcount -- /path_to_scan/

# Example 3: Save results as .txt
cargo run -p mzcount -- /path_to_scan/ -- -o -t

# Example 4: Save results to a case folder
cargo run -p mzcount -- /path_to_scan/ -- -o -t --case CaseName
```

If no path is provided, the tool will prompt you to enter it interactively.

Use `-o` to save output and `-t` to specify plain text format.

When `--case` is used, output is saved under:

```
saved_output/cases/CaseName/mzcount/
```

Otherwise, output is saved under:

```
saved_output/mzcount/
```

7.14 MZHash

Note: `MZHash` replaces the deprecated `MZMd5`.

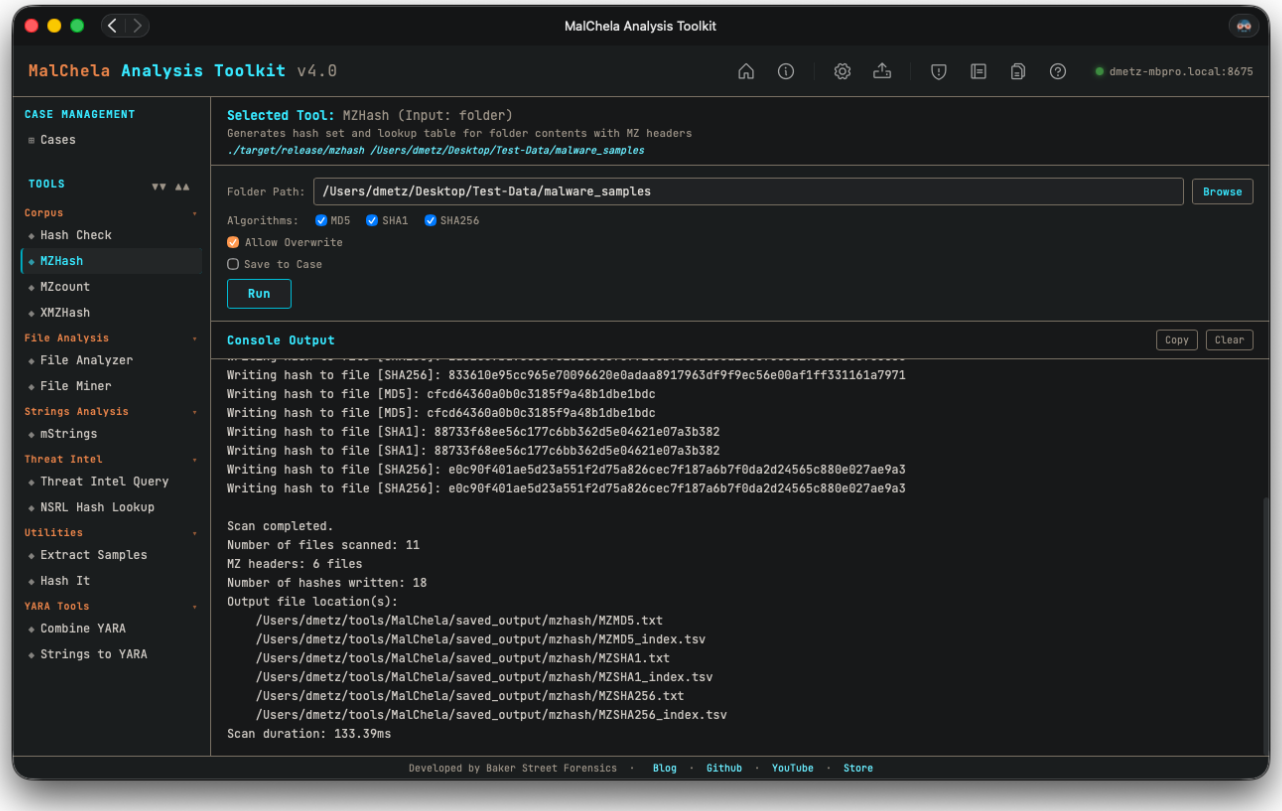
`MZHash` recursively scans a folder and generates hashes for all files that begin with the MZ header — the signature of Windows PE executables. It's useful for building known-good or known-bad hash sets during triage, reverse engineering, or threat hunting.

You can select one or more hash algorithms at runtime: **MD5**, **SHA1**, or **SHA256**. If multiple algorithms are selected, a hash file and TSV lookup table will be generated for each.

The program outputs: - A text file with one hash per line. - A TSV (tab-separated values) file with full file paths and their corresponding hashes.

By default, hashes are saved to `saved_output/mzhash/`. If the file already exists, the user will be prompted before overwriting.

These hash sets (.tsv preferred) can be used with HashCheck.



MZHash

🔧 CLI Syntax

```
# Example 1: Generate default SHA256 hashes
cargo run -p mzhash -- /path_to_directory/

# Example 2: Generate all three hash types (MD5, SHA1, SHA256)
cargo run -p mzhash -- /path_to_directory/ -a MD5 -a SHA1 -a SHA256

# Example 3: Generate SHA1 and SHA256 only
cargo run -p mzhash -- /path_to_directory/ -a SHA1 -a SHA256

# Example 4: Save output to a case folder
cargo run -p mzhash -- /path_to_directory/ -a SHA256 --case CaseName
```

If `--case` is specified, output will be saved to:

```
saved_output/cases/CaseName/mzhash/
```

Otherwise, hashes are saved to:

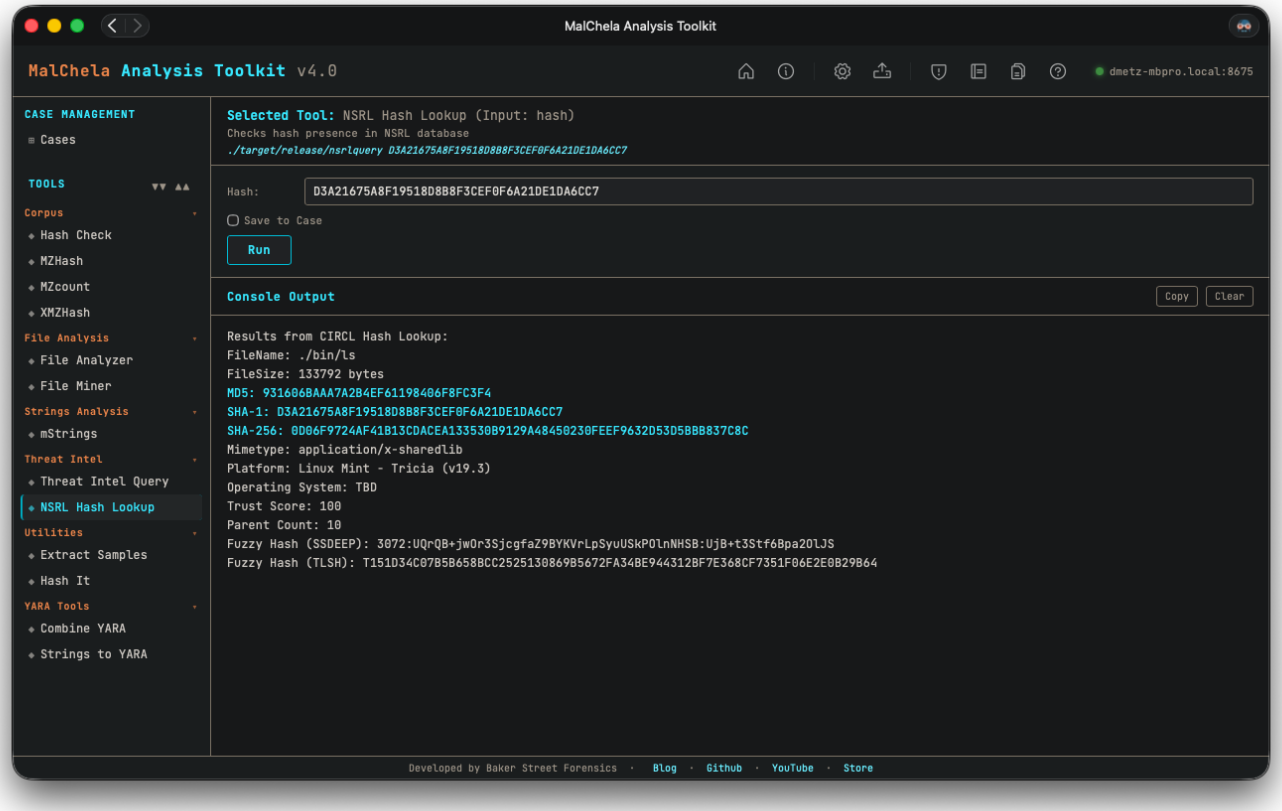
```
saved_output/mzhash/
```

You can combine multiple `-a` flags in any order.

7.15 NSRLQuery

NSRL Query checks a file hash against the National Software Reference Library (NSRL) by querying the CIRCL hash lookup service. It helps identify known, trusted software — allowing analysts to filter out benign files and focus on unknown or suspicious ones during forensic triage.

This is a live network lookup with no API key involved — there's no local NSRL database file to fall back on. In an air-gapped environment, enable Offline Mode to skip the network call cleanly instead of letting it fail.



NSRL Hash Lookup

🔧 CLI Syntax

```
cargo run -p nsrlquery -- -o -t d41d8cd98f00b204e9800998ecf8427e
```

Performs a lookup using the CIRCL hashlookup API and saves the result as a `.txt` file.

Use `-o` to save output and include one of the following format flags: - `-t` → Save as `.txt` - `-j` → Save as `.json` - `-m` → Save as `.md`

If no hash is provided, the tool will prompt you to enter one interactively:

```
Enter the hash value:
```

Only MD5 and SHA1 hashes are supported. If an unsupported hash length is entered:

```
Error: Unsupported hash length. Please enter a valid MD5 (32 chars) or SHA1 (40 chars) hash.
```

To associate output with a case folder:

```
cargo run -p nsrlquery -- -o -j --case APT2025 d41d8cd98f00b204e9800998ecf8427e
```

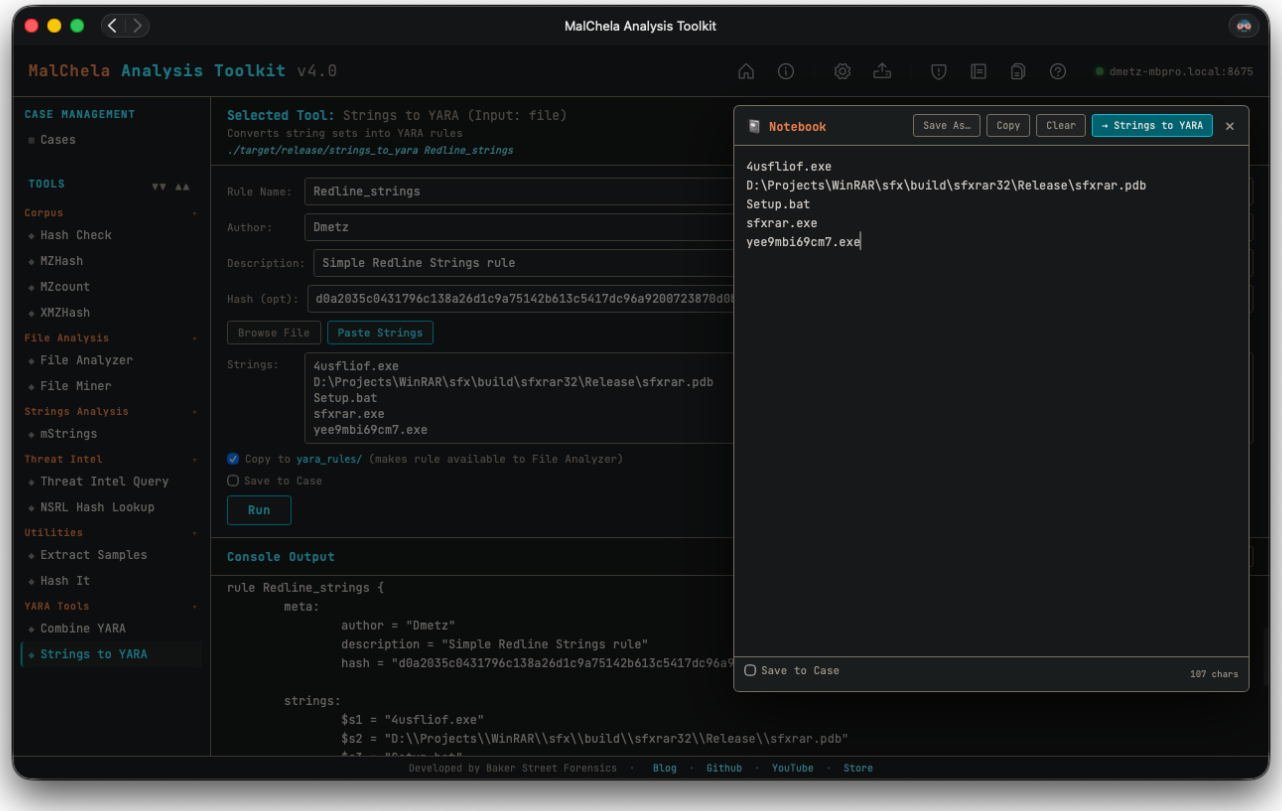
This saves the `.json` output in `saved_output/cases/APT2025/nsrlquery/`.

7.16 StringsToYARA

Strings to YARA helps you rapidly build custom YARA rules by prompting for a rule name, optional metadata, and a list of string indicators. It integrates with the MalChela scratchpad, allowing you to paste or collect candidate strings interactively.

Lines beginning with hash: are deliberately ignored during rule generation — this lets you use the scratchpad to track hashes alongside strings without polluting your YARA rule content.

Any **defanged** line (`hxxp://` , `hxxps://` , `fxxp://` , or `[.]` in place of a literal dot — the same defanging mStrings and the Analyze rollup use when displaying network IOCs) is automatically refanged back to the real string before it's written into the rule. YARA matches raw bytes in the target file, which contain the real string, never the defanged display form — so a URL copied straight out of a report generates a rule that actually matches something, without you having to manually fix it up first. Lines that were never defanged pass through unchanged.



Strings to YARA

🔧 CLI Syntax

```
cargo run -p strings_to_yara -- RuleName Author Description Hash /path/to/strings.txt --case CaseName
```

You can supply up to five positional arguments. If any are omitted, the tool will prompt you interactively.

```
Enter rule name:
Enter author:
Enter description:
Enter hash (optional):
Enter path to string list file:
```

If the `--case` flag is supplied, the resulting YARA rule will be saved in the corresponding `saved_output/cases/CaseName/` directory.

Lines in the string file that begin with `hash:` are ignored and will not be included in the generated rule.

7.17 Unpack

Unpack detects packers, protectors, installers, and bundlers in PE, ELF, and Mach-O files, and statically recovers the payload wherever that's possible without running the sample. Every detection is reported with the evidence behind it (section names, magic offsets, structural anomalies), so the call can be checked rather than trusted, and every recovered payload is fed back through Unpack again — a dropper inside an installer inside a UPX stub comes out as a layer tree, not just the outermost layer.

Static only. Unpack never executes or emulates the sample. Where a packer can only be removed by running its stub (Themida, VMProtect, Enigma, custom crypters), Unpack identifies it, shows the evidence, and says so.

What It Handles

Detection	Formats	Recovery	Needs
UPX	PE, ELF, Mach-O	Decompressed with `upx -d` on a working copy	`upx`
UPX, tampered	ELF, PE	Headers repaired first, then `upx -d` — see below	`upx`
Mach-O universal (fat)	Mach-O	Split into one thin binary per architecture	—
PyInstaller	PE, ELF, Mach-O	CArchive extracted natively; entry scripts rebuilt as `.pyc` with the correct header	—
Python bytecode (`.pyc`), including obfuscated	`.pyc` (CPython 3.7–3.14)	Parsed natively; common obfuscation layers peeled; names, function names, and string constants written to `<name>_pyc_strings.txt` for mStrings — see below	—
.NET embedded resources / Costura.Fody	.NET PE	ManifestResources extracted natively; `costura.*.compressed` assemblies deflate-decompressed	—
NSIS installer	PE	Extracted with 7-Zip	`7zz` / `7z`
Inno Setup installer	PE	Extracted with innoextract	`innoextract`
Overlay	PE, ELF, Mach-O	Appended data carved out (archives, executables, or high-entropy blobs)	—
Embedded executable	any	Structurally validated PE/ELF/Mach-O images carved out of the file	—
Named packers / protectors	PE	Identified only: ASPack, MPRESS, PECompact, Petite, NsPack, RLPack, MEW, kkrunchy, Themida, WinLicense, VMProtect, Enigma, and others	—
.NET obfuscators	.NET PE	Identified only: ConfuserEx, SmartAssembly, Dotfuscator, Babel, and others	—
shc-compiled shell script	ELF, Mach-O	Identified only	—
Unknown packer (heuristic)	PE, ELF, Mach-O	Identified only: high-entropy code, minimal imports, W+X sections, entry point outside the first code section	—
Go binary	PE, ELF, Mach-O	Informational — not packing, but a hint to use GoReSym	—

The three external helpers are optional. Everything else is native Rust and works the same on macOS, Linux, and Windows. Check what's available with:

```
./target/release/unpack --check
```

Helper	macOS	Linux	Windows	Override
UPX	`brew install upx`	`apt install upx-ucl`	`choco install upx`	`MALCHELA_UPX`
7-Zip	`brew install sevenzip`	`apt install 7zip`	`choco install 7zip`	`MALCHELA_7Z`
innoextract	`brew install innoextract`	`apt install innoextract`	`choco install innoextract`	`MALCHELA_INNOEXTRACT`

None of the helpers execute the sample — they only parse and decompress it.

Tampered UPX

Commodity malware — Mirai and its descendants especially — routinely damages UPX headers so that `upx -d` refuses the file while the stub still runs. Unpack detects and reverses the common tricks on a working copy (the original is never modified), and records every byte it changed in the report:

- **ELF:** the `UPX!` magic replaced with another four-byte value everywhere it appears (in `l_info` and the trailing pack header), and/or `p_info.p_filesize` / `p_blocksize` zeroed. Unpack restores the magic and rebuilds `p_info` from the trailing pack header's original file size — including big-endian targets (MIPS, PowerPC), whose pack header is itself big-endian.
- **PE:** the `UPX0` / `UPX1` sections renamed, and the pack header magic wiped. Unpack recognises UPX's section layout (an empty first section with a large virtual size, followed by a high-entropy section holding the entry point), renames the sections back, and restores the pack header magic after UPX's version string.

If the repair isn't enough, the report shows the repairs attempted and `upx`'s own error.

CLI Syntax

```
# Unpack next to the input, into <name>_unpacked/
./target/release/unpack /path/to/sample

# Choose the output directory
./target/release/unpack /path/to/sample --out /path/to/output

# Save under a case (payloads go to saved_output/cases/Case123/unpack/<name>_unpacked/)
./target/release/unpack /path/to/sample --case Case123 -o -m

# Machine-readable summary on stdout (used by Analyze, the web interface, and MCP)
./target/release/unpack /path/to/sample --json
```

```
# Detection and native handlers only — never call upx / 7-Zip / innoextract
./target/release/unpack /path/to/sample --no-external
```

Option	Default	Purpose
`--out DIR`	`<input>_unpacked`	Where recovered payloads are written
`--case NAME`	—	Route payloads and the saved report into a case
`--json`	—	JSON summary on stdout (verdict, payloads, MITRE IDs, full layer tree)
`-o` with `-m` / `-t` / `-j`	Markdown	Save the report (`saved_output/unpack/` or the case's `unpack/` folder)
`--max-depth N`	3	Maximum number of layers to unpack
`--max-output-mb MB`	512	Cap on total bytes written across the run (zip-bomb guard)
`--no-external`	—	Skip external helpers
`--only-if-found`	—	With `-o`, only save a report when something was detected
`--check`	—	List the external helpers that were found

Recovered payloads are deduplicated by SHA256, archive-supplied names are sanitised (no absolute paths or `..`), and an empty output directory is removed when nothing was recovered.

The Report

The Markdown report contains:

- **Packer verdict** — one line naming what was found and how many payloads were recovered (Analyze's rollup surfaces this line in its Sample Snapshot).
- **Detections** — every verdict at every layer, with type, confidence, and evidence, plus a note on what to do next where no static path exists.
- **Actions** — which handler ran, its status (`ok`, `partial`, `failed`, or `skipped`), and exactly what it did, including any header repairs.
- **Recovered Payloads** — path, format, size, SHA256, and how each one was produced.
- **Layer Tree** — the full parent → child chain, for example:

```
dropper.exe [PE32 (x86)] — NSIS installer
└── payload.exe [PE32+ (x86_64)] — UPX (tampered)
    └── payload_upx.exe [PE32+ (x86_64)]
```

- **Section Entropy** for the input, with high-entropy sections marked.
- **MITRE ATT&CK** — `T1027.002` (Software Packing) when a packer, protector, or crypter is detected, and `T1027.009` (Embedded Payloads) when an installer, bundler, overlay, or embedded executable is found.

Analyze Integration

Analyze runs Unpack automatically on every executable File Miner finds. When Unpack recovers payloads, each one is classified by File Miner and analysed with its own suggested tools — so File Analyzer, mStrings, and Threat Intel Query see the real payload, not just the packer stub. In the MalChela Summary, the packed file's section includes the Unpack report, the Sample Snapshot lists it under **Packing / Obfuscation**, the Component Inventory lists every recovered payload, and each payload's section is marked **Unpacked from** its parent. Clean executables add nothing to the rollup or the case.

File Analyzer's **Packed** field uses the same detector, and names the packer when one is found.

Python Bytecode and Obfuscators

Python stealers (Luna Grabber, Blank Grabber, and many others) ship as PyInstaller bundles, and the real program is the entry script, a `.pyc`. It's usually obfuscated, so a strings pass over it shows almost nothing. BlankOBF-style builds wrap the code in layers: an xz or zlib blob in the module's constants, Python source holding base64 split into pieces (one rot13'd, some reversed), and finally a marshalled code object.

Unpack parses CPython's marshal format itself and follows those layers to the innermost code object. It never imports, compiles, unmarshals, or runs anything in a Python interpreter. Peeled layers handled: xz, zlib, gzip, and LZMA blobs; base64 (as is, reversed, or rot13'd); BlankOBF's split-and-reverse concatenation; and nested `.pyc` or marshal data.

It writes:

- `<name>_pyc_strings.txt`: every code object's function name, the names it uses, and its string constants (byte-tuple names decoded), innermost layer first. This is the payload Analyze runs mStrings on, so the matrix and IOCs come from the stealer's own code.
- `<name>_layerN_source.py.txt` and `<name>_layerN.marshal`: each intermediate layer, for manual review. The source layers are saved as `.txt` so they can't be run by double-clicking.

An obfuscated `.pyc` is reported as **Obfuscated Python bytecode** (Crypter) and tagged T1027.013 Encrypted/Encoded File, not Software Packing. A plain `.pyc` is informational, and its strings file is still written.

Point Analyze at a standalone `.pyc` and File Miner routes it to Unpack and mStrings the same way.

Limits

- Static only: protectors and custom crypters that decrypt in memory are identified, not unpacked. For those, use emulation (e.g. unipacker, Speakeasy) or a sandbox / memory dump — MalChela's Volatility 3 integration can take it from there.
- UPX on Mach-O relies on `upx` supporting that particular binary; tampered-header repair is implemented for ELF and PE.
- PyInstaller's `PYZ` archive is extracted as a single `.pyz` file; individual library modules inside it are not unpacked. The author's entry script(s) — what you usually want — are recovered as standalone `.pyc` files.
- `.pyc` parsing covers CPython 3.7–3.14. Bytecode is not decompiled: you get names and constants, not control flow. PyArmor and other obfuscators that encrypt code objects with a runtime key are not peeled.
- shc-compiled scripts are detected, but not yet decrypted.

7.18 XMZHash

Note: `XMZHash` replaces the deprecated `XMZMd5`.

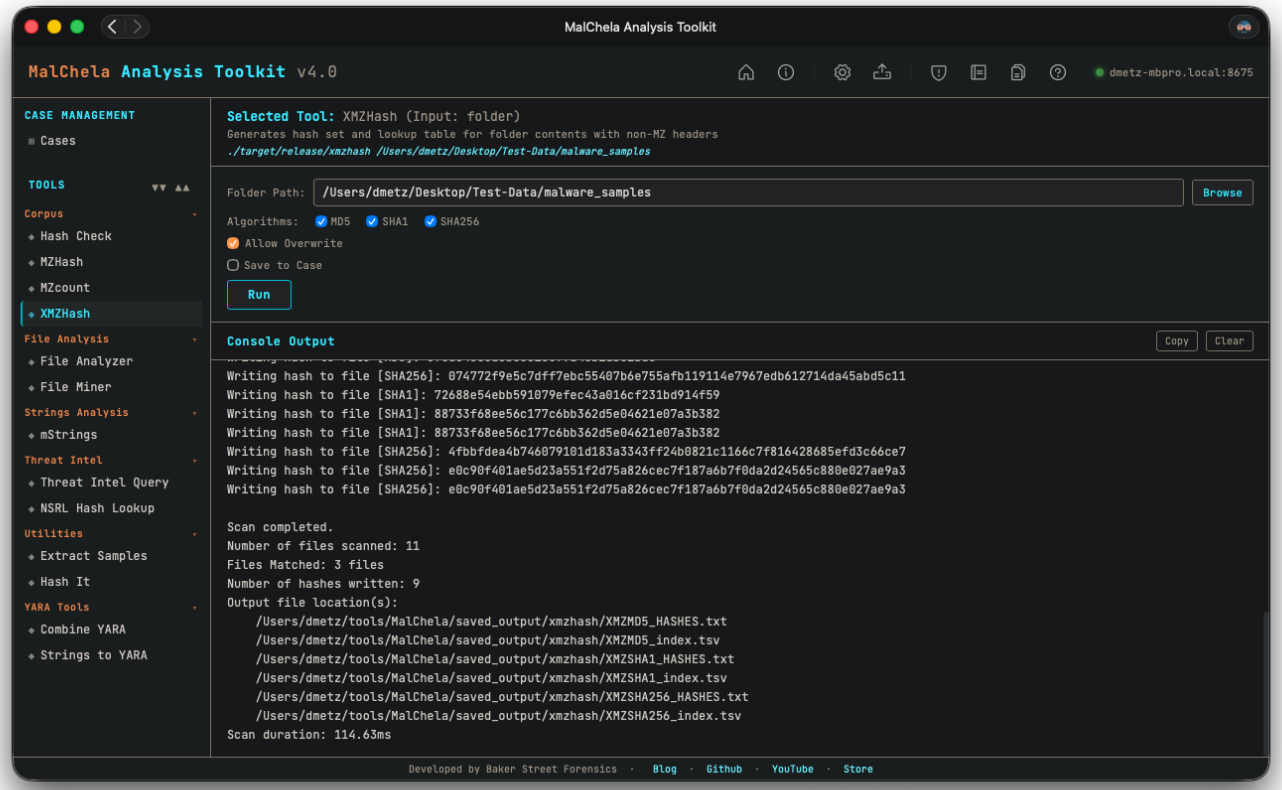
`XMZHash` recursively scans a directory and generates hashes for all files that **do not** match common binary or archive signatures such as MZ, ZIP, or PDF. It's ideal for uncovering unusual or misclassified files that may require deeper inspection or reverse engineering. Use this on a malware corpus to help surface non-Windows malware samples.

You can select one or more hash algorithms at runtime: **MD5**, **SHA1**, or **SHA256**. If multiple algorithms are selected, a hash file and TSV lookup table will be generated for each.

The program outputs: - A text file with one hash per line. - A TSV (tab-separated values) file with full file paths and their corresponding hashes.

By default, hashes are saved to `saved_output/mzhash/`. If the file already exists, the user will be prompted before overwriting.

These hash sets (.tsv preferred) can be used with HashCheck.



XMZHash

🔧 CLI Syntax

```
cargo run -p xmzhash -- /path_to_directory/
```

Generates SHA256 hashes (default).

```
cargo run -p xmzhash -- -a MD5 -a SHA1 -a SHA256 /path_to_directory/
```

Generates all three hash types.

```
cargo run -p xmzhash -- -a SHA1 -a SHA256 /path_to_directory/
```

Generates SHA1 and SHA256 only.

```
cargo run -p xmzhash -- -a SHA256 /path_to_directory/ --case MyCase
```

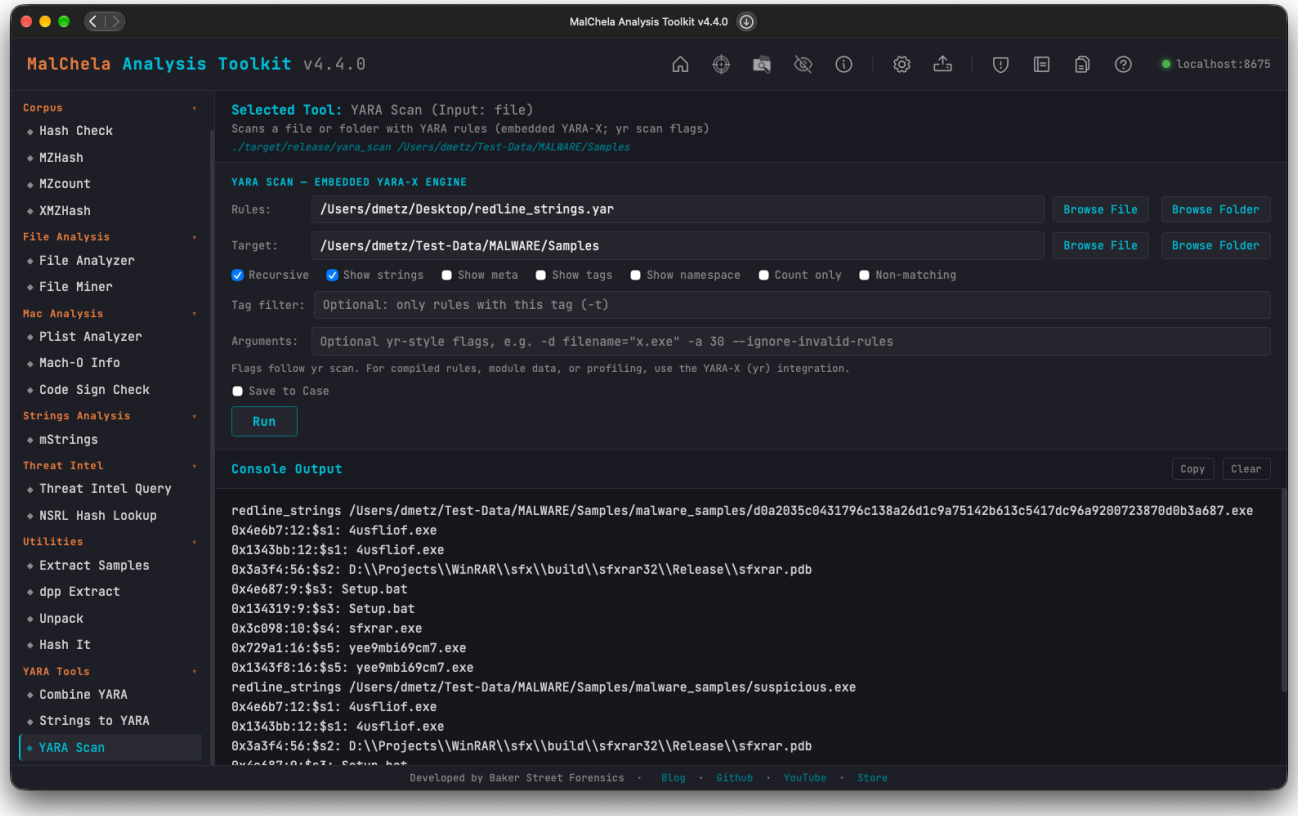
Saves results to the specified case folder.

You can combine multiple `-a` flags in any order. If no directory is passed, you will be prompted.

7.19 YARA Scan

YARA Scan runs YARA rules against a file or folder on MalChela's embedded YARA-X engine, the same engine the other MalChela tools use. Nothing extra to install. Its command line follows VirusTotal's `yr scan`: the same arguments, flag names, and output formats, so a command written for one runs on the other.

YARA Scan covers the options used in day-to-day scanning. For everything else, such as compiled rules or module data, install `yr` and use the YARA-X integration.



YARA Scan

🔧 CLI Syntax

```
./target/release/yara_scan [OPTIONS] [NAMESPACE:]RULES_PATH... TARGET_PATH
```

- `RULES_PATH` is a `.yar` / `.yara` file or a folder. Folders are walked recursively for `.yar` and `.yara` files. Give more than one to combine rule sets, and prefix one with `NAME:` to put its rules in that namespace (`mal:rules/`).
- `TARGET_PATH` is the file or folder to scan. Without `-r`, only the files directly in a folder are scanned.

```
# Scan a folder of samples, recursively, showing matching strings and tags
./target/release/yara_scan -r -s -g rules/ samples/

# Two rule sets in their own namespaces, with an external variable
./target/release/yara_scan -e -d filename=\"invoice.exe\" mal:rules/malware/ pup:rules/pup/ sample.exe

# Machine-readable output
./target/release/yara_scan -r -o ndjson -s -m rules/ samples/

# Save a Markdown report into a case
./target/release/yara_scan -r -s --case Acme --save rules/ samples/
```

Supported `yr scan` Options

Option	Description
`-c`, `--count`	Print only the number of matching rules per file
`-d`, `--define VAR=VALUE`	Define an external variable. `VALUE` is an integer, float, `true` / `false`, or a quoted string (`-d name=\"x\"`)
`--disable-console-logs`	Accepted for compatibility. `console` module output is never printed
`-w`, `--disable-warnings`	Don't print compiler warnings
`-e`, `--print-namespace`	Print each rule's namespace
`-g`, `--print-tags`	Print each rule's tags
`--ignore-invalid-rules`	Skip rule files that fail to compile, and report them as warnings
`-I`, `--ignore-module MODULE`	Ignore rules that use this module
`--include-dir PATH`	Extra directory to search for `include`d files. Each rule file's own folder is always searched
`--max-matches-per-pattern N`	Cap the matches recorded per pattern
`-n`, `--negate`	Print the rules that did *not* match
`-o`, `--output-format FORMAT`	`text` (default), `ndjson` (one JSON object per file), or `json` (one document)
`--path-as-namespace`	Use each rule file's path as its namespace
`-m`, `--print-meta`	Print rule metadata
`-s`, `--print-strings[=N]`	Print each matching string with its offset and length, limited to `N` characters (default 120)
`-r`, `--recursive[=MAX_DEPTH]`	Scan subfolders of the target, optionally limited in depth (`-r=2`)
`--relaxed-re-syntax`	Accepted for compatibility. Relaxed regular-expression parsing is always on, so rules written for libyara compile
`--scan-list`	`TARGET_PATH` is a text file listing the paths to scan, one per line
`-z`, `--skip-larger SIZE`	Skip files larger than `SIZE` bytes
`-t`, `--tag TAG`	Report only rules tagged `TAG`
`-p`, `--threads N`	Number of scanning threads (default: one per core)
`-a`, `--timeout SECONDS`	Abort a file's scan after this many seconds

`-r` and `-s` take their optional value with `=` (`-r=2`, `-s=40`), so `-r rules/ samples/` never reads `rules/` as a depth. `yr` works the same way.

Not Supported: Use `yr`

These `yr scan` options are refused with a message pointing to `yr`, never silently ignored:

- `-C`, `--compiled-rules`
- `-x`, `--module-data`
- `-f`, `--fast-scan`
- `--profiling`
- `--cpu-limit`
- `--no-mmap`

`yr`'s other subcommands, including `check`, `compile`, `dump`, `fmt`, and `debug`, are not part of YARA Scan either.

MalChela Additions

`yr` already uses `-o`, `-t`, and `-m`, so YARA Scan's MalChela options are long-only:

Option	Description
`--save[=md|txt|json]`	Save a report: Markdown by default, the text output, or the JSON document
`--case NAME`	Save reports to `saved_output/cases/NAME/yara_scan/` and register them with the case

The Markdown report lists the rules and target, a summary table (file, rule, namespace, tags, and match count), and the matching strings for each file.

Differences from `yr`

- **File order.** `yr` prints each file as soon as its scan finishes, so the order changes from run to run. YARA Scan sorts its output by path.
- **Rule order within a file.** Both tools list a file's rules in the order they were compiled. `yr` compiles a rules folder in directory-listing order, and YARA Scan compiles it in sorted path order, so the same rules can appear in a different order. The results themselves are identical.
- **Regular expressions.** Relaxed parsing is always on. A rule that `yr` rejects under its default strict parsing may compile here, but not the other way around.
- **JSON `version`.** The `-o json` document's `version` field names MalChela's YARA Scan rather than `yr`.

Web Interface

YARA Tools → YARA Scan has a simplified form: a rules file or folder, a target file or folder, checkboxes for the common options, a tag filter, and an **Arguments** field for any other supported flag. Choose **Save to Case** to write a report to the active case.

MCP

The `yara_scan` MCP tool runs the same scan and returns NDJSON with each rule's namespace, tags, metadata, and matching strings. When a case is active, it also saves a Markdown report there.

8. Mac Analysis

8.1 Overview

While many MalChela tools are file-type agnostic — Hash It, Threat Intel Query, File Miner, and others work the same regardless of what platform a sample targets — the tools on this page have been specifically implemented for the analysis of macOS malware: Mach-O binaries, `.app` bundles, code signing, property lists, and Apple's DMG/PKG installer formats.

8.1.1 Mac Analysis Tools

Dedicated tools for static analysis of macOS binaries, bundles, and property lists.

Program	Function
Analyze	One-click auto-triage for a file, folder, or .app bundle — dispatches every relevant tool, including the Mac Analysis stack, and produces a combined rollup report
Code Sign Check	Inspects macOS code signing: Developer-signed vs. ad-hoc vs. unsigned, Team ID, Bundle ID, entitlements, and get-task-allow flag
dpp Extract	Unwraps a .dmg or .pkg (UDIF → HFS+/APFS → XAR → PBZX/CPIO) to reach the real payload files inside
Mach-O Info	Parses Mach-O binaries: architecture, linked libraries, section entropy, symbol status, RPATH entries, and deprecated crypto library detection
mStrings	Extracts strings, IOCs, and MITRE ATT&CK matches from Mach-O binaries and .app bundles
Plist Analyzer	Parses .plist files and .app bundle Info.plist for malware indicators: hidden background agent, ATS disabled, custom URL schemes, env injection
Unpack	Splits fat (universal) Mach-O binaries into their architectures and unpacks UPX-packed Mach-O, recursing into each layer; never executes the sample

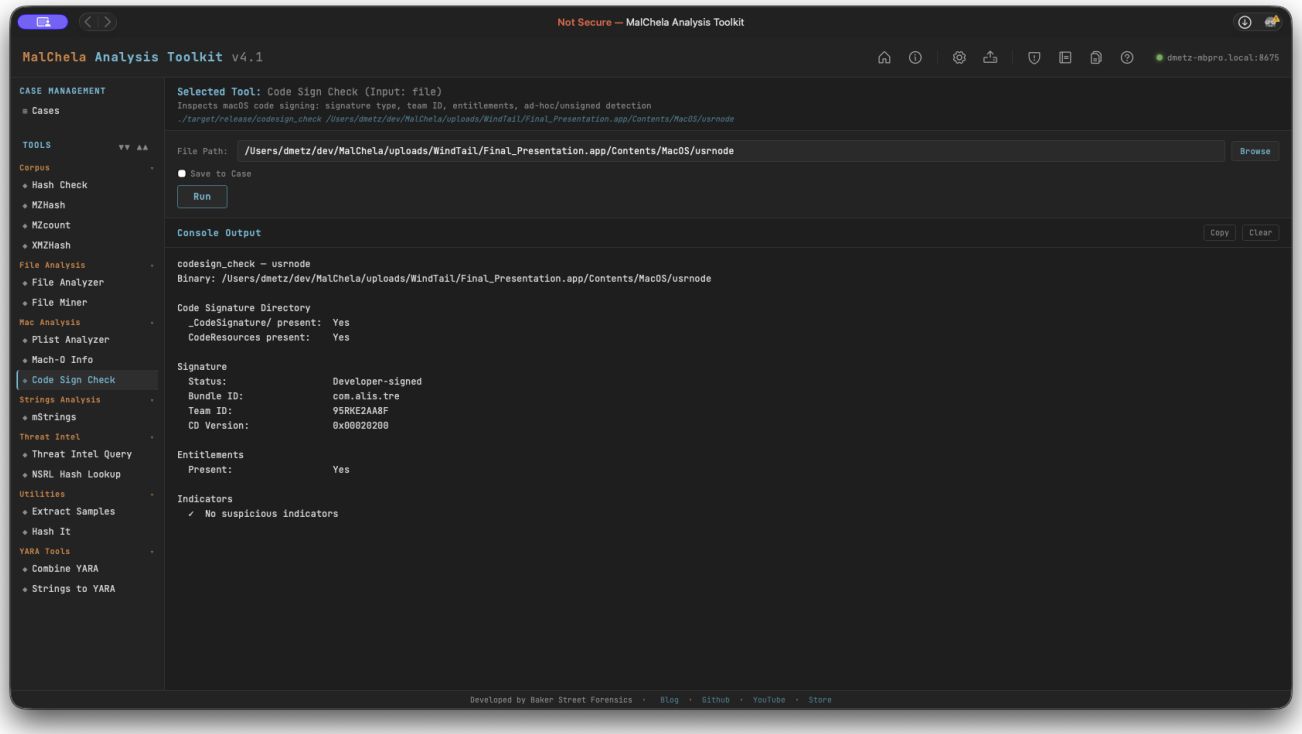
Analyze, mStrings, and Unpack work across platforms, so their full pages live under Core Tools: Analyze, mStrings, and Unpack.

Code Sign Check, Mach-O Info, mStrings, and Plist Analyzer auto-resolve a .app bundle's main executable — point them at the bundle directly. Run those four together against a bundle or binary with `./mac_stack.sh`.

8.2 Code Sign Check

Code Sign Check inspects macOS code signing by parsing the code signature superblob directly from the Mach-O binary — no dependency on system `codesign` tooling. It identifies whether a binary is Developer-signed, ad-hoc signed, or completely unsigned, and extracts the Team ID, Bundle ID, entitlements, and CodeDirectory version.

Accepts either a `.app` bundle path (resolves the binary via `Info.plist`) or a direct path to a Mach-O binary. Handles fat/universal binaries.



Code Sign Check

What It Checks

Code Signature Directory - Presence of `_CodeSignature/` directory within the bundle - Presence of `CodeResources` (bundle resource seal)

Signature Status - Developer-signed — CMS blob present, certificate chain embedded - **Ad-hoc** — `CS_ADHOC` flag set in CodeDirectory and/or no CMS blob; self-signed, not from a developer account - **Unsigned** — no `_CodeSignature/` directory and no valid superblob

Extracted Fields - Bundle ID (from CodeDirectory `identOffset`) - Team ID (from CodeDirectory `teamIDOffset`, version ≥ 0x20200) - CodeDirectory version and flags - Entitlements presence - `get-task-allow` entitlement (marks a debug/development build — not App Store or notarized)

Indicators Flagged

Indicator	Significance
No `_CodeSignature/`	Binary is unsigned
No CMS blob	Ad-hoc signature — not issued by a developer account
`CS_ADHOC` flag set	Ad-hoc signing confirmed in CodeDirectory flags
No Team ID	Ad-hoc, self-signed, or very old signing format
`get-task-allow`	Debug build — allows task port access; not notarized

Note: By default, Code Sign Check reports what is embedded in the binary and nothing more — a Developer-signed result means a real certificate was used at signing time, not that it's still valid. Pass `--check-revocation` (see below) for a live OCSP check against Apple's revocation infrastructure.

Revocation (OCSP) Check

`--check-revocation` queries Apple's OCSP responder to check whether the Developer ID certificate that signed the binary has since been revoked — a strong, independent signal for malware triage: Apple actively revokes certificates tied to known-malicious signers once discovered, so a `REVOKED` result on an otherwise unremarkable binary is itself a finding.

This is opt-in and off by default because it requires live network access — it does **not** run as part of a plain Code Sign Check invocation.

How it works: the certificate chain is extracted from the Mach-O's embedded CMS blob and handed to the system `openssl` client (`pkcs7` / `x509` / `ocsp`), which performs the actual query. Apple's codesign tool emits that CMS blob as BER with indefinite-length encoding — valid BER, not valid DER — so this deliberately uses openssl's BER-tolerant ASN.1 parser rather than a strict-DER library, which would reject the blob outright. Leaf vs. issuer certificate is identified generically (the leaf is whichever certificate in the embedded chain wasn't used to issue any other certificate in the set), so it isn't tied to a fixed chain length.

Possible statuses:

Status	Meaning
`GOOD`	Certificate is valid and not revoked, as of the query
`REVOKED`	Apple has revoked this certificate — shown with the reason and revocation date from the OCSP response
`UNKNOWN`	The responder has no record of this certificate
`Not checked`	Query wasn't attempted or couldn't complete — reason given (offline mode, ad-hoc/unsigned binary, no OCSP URL in the certificate — common for pre-~2016 Developer ID certs, network/openssl failure, etc.)

A `REVOKED` result also surfaces as a warning in the **Indicators** section and in saved reports.

Offline Mode: this check honors Offline Mode directly (`MALCHELA_OFFLINE`) — it self-skips with a clean "Not checked (offline mode)" result rather than attempting the network call, the same convention every other network-touching MalChela tool follows. This is checked inside `codesign_check` itself, not just by callers like Analyze, so it's honored no matter how `--check-revocation` gets invoked.

In Analyze: Analyze automatically passes `--check-revocation` for every Code Sign Check run it dispatches, scoped to the **top-level binary only** — not embedded frameworks/plugins/XPC helpers, since a single bundle can carry dozens of those (Audacity.app alone embeds 90+ signed dylibs), each of which would otherwise trigger its own OCSP query. It's skipped automatically whenever Offline Mode is on.

Not currently exposed as a toggle in the standalone PWA Code Sign Check panel or the MCP `codesign_check` tool — use the CLI flag directly, or run it through Analyze, for now.

PWA Usage

Select **Code Sign Check** from the Mac Analysis category. Enter the path to a `.app` bundle or a Mach-O binary. File Miner will suggest Code Sign Check for any file it identifies as `x-mach-binary`.

🔧 CLI Syntax

```
# Check a .app bundle
cargo run -p codesign_check -- /path/to/Sample.app

# Check a Mach-O binary directly
cargo run -p codesign_check -- /path/to/binary

# Also check OCSP revocation status (requires network access and openssl on PATH)
cargo run -p codesign_check -- /path/to/binary --check-revocation

# Save output as Markdown to a case folder
cargo run -p codesign_check -- /path/to/Sample.app -o -m --case CaseXYZ
```

Use `-o` to save output and include one of the following format flags: - `-t` → Save as `.txt` - `-m` → Save as `.md`

When `--case` is used, output is saved to:

```
saved_output/cases/CaseXYZ/codesign_check/
```

Otherwise, results are saved to:

```
saved_output/codesign_check/
```

8.3 dppExtract

dpp Extract unwraps Apple disk image (`.dmg`) and installer package (`.pkg`) containers — formats none of the other Core Tools can see through directly. A raw `.dmg` reads as `Unknown or unrecognized file type` to File Analyzer; a `.pkg` is a XAR archive wrapping a PBZX-compressed CPIO payload, not a Mach-O binary. dpp Extract walks the whole chain — **UDIF → HFS+/APFS → XAR → PBZX/CPIO** — and hands back the real files inside, so the rest of the suite (File Analyzer, mStrings, Mach-O Info, Code Sign Check) can analyze them normally.

It's built on the `dpp` Rust crate.

How It Works

1. Opens the `.dmg` (or `.pkg` directly) and mounts its filesystem — auto-detecting HFS+ vs. APFS, and all four DMG compression schemes Apple uses (Zlib, Bzip2, LZFSE, XZ).
2. If the filesystem contains one or more `.pkg` installers, extracts each component's **Payload** (the actual files the installer drops) and **Scripts** (preinstall/postinstall) archives — both are decoded whether they're the modern PBZX-wrapped format or the classic gzip-compressed CPIO format older installers use.
3. If there's no `.pkg` inside — just a bare app or binary sitting directly on the disk image — extracts the raw filesystem tree instead, since that already **is** the payload.
4. Extraction is entry-by-entry rather than all-or-nothing: a single unreadable or malformed entry (a handful of real-world DMGs trip a decoding quirk in the underlying HFS+/APFS crate) is skipped and reported, not treated as a fatal error for the whole volume.

> **Why Scripts matters:** some installers (seen in both objective-see's oRAT and Shlayer samples) ship an empty or near-empty Payload and put their actual malicious behavior entirely in the postinstall script instead — a curl-and-execute one-liner, or a fingerprint-and-callback sequence. Extracting Payload alone would silently miss that; dpp Extract always pulls both.

CLI Syntax

```
# Extract a DMG or PKG next to itself, in <name>_extracted/
cargo run -p dpp_extract /path/to/sample.dmg

# Extract to a specific output directory
cargo run -p dpp_extract /path/to/sample.pkg -o /path/to/output

# Save under a case
cargo run -p dpp_extract /path/to/sample.dmg --case Case123

# Machine-readable summary (used internally by Analyze and the web interface)
cargo run -p dpp_extract /path/to/sample.dmg --json
```

When `--case` is provided without `-o`, output is saved under:

```
saved_output/cases/Case123/dpp_extract/
```

Analyze Integration

Analyze auto-detects a `.dmg` / `.pkg` target and runs dpp Extract before handing the result to File Miner — there's no need to run dpp Extract by hand first. Because a trojanized installer's payload commonly bundles the full legitimate app it trojanized alongside the injected malicious file(s), the extracted tree is often well over Analyze's 25-file auto-run cap (EvilQuest: 624 files for one Mach-O of actual interest); when that happens, Analyze hands off to File Miner's interactive table instead of erroring out or auto-running everything. See Analyze and File Miner for that flow.

Known Limitations

dpp Extract inherits the maturity of the underlying `dpp` crate. Two failure modes have been observed against real-world samples and are not currently recoverable:

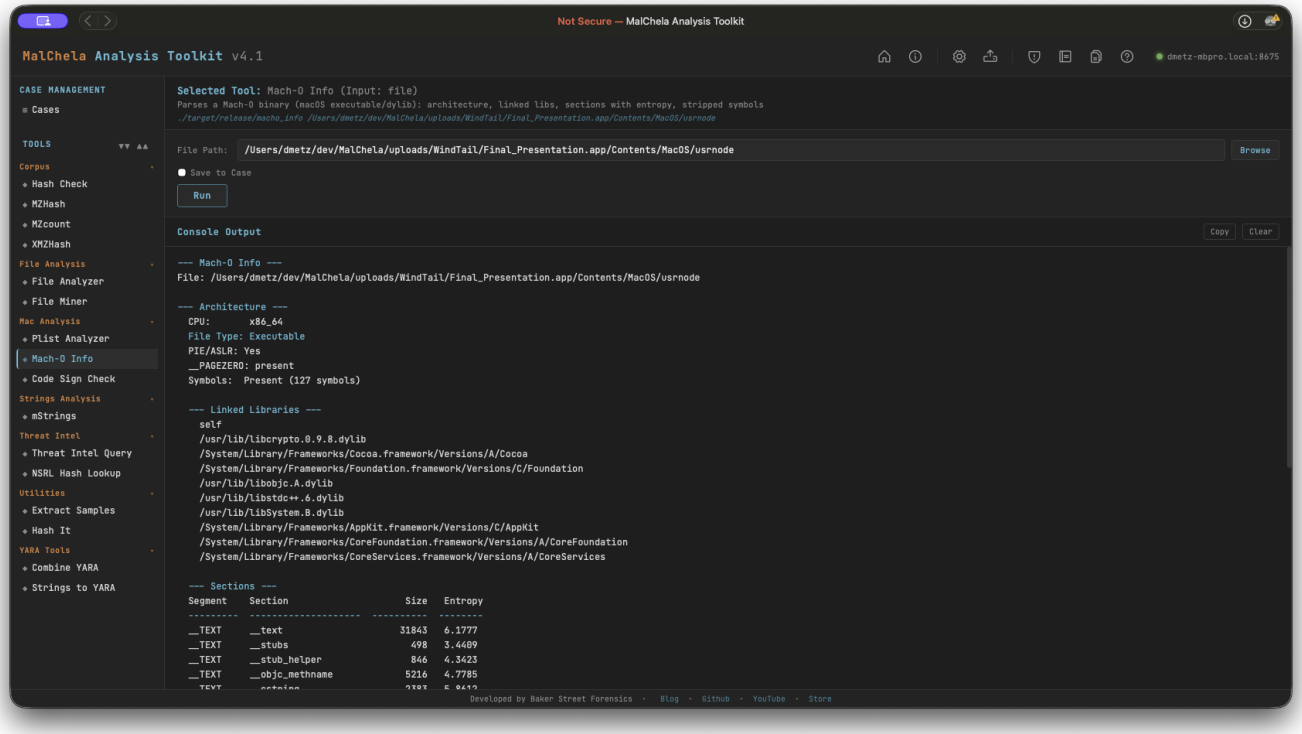
- A `.dmg` that isn't a standard UDIF file (missing the expected `koly` trailer) fails to open at all.
- A small number of DMGs have hit what looks like a broken by-name catalog lookup in the HFS+ decoder — the volume opens and walks fine, but every file read fails, extracting zero files.

Both fail safely (a clear error, or a zero-file result with every skipped entry listed) rather than crashing or silently corrupting output.

8.4 Mach-O Info

Mach-O Info performs static analysis on macOS Mach-O binaries, including both thin (single-architecture) and fat/universal binaries. It reports architecture details, linked libraries, RPATH entries, per-section entropy, and symbol table status. Suspicious indicators are flagged automatically in the Indicators section.

Accepts either a `.app` bundle path or a direct path to a Mach-O binary. When given a bundle, the main executable is auto-resolved via `Info.plist`'s `CFBundleExecutable`, falling back to the sole binary in `Contents/MacOS/` if that lookup fails.



Mach-O Info

Analysis Sections

Architecture - CPU type and subtype (x86_64, arm64, arm64e) - File type (Executable, Dynamic library, Bundle, etc.) - PIE/ASLR status (`MH_PIE` flag) - `__PAGEZERO` presence and size (zero-sized = privilege escalation risk) - Symbol table: present count or stripped

Linked Libraries All dylibs declared in load commands, including system frameworks and third-party libraries.

RPATH Entries Relative paths searched for dylib resolution — a vector for dylib hijacking if writable.

Sections with Entropy Per-section entropy table. Sections above 7.0 are flagged as potentially packed or encrypted.

Indicators Flagged

Indicator	Significance
Stripped symbol table	Adversarial hardening — hinders analysis
Zero-sized `__PAGEZERO`	Classic privilege escalation technique in older macOS malware
High-entropy section (> 7.0)	Possible packed or encrypted payload
RPATH entries	Potential dylib hijacking surface
Imports `system()`, `popen()`, `exec*()`, or `posix_spawn()`	Can launch a shell or other programs (osascript, curl, …) with arguments built at runtime — common in stealers that hide their commands as encrypted strings
Deprecated crypto library	`libcrypto.0.9.8` / `libssl.0.9.8` — EOL OpenSSL with numerous known CVEs
C2 dylib triad	CoreFoundation + SystemConfiguration + Security with minimal other imports — common implant pattern

PWA Usage

Select **Mach-O Info** from the Mac Analysis category and provide the path to a `.app` bundle or a Mach-O binary. File Miner will suggest Mach-O Info for any file it identifies as `x-mach-binary`.

🔧 CLI Syntax

```
# Analyze a Mach-O binary
cargo run -p macho_info -- /path/to/binary

# Analyze a .app bundle (main executable resolved automatically)
cargo run -p macho_info -- /path/to/Sample.app

# Save output as Markdown to a case folder
cargo run -p macho_info -- /path/to/binary -o -m --case CaseXYZ
```

Use `-o` to save output and include one of the following format flags: - `-t` → Save as `.txt` - `-j` → Save as `.json` - `-m` → Save as `.md`

When `--case` is used, output is saved to:

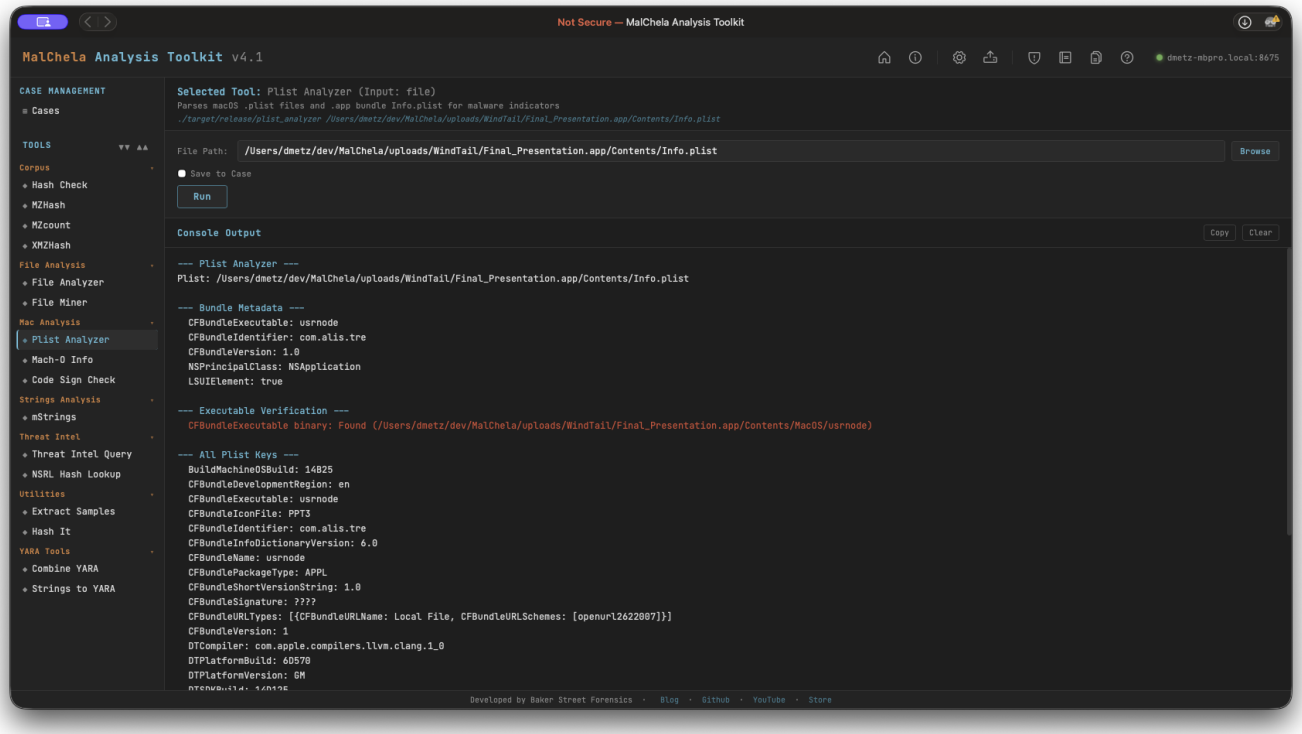
```
saved_output/cases/CaseXYZ/macho_info/
```

Otherwise, results are saved to:

```
saved_output/macho_info/
```

8.5 Plist Analyzer

Plist Analyzer parses macOS `.plist` files and `.app` bundle `Info.plist` for static malware indicators. It extracts bundle metadata, verifies the declared executable is present, and flags suspicious key combinations commonly found in macOS malware — including background-only agents, disabled App Transport Security, custom URL scheme registration, and environment variable injection.



Plist Analyzer

Indicators Checked

Indicator	Key	Why It Matters
Hidden background agent	`LSUIElement` / `NSUIElement` = true	App runs with no Dock icon — common stealth technique
ATS disabled	`NSAllowsArbitraryLoads` = true	Allows unencrypted HTTP connections — classic C2 channel
Custom URL scheme	`CFBundleURLTypes`	Non-standard URL handler registration — used for persistence or IPC
No creator code	`CFBundleSignature` = `????`	Missing creator code — common in unsigned tools and malware
Env var injection	`LSEnvironment` present	Malware may inject environment variables at launch
Missing executable	`CFBundleExecutable` not in `Contents/MacOS/`	Bundle metadata declares a binary that doesn't exist
Extra binaries	Additional files in `Contents/MacOS/`	Undeclared executables alongside the declared one

PWA Usage

Select **Plist Analyzer** from the Mac Analysis category. Enter the path to: - A `.plist` file directly, or - A `.app` bundle directory — the tool will automatically locate `Contents/Info.plist`

File Miner will suggest Plist Analyzer for `.plist` files it encounters during a folder scan.

🔧 CLI Syntax

```
# Analyze a .plist file
cargo run -p plist_analyzer -- /path/to/Info.plist

# Analyze a .app bundle (Info.plist resolved automatically)
cargo run -p plist_analyzer -- /path/to/Sample.app

# Save output as Markdown to a case folder
cargo run -p plist_analyzer -- /path/to/Info.plist -o -m --case CaseXYZ
```

Use `-o` to save output and include one of the following format flags: - `-t` → Save as `.txt` - `-j` → Save as `.json` - `-m` → Save as `.md`

When `--case` is used, output is saved to:

```
saved_output/cases/CaseXYZ/plist_analyzer/
```

Otherwise, results are saved to:

```
saved_output/plist_analyzer/
```

9. Third-Party Tools

9.1 Integrating Third-Party Tools

MalChela supports the integration of external tools such as Python-based utilities (`oletools` , `oledump`) and high-performance YARA engines (`yara-x`). These tools expand MalChela's capabilities beyond its native Rust-based toolset.

Tools now require `exec_type` (e.g., `cargo` , `binary` , `script`) to define how they are launched, and `file_position` to clarify argument order when needed.

To integrate a new tool into the web interface, ensure the tool: - Accepts CLI arguments in the form `toolname [args] [input]` - Outputs results to stdout - Is installed and available in `$PATH`

```
- name: toolname
  description: "Short summary of tool purpose"
  command: ["toolname"]
  input_type: file  # or folder or hash
  category: "File Analysis"  # or other web interface category
  optional_args: []
  exec_type: binary  # or cargo / script
  file_position: last  # or first, if required
```

You can switch to a prebuilt `tools.yaml` for REMnux mode via the web interface configuration panel — useful for quick setup in forensic VMs.

9.2 Configuration Reference

9.2.1 Tool Configuration

MalChela uses a central `tools.yaml` file to define which tools appear in the web interface, along with their launch method, input types, categories, and optional arguments. This YAML-driven approach allows full control without editing source code.

Key Fields in Each Tool Entry

Field	Purpose
name	Internal and display name of the tool
description	Shown in web interface for clarity
command	How the tool is launched (binary path or interpreter)
exec_type	One of `cargo`, `binary`, or `script`
input_type	One of `file`, `folder`, or `hash`
file_position	Controls argument ordering
optional_args	Additional CLI arguments passed to the tool
category	Grouping used in the web interface left panel

⚠️ All fields except `optional_args` are required.

9.2.2 Swapping Configs: REMnux Mode and Beyond

MalChela supports easy switching between tool configurations via the web interface.



YAML Config Tool

To switch:

- Open the **Configuration Panel**
- Use **"Select tools.yaml"** to point to a different config
- Restart the web interface or reload tools

This allows forensic VMs like REMnux to use a tailored toolset while keeping your default config untouched.

A bundled `tools_remnux.yaml` is included in the repo for convenience.

KEY TIPS

- Always use `file_position: "last"` unless the tool expects input before the script
- For scripts requiring Python, keep the script path in `optional_args[0]`
- For tools installed via `pipx`, reference the binary path directly in `command`

9.2.3 Backing Up and Restoring tool.yaml

The MalChela web interface provides built-in functionality to back up and restore your `tools.yaml` configuration file.

Backup

To create a backup of your current `tools.yaml`:

- Open the **Configuration Panel**
- Click the **“Back Up Config”** button
- A timestamped copy of `tools.yaml` will be saved to the default location

You’ll see a confirmation message when the operation completes successfully.

Restore

To restore from a previous backup:

- Click the **“Restore Config”** button in the Configuration Panel
- Select a previously saved backup file
- The selected file will overwrite the current configuration

This feature makes it easy to experiment with custom tool setups while retaining a safety net for recovery.

9.3 Enhanced Integrations

Enhanced configurations have been preconfigured for several third-party tools such as TShark and Volatility, enabling streamlined integration with MalChela. These tools now support saving output directly into the active Case folder when launched from the web interface, preserving structured reports and maintaining context across sessions.

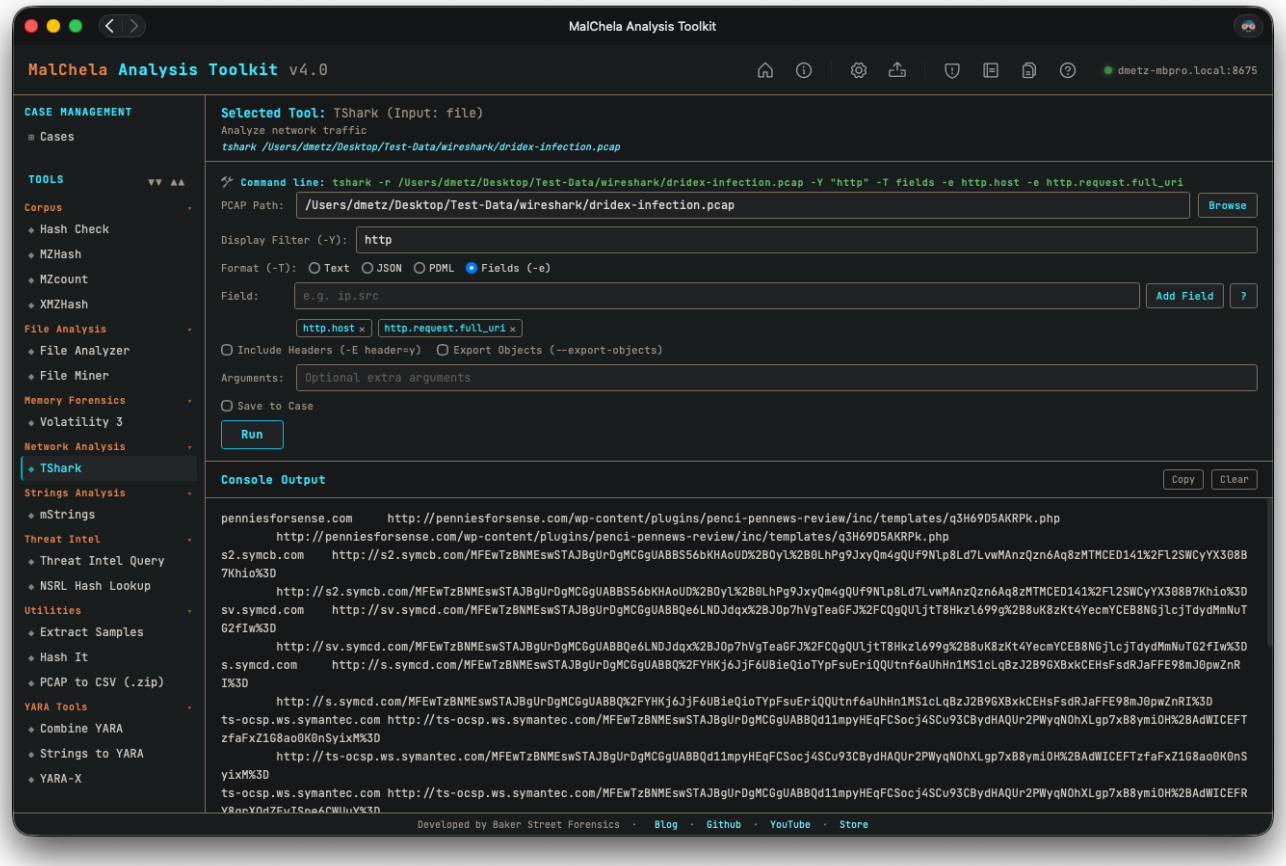
Additionally, dedicated setup instructions are provided for Python-based tools like oledump and olevba, as well as installation guidance for utilities like YARA-X to ensure consistent and reliable operation across environments.

9.4 TShark

TShark Field Reference Panel

If TShark is included in your `tools.yaml` (or if you're using the REMnux configuration), the web interface offers a powerful set of tools to assist with display filter creation and usage. This includes both an integrated filter builder and a TShark Field Reference panel.

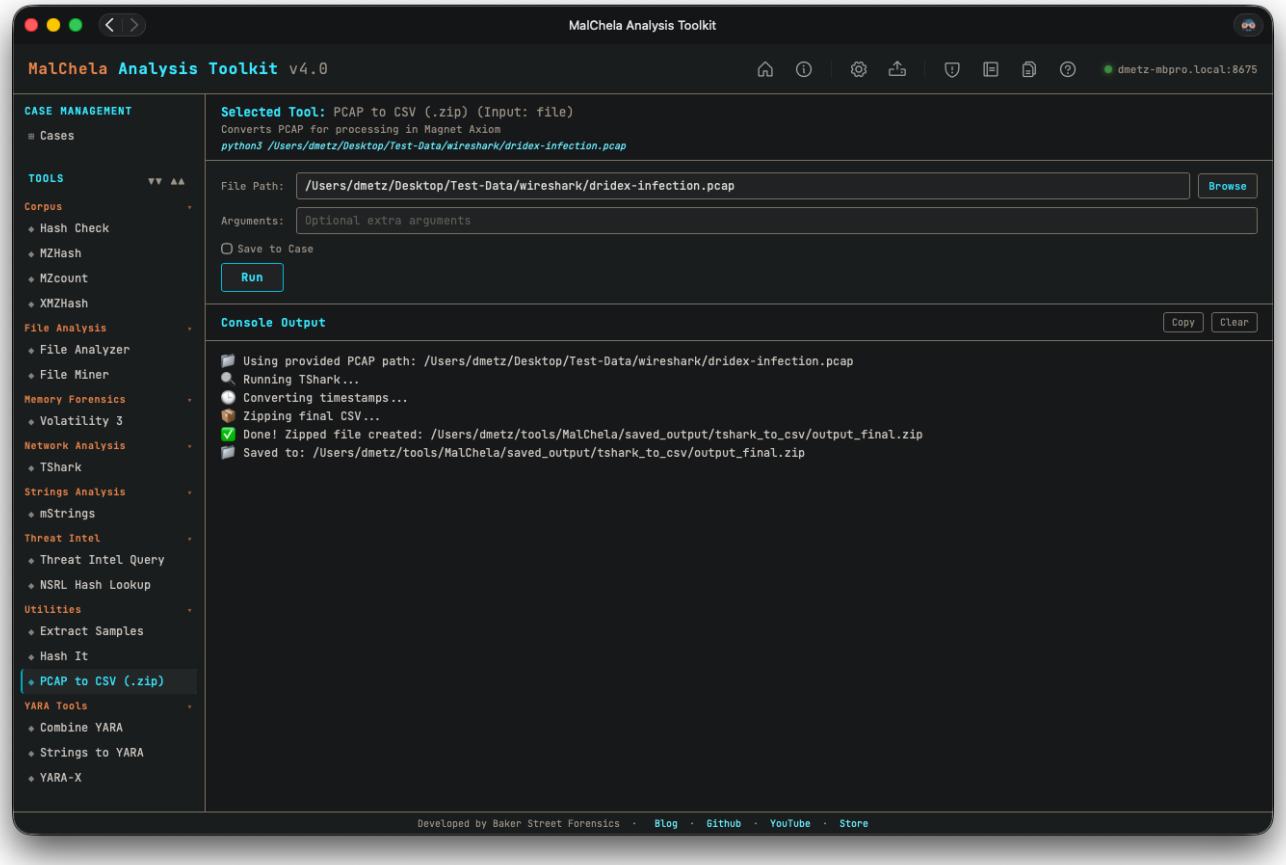
- The filter builder allows users to construct and modify complex TShark display filters directly within the web interface, with real-time syntax support and validation.
- The "?" icon next to filter fields launches the Field Reference panel, which provides searchable field definitions, examples, tooltips, and a copy-to-clipboard feature.
- Together, these tools help analysts visually explore and test filter syntax without needing to memorize protocol-specific field names.
- Output can be saved to the default location or to a specific case directory.



TShark

9.5 PCAP to CSV Conversion

The `tshark_to_csv.py` script is a helper utility that converts `.pcap` or `.pcapng` files into structured `.csv` files using `tshark`. This allows packet captures to be processed and analyzed using familiar spreadsheet or database tools.



PCAP to CSV

The script uses `tshark` to extract key fields from each packet and writes them to a CSV file with headers. Users can specify custom fields, or rely on a default set commonly useful for forensic review.

Timestamps in the CSV output are converted into a human-readable ISO 8601 format with microsecond precision (`YYYY-MM-DDTHH:MM:SS.ssssssZ`), ensuring compatibility with downstream tools such as the Magnet Custom Artifact Generator.

This script is included in the MalChela repository under `Utilities/` , and is referenced in `tools.yaml` but commented out by default. Uncomment the entry in `tools.yaml` to enable it in the web interface.

Requirements:

- Python 3
- Wireshark/tshark (must be installed and available in PATH)

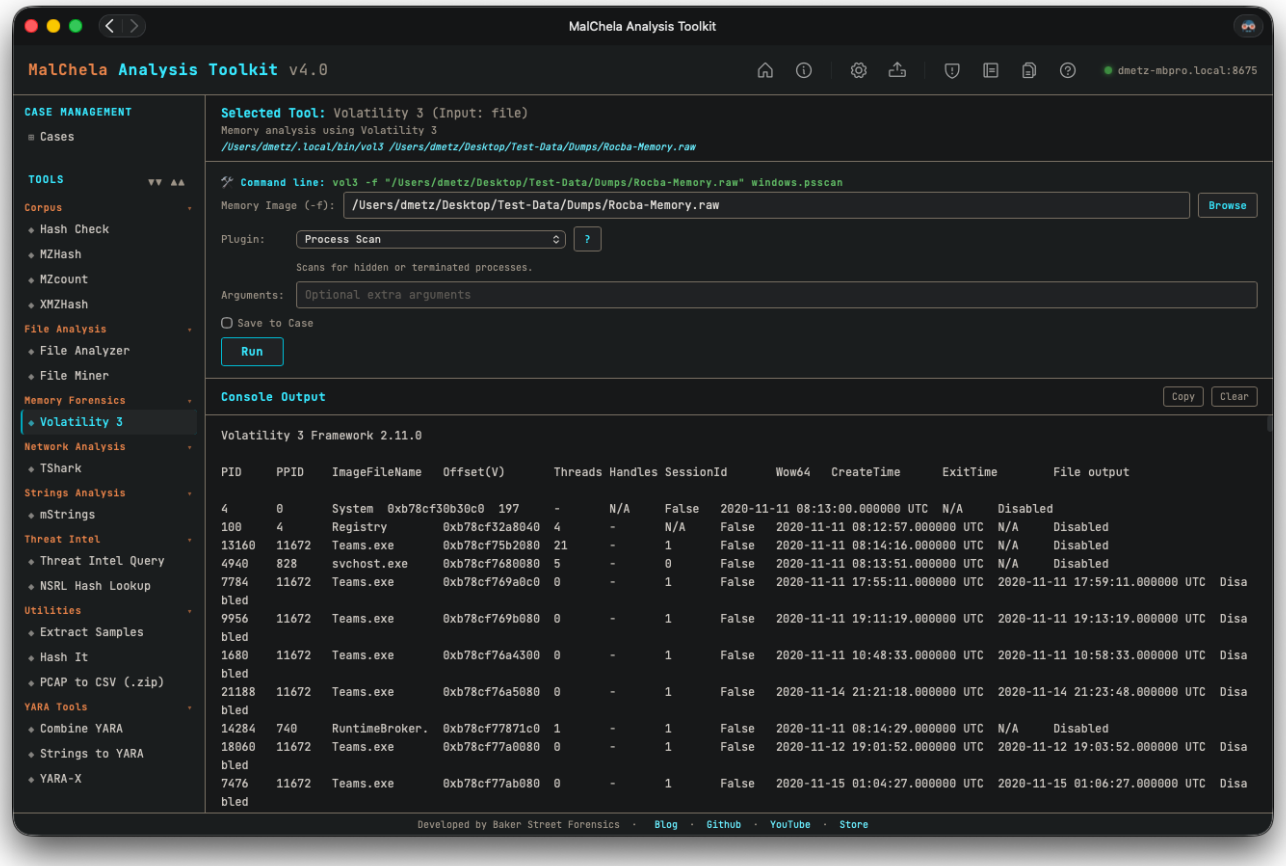
9.6 Volatility 3

MalChela integrates support for **Volatility 3**, a powerful memory forensics framework. This tool enables analysts to examine memory dumps for signs of compromise, persistence mechanisms, and malicious activity.

9.6.1 Integration Overview

Volatility 3 is available in MalChela as an enhanced third-party tool. The web interface provides a dedicated interface for selecting plugins, supplying arguments, and reviewing results — all within a structured panel that mimics the CLI workflow but adds quality-of-life improvements like:

- Live search and categorized plugin reference
- Plugin-specific argument helpers
- Color-coded output for easier review
- Output saving and file dump options
- Output can be saved to the default location or to a specific case directory.



Volatility 3



Volatility Plugin Reference

9.6.2 Requirements

To use Volatility 3 within MalChela:

- You must have `vol3` (Volatility 3 CLI) installed and accessible in your system `$PATH`.
- On REMnux, `vol3` is preinstalled and configured automatically.
- On macOS or Linux, you can install it via pip: `pip install volatility3`

9.6.3 tools.yaml Configuration

To use Volatility 3 with the web interface launcher, ensure the correct `command` value is defined in your `tools.yaml` configuration. Depending on your environment, the binary may be installed under different names or locations.

Two common examples:

```
- name: Volatility 3
  description: "Memory analysis using Volatility 3"
  command: ["/Users/dmetz/.local/bin/vol3"]
  input_type: "file"
  file_position: "first"
  category: "Memory Forensics"
  gui_mode_args: []
  exec_type: binary

- name: Volatility 3
  description: "Memory analysis using Volatility 3"
  command: ["vol3"]
  input_type: "file"
  file_position: "first"
  category: "Memory Forensics"
  gui_mode_args: []
  exec_type: script
```

Make sure that the specified binary path or command is accessible in your system's `$PATH`.

9.6.4 Example Use Cases

- Enumerate processes: `windows.pslist`
- Dump suspicious files from memory: `windows.dumpfiles --dump-dir /output/path`
- Detect injected code: `windows.malfind`
- YARA scanning on memory: `windows.vadyarascan --yara-file rules.yar`

9.6.5 Output and Reports

All plugin results are streamed to the web interface console with formatting preserved. Where supported, plugins that produce dumped files will output to a user-specified folder.

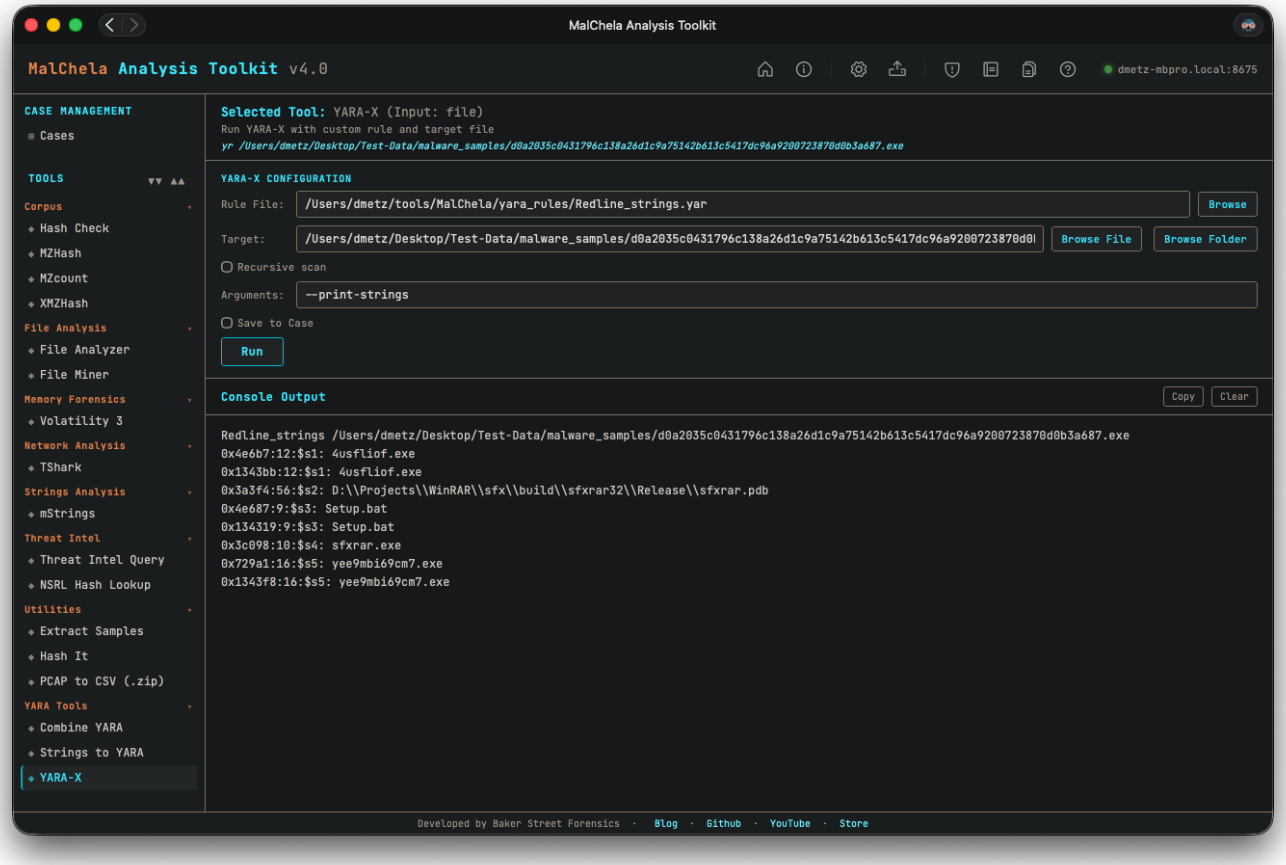
9.6.6 Known Limitations

- Some plugins require symbol files (`.pdb`) to function correctly. Volatility will display a warning if missing.
- Ensure sufficient system memory when analyzing large memory dumps.

9.6.7 Additional Resources

- Volatility 3 GitHub
- Official Documentation

9.7 Installing and Configuring YARA-X



YARA-X

YARA-X is VirusTotal's Rust rewrite of YARA. MalChela uses it in two separate ways, and only the second one needs anything installed:

- **The built-in engine.** As of v4.4, every MalChela tool that matches YARA rules has the yara-x library compiled in: YARA Scan, File Analyzer, MZHash, XMZHash, MZcount, and mismatchminer. There is nothing to install or configure. For scanning with your own rules, start with **YARA Scan**: it takes the same arguments as `yr scan` for the common options, and saves reports to a case.
- **The YARA-X integration.** An optional panel for running VirusTotal's own `yr` command-line scanner, with its full set of options. Use it for what YARA Scan doesn't cover, such as compiled rules (`-C`), module data (`-x`), `--fast-scan`, `--profiling`, or `yr`'s other subcommands. `yr` is a separate program, and building MalChela does not produce it.

9.7.1 Installing `yr`

Pick one:

- **With Cargo** (you already have Rust if you built MalChela):

  `bash`
  `cargo install yara-x-cli`

  This puts `yr` in `~/.cargo/bin`, which rustup adds to your `PATH`.
- **From a release build:** Download the archive for your platform from github.com/VirusTotal/yara-x/releases, extract it, and put `yr` in a directory on your `PATH`.
- **With Homebrew** (macOS):

  `bash`
  `brew install yara-x`

Confirm it runs:

```
yr --version
```

If `yr` is installed somewhere that isn't on the server's `PATH`, add its directory to `tool_paths` in `server_config.yaml`.

9.7.2 Enabling the Panel

The YARA-X entry is already in `tools.yaml`, commented out under **Special Tools Integrations**. Uncomment it:

```
- name: YARA-X
  description: "Run YARA-X with custom rule and target file"
  command: ["yr"]
  input_type: file
  file_position: "last"
  category: "YARA Tools"
  optional_args: []
  exec_type: binary
  cargo: false
```

Keep `command: ["yr"]` as written. The web interface recognizes `yr` and opens the YARA-X panel in place of the generic tool form: a rules path, a target file or folder, a recursive option, and extra arguments. After a restart, it appears under **YARA Tools** and is counted under **Integrations** on the home page.

9.7.3 Using YARA-X Rules

- The panel runs `yr scan [--recursive] [extra args] <rules> <target>`.
- The rules path is usually a single `.yar` or `.yara` file. `yr` also accepts a folder of rules, or you can merge a folder into one file first with **Combine YARA**.
- YARA-X is highly compatible with YARA, but it doesn't support every legacy module or construct. `yr check <rules>` reports any rules that won't compile.

9.7.4 Tips

- See the YARA-X documentation for command-line options and the differences from YARA.

9.8 Python Integrations

Configuring Python-Based Tools (oletools & oledump)

MalChela supports Python-based tools as long as they are properly declared in `tools.yaml`. Below are detailed examples and installation instructions for two commonly used utilities:

🔧 `olevba` (FROM `oletools`)

Install via `pipx`:

```
pipx install oletools
```

This installs `olevba` as a standalone CLI tool accessible in your user path.

`tools.yaml` configuration example:

```
- name: olevba
  description: "OLE document macro utility"
  command: ["/Users/youruser/.local/bin/olevba"]
  input_type: "file"
  file_position: "last"
  category: "Office Document Analysis"
  optional_args: []
  exec_type: script
```

Notes:

- `olevba` is run directly (thanks to pipx)
- No need to specify a Python interpreter in `command`
- Ensure the path to `olevba` is correct and executable

—

`oledump` (STANDALONE SCRIPT)

Manual installation:

```
mkdir -p ~/Tools/oledump
cd ~/Tools/oledump
curl -O https://raw.githubusercontent.com/DidierStevens/DidierStevensSuite/master/oledump.py
chmod +x oledump.py
```

Make sure the script path in `optional_args` is absolute, and that the file is executable if it's run directly (not through a Python interpreter in `command`).

Dependencies:

```
python3 -m pip install olefile
```

Alternatively, create a virtual environment to isolate dependencies:

```
python3 -m venv ~/venvs/oledump-env
source ~/venvs/oledump-env/bin/activate
pip install olefile
```

`tools.yaml` configuration example:

```
- name: oledump
  description: "OLE Document Dump Utility"
  command: ["/usr/local/bin/python3"]
  input_type: "file"
  file_position: "last"
  category: "Office Document Analysis"
  optional_args: ["/Users/youruser/Tools/oledump/oledump.py"]
  exec_type: script
```

Notes:

- The web interface ensures correct argument order: `python oledump.py <input_file>`
- `command` points to the Python interpreter
- `optional_args` contains the path to the script

10. REMnux Mode



MalChela includes built-in support for running in **REMnux Mode**, a configuration designed specifically for seamless operation within the REMnux malware analysis distribution.

10.1 How REMnux Mode Works

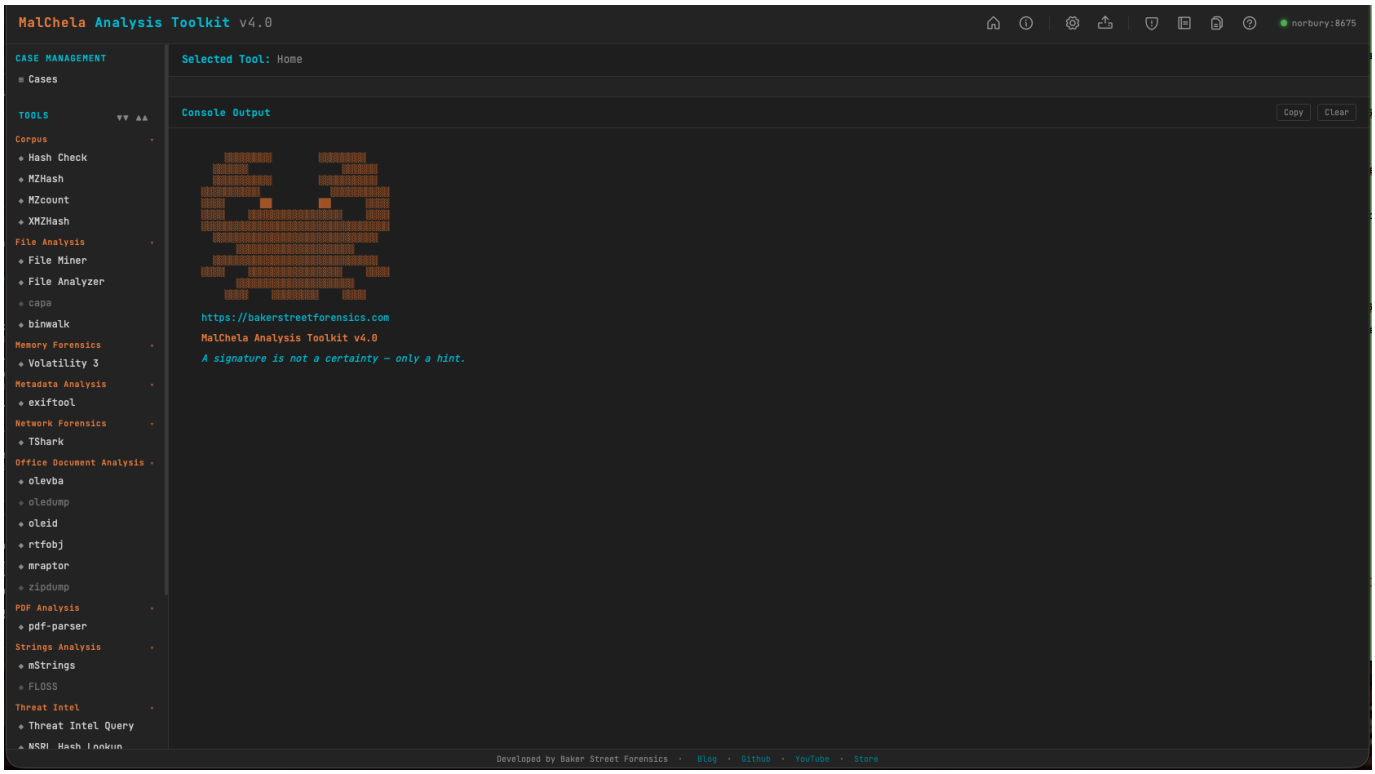
REMnux Mode is a configuration profile that aligns MalChela's behavior with the REMnux malware analysis distribution. It adjusts paths, tool definitions, and web interface presentation to match the REMnux environment.

This mode is manually enabled by selecting **"Load REMnux"** from the tools.yaml Configuration Panel in the web interface. Once selected, a REMnux-specific `tools.yaml` file is loaded and remains active until you replace it with another configuration.



Enabling REMnux mode

Note: Whenever you change the tools.yaml, you can use the "Reload Sidebar" button to update the tool listings in the sidebar.



MalChela in REMnux Mode

10.2 Preconfigured Tools in REMnux Mode

The following tools will appear in the web interface when REMnux Mode is enabled. Each is preconfigured with known-good paths for the REMnux environment:

10.2.1 File Analysis

Tool	Description	Command
binwalk	Scan binary files for embedded files	`binwalk`
capa	Detects capabilities in binaries via rules	`capa`
FLOSS	Extract obfuscated strings from binaries	`floss`

10.2.2 Memory Forensics

Tool	Description	Command
Volatility 3	Memory analysis using Volatility 3	`vol3`

10.2.3 Metadata Analysis

Tool	Description	Command
exiftool	Extract metadata from files	`exiftool`

10.2.4 Network Forensics

Tool	Description	Command
TShark	Analyze network traffic	`tshark`

10.2.5 Office Document Analysis

Tool	Description	Command
mraptor	Detect auto-executing macros in Office docs	`mraptor`
oledump	Dump streams from OLE files	`oledump.py`
oleid	Analyze OLE files for suspicious indicators	`oleid`
olevba	Extract VBA macros from OLE files	`olevba`
rtfobj	Extract embedded objects from RTF files	`rtfobj`
zipdump	Parses and analyzes suspicious PDF structures	`zipdump.py`

10.2.6 PDF Analysis

Tool	Description	Command
pdf-parser	Parse structure and objects of a PDF file	`python3 /usr/local/bin/pdf-parser.py`

10.2.7 Utilities

Tool	Description	Command
clamscan	Antivirus scan using ClamAV	`clamscan`
strings	Extracts printable strings from binary files	`strings`

10.3 Benefits

- Tools like **Volatility 3**, **FLOSS**, **oledump**, and **olevba** are preconfigured and ready to go
- `tools.yaml` is auto-tailored to the REMnux environment
- You can still customize your tool entries, but defaults are optimized for REMnux paths and permissions
- Useful for education, triage labs, and portable analysis setups

10.4 Customizing the REMnux Experience

Although REMnux Mode provides sane defaults, you can still:

- Override tool entries in `tools.yaml`
- Add new third-party tools via the web interface or YAML
- Use the Configuration Panel to backup/restore your configuration

---

For more information about REMnux, visit REMnux.org.

11. Support

11.1 Support & Contribution

The MalChela project is open source and actively maintained. Contributions, feedback, and bug reports are always welcome. You can find the project on GitHub, where issues and pull requests are encouraged.

11.2 Known Limitations & Platform Notes

MalChela is designed to be cross-platform but has some current limitations:

- The **CLI** runs well on macOS, Linux, and WSL environments.
- The **web interface** runs on any platform with a browser. Start the server with `python server/malchela_server.py` and open `http://localhost:8675`.
- File paths must use **POSIX-style formatting** (e.g., `/home/user/file.txt`). Windows-style paths are not supported.
- If the `exec_type` field is missing or misconfigured in `tools.yaml`, web interface execution may fail or behave incorrectly.
- The `category` field in `tools.yaml` no longer impacts web interface execution behavior—it is only used for grouping in the interface.